



Federal Ministry
of Research, Technology
and Space



*Forschungsinstitut
Cyber Defence
Universität der Bundeswehr München*

DECODED QUANTUM INTERFEROMETRY

FROM CODING THEORY TO QUANTUM OPTIMISATION

Simon Thelen, Wolfgang Maurer

15.09.2026

Article

Optimization by decoded quantum interferometry

<https://doi.org/10.1038/s41586-025-09527-5>

Received: 4 December 2024

Accepted: 13 August 2025

Published online: 22 October 2025

Open access

Check for updates

Stephen P. Jordan^{1,2,3}, Noah Shutty^{1,2,3}, Mary Wootters^{2,3}, Adam Zalcman¹, Alexander Schmidhuber⁴, Robbie King^{1,5}, Sergei V. Isakov¹, Tanuj Khattar⁶ & Ryan Babbush¹

Achieving superpolynomial speed-ups for optimization has long been a central goal for quantum algorithms¹. Here we introduce decoded quantum interferometry (DQI), a quantum algorithm that uses the quantum Fourier transform to reduce optimization problems to decoding problems. When approximating optimal polynomial fits over finite fields, DQI achieves a superpolynomial speed-up over known classical algorithms. The speed-up arises because the algebraic structure of the problem is reflected in the decoding problem, which can be solved efficiently. We then investigate whether this approach can achieve a speed-up for optimization problems that lack an algebraic structure but have sparse clauses. These problems reduce to decoding low-density parity-check codes, for which powerful decoders are known^{2,3}. To test this, we construct a max-XORSAT instance for which DQI finds an approximate optimum substantially faster than general-purpose classical heuristics, such as simulated annealing. Although a tailored classical solver can outperform DQI on this instance, our results establish that combining quantum Fourier transforms with powerful decoding primitives provides a promising new path towards quantum speed-ups for hard optimization problems.

NP-hardness results indicate that finding exact optima and even sufficiently good approximate optima for worst-case instances of many optimization problems is probably out of reach for polynomial-time algorithms both classical and quantum¹. Nevertheless, there remain combinatorial optimization problems, such as the closest vector problem, for which there is a large gap between the best approximation achieved by a polynomial-time classical algorithm² and the strongest complexity-theoretic inapproximability result³. When considering average-case complexity, such gaps become more prevalent, as few average-case inapproximability results are known. These gaps present a potential opportunity for quantum computers, namely achieving in polynomial time an approximation that requires superpolynomial time to achieve using known classical algorithms.

Quantum algorithms for combinatorial optimization have been the subject of intense research over the last three decades^{4–13}, which has uncovered some evidence of a possible superpolynomial quantum speed-up for certain optimization problems^{14–20}. Nevertheless, the problem of finding a superpolynomial quantum advantage for optimization is extremely challenging and remains largely open.

Here we propose a quantum algorithm for optimization that uses interference patterns as its main underlying principle. We call this algorithm decoded quantum interferometry (DQI). DQI uses a quantum Fourier transform to arrange that amplitudes interfere constructively on symbol strings for which the objective value is large, thereby enhancing the probability of obtaining good solutions upon measurement. Most previous approaches to quantum optimization

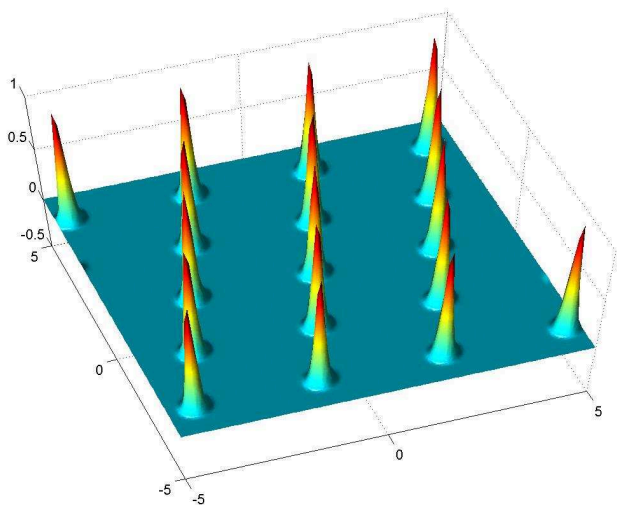
have been Hamiltonian-based⁸, with a notable exception being the superpolynomial speed-up due to Chen, Liu and Zhandry¹⁶ for finding short lattice vectors, which uses Fourier transforms and can be seen as an ancestor of DQI. Whereas Hamiltonian-based quantum optimization methods are often regarded as exploiting the local structure of the optimization landscape (for example, tunnelling across barriers²¹), our approach instead exploits the sparsity that is routinely present in the Fourier spectrum of the objective functions for combinatorial optimization problems, and it can also exploit more elaborate structure in the spectrum if present.

Before presenting evidence that DQI can efficiently obtain approximate optima not achievable by known polynomial-time classical algorithms, we quickly illustrate the essence of the DQI algorithm by applying it to max-XORSAT. We use max-XORSAT as our first example because, although it is not the problem on which DQI has achieved its greatest success, it is the context in which DQI is simplest to explain.

Given an $m \times n$ matrix B with $m > n$, the max-XORSAT problem is to find an n -bit string x satisfying as many as possible of the m linear mod-2 equations $Bx = v$. As we are working modulo 2, we regard all entries of the matrix B and the vectors x and v as coming from the finite field $\mathbb{F}_2$. The max-XORSAT problem can be rephrased as maximizing the objective function

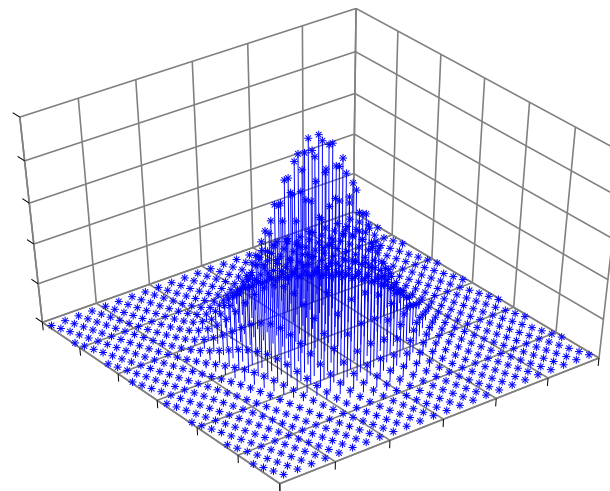
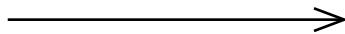
$$f(x) = \sum_{i=1}^m (-1)^{v_i + b_{ii}x_i}, \quad (1)$$

¹Google Quantum AI, Venice, CA, USA. ²Department of Computer Science, Stanford University, Stanford, CA, USA. ³Department of Electrical Engineering, Stanford University, Stanford, CA, USA. ⁴Center for Theoretical Physics, Massachusetts Institute of Technology, Cambridge, CA, USA. ⁵Department of Computing and Mathematical Sciences, Caltech, Pasadena, CA, USA. ⁶email: stephenjordan@google.com; shutty@google.com



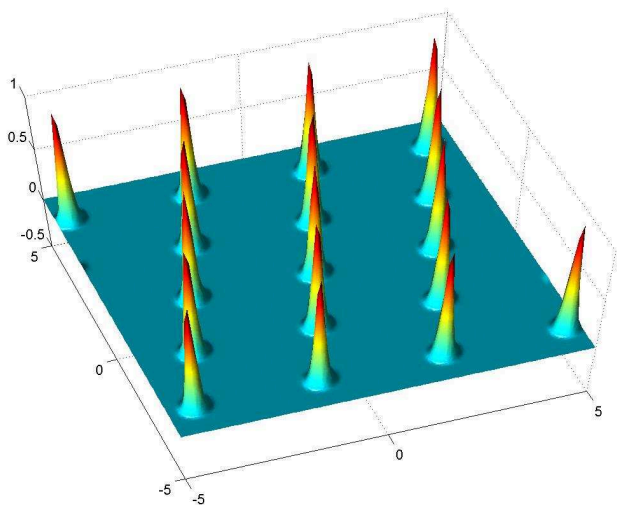
Decoding problem:
Decode to closest codeword

QFT



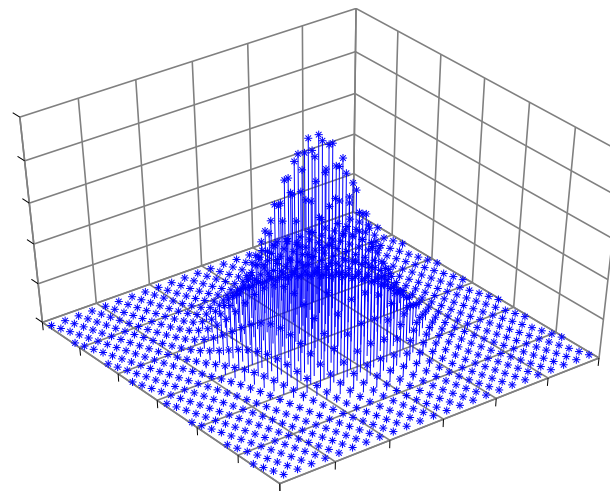
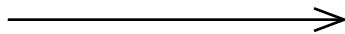
Optimisation problem:
Find solution vector close to optimum

Source: Oded Regev 2009



Decoding problem:
Decode to closest codeword

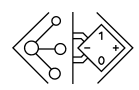
QFT



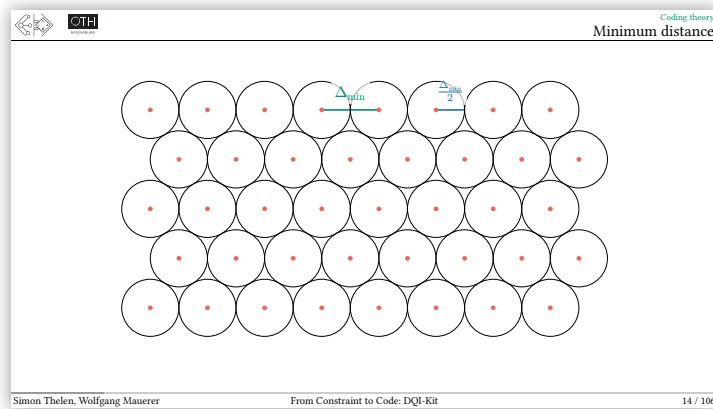
Optimisation problem:
Find solution vector close to optimum

- Potential quantum advantage
- Is analyzable

Source: Oded Regev 2009

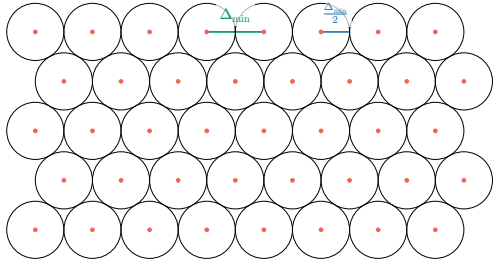


Coding Theory



Coding Theory

 **OTH** Coding theory
Minimum distance



Simon Thelen, Wolfgang Maurer From Constraint to Code: DQI-Kit 14 / 106

The DQI Algorithm

 **OTH** Decoded Quantum Interferometry
The idea of DQI

Decoded Quantum Interferometry

Idea:
Create superposition over all solutions,
better solution gets higher amplitude

Measurement: ⇒ Likely a good solution

$$\sim \sum_x f(x)^3 |x\rangle$$

|000> |001> |010> |011> |100> |101> |110> |111>



Simon Thelen, Wolfgang Maurer From Constraint to Code: DQI-Kit 40 / 106

Coding Theory

Coding theory
Minimum distance

Simon Thelen, Wolfgang Maurer

From Constraint to Code: DQI-Kit

14 / 106

The DQI Algorithm

Decoded Quantum Interferometry
The idea of DQI

Idea:
Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^3 |x\rangle$$

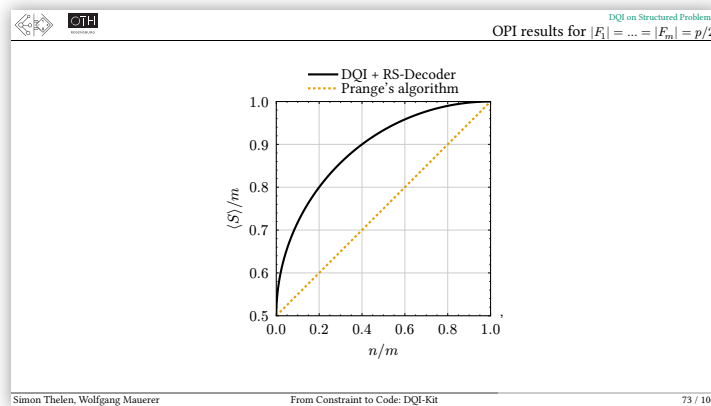
$|000\rangle$ $|001\rangle$ $|010\rangle$ $|011\rangle$ $|100\rangle$ $|101\rangle$ $|110\rangle$ $|111\rangle$

Simon Thelen, Wolfgang Maurer

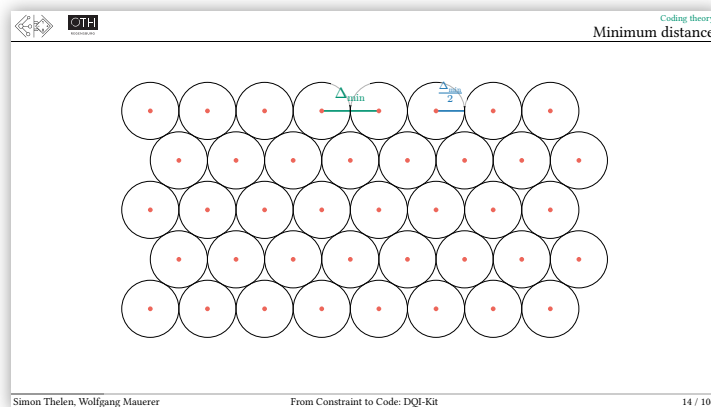
From Constraint to Code: DQI-Kit

40 / 106

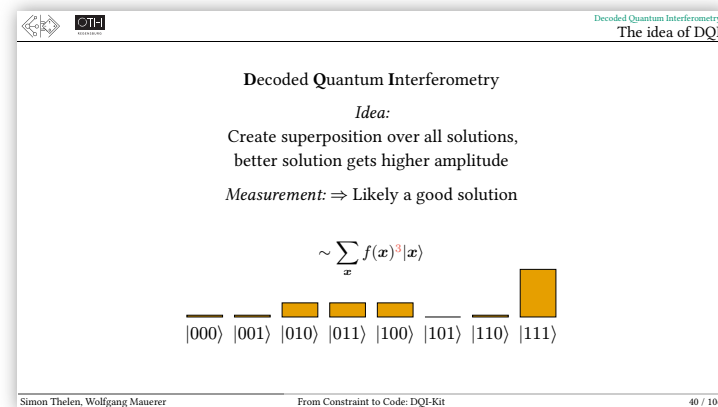
What DQI can do?



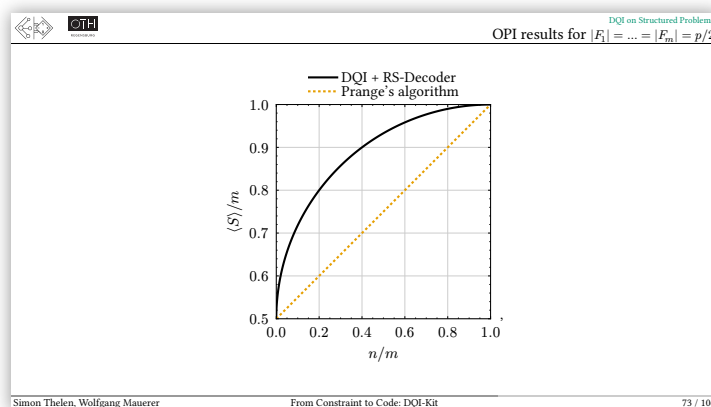
Coding Theory



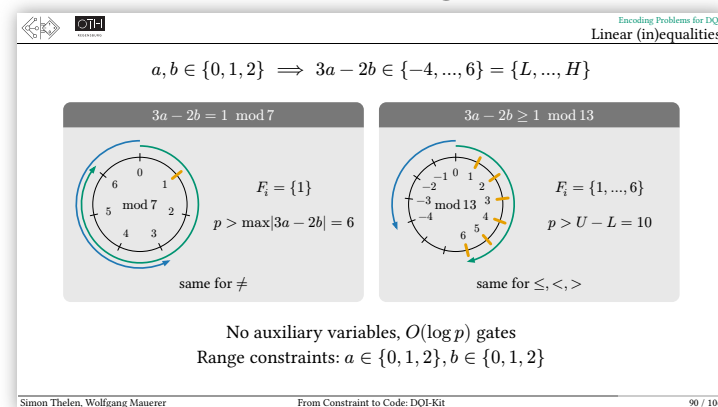
The DQI Algorithm



What DQI can do?



Problem Encodings for DQI



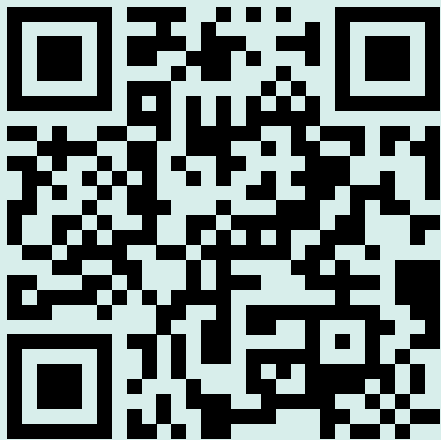
DQI-Kit



Exercise *n*

Online Jupyter Notebook

`dqi-tutorial.lfdr.de`



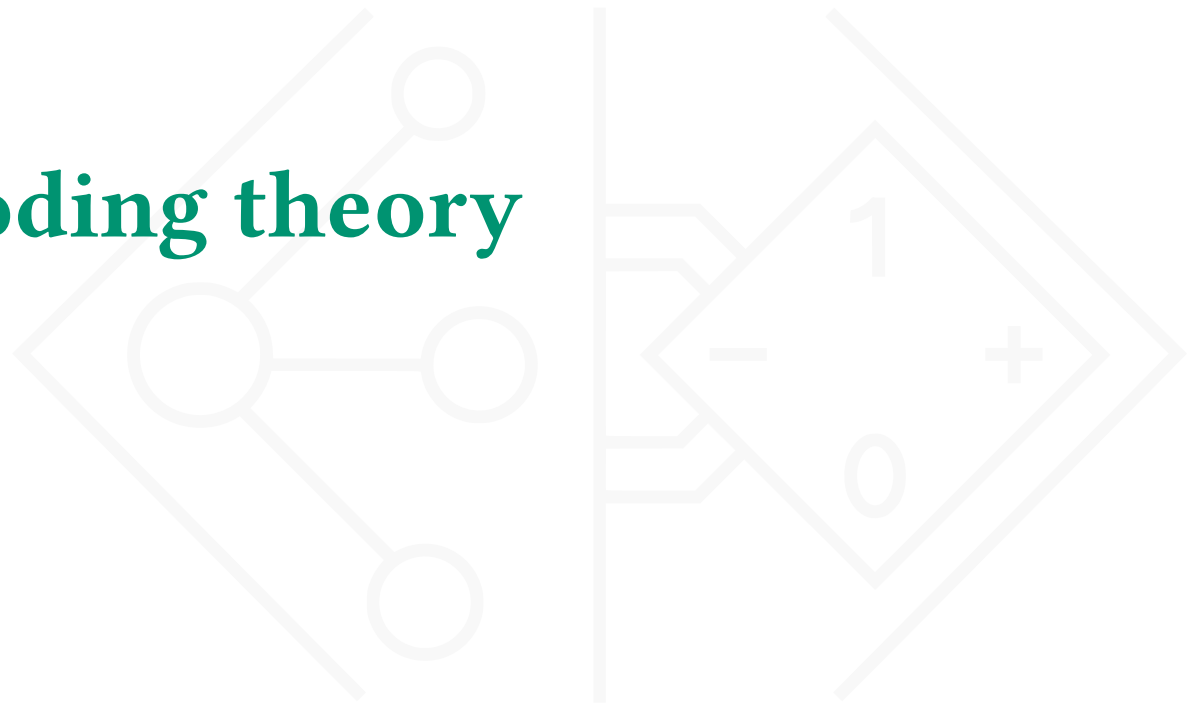
Password: `maxlinsat`

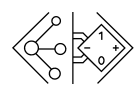
Alternative: Tutorial-Repo

`github.com/lfdr/qce2026-dqi-tutorial`



Coding theory





Sent

Received

Sent

1	1	1	0
---	---	---	---

Received

1	0	1	0
---	---	---	---

Sent

1	1	1	0
---	---	---	---

Received

1	0	1	0
---	---	---	---

1 undetected

Sent

1	1	1	0
---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

Received

1	0	1	0
---	---	---	---

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

1 undetected

Sent

1	1	1	0
---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

Received

1	0	1	0
---	---	---	---

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

1 undetected

1 detected

Sent

1	1	1	0
---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

Received

1	0	1	0
---	---	---	---

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

1 undetected

1 detected

Sent

1	1	1	0
---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

Received

1	0	1	0
---	---	---	---

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

1 undetected

1 detected

2 undetected

Sent

1	1	1	0
---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

Received

1	0	1	0
---	---	---	---

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	0	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

1 undetected

1 detected

2 undetected

Sent

1	1	1	0
---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

Received

1	0	1	0
---	---	---	---

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	0	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

1 undetected

1 detected

2 undetected

1 corrected

Sent

1	1	1	0
---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

1	1	1	0	1	1	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

Received

1	0	1	0
---	---	---	---

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

1	1	1	0	1	1	0	0	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

1	0	1	0	1	1	1	0	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

1 undetected

1 detected

2 undetected

1 corrected

Sent

1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

Received

1 0 1 0

1 1 1 1 1 1 1 0

0 1 1 0 0 1 1 0

1 1 1 0 1 1 0 0 1 1 1 0

1 0 1 0 1 1 1 0 1 1 1 1

1 undetected

1 detected

2 undetected

1 corrected

2 corrected

Sent

1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

Received

1 0 1 0

1 1 1 1 1 1 1 0

0 1 1 0 0 1 1 0

1 1 1 0 1 1 0 0 1 1 1 0

1 0 1 0 1 1 1 0 1 1 1 1

1 1 1 0 1 0 1 0 1 0 1 0

1 undetected

1 detected

2 undetected

1 corrected

2 corrected

Sent

1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

Received

1 0 1 0

1 1 1 1 1 1 1 0

0 1 1 0 0 1 1 0

1 1 1 0 1 1 0 0 1 1 1 0

1 0 1 0 1 1 1 0 1 1 1 1

1 1 1 0 1 0 1 0 1 0 1 0

1 undetected

1 detected

2 undetected

1 corrected

2 corrected

2 detected

Sent

1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

1 1 1 0 1 1 1 0 1 1 1 0

Received

1 0 1 0

1 1 1 1 1 1 1 0

0 1 1 0 0 1 1 0

1 1 1 0 1 1 0 0 1 1 1 0

1 0 1 0 1 1 1 0 1 1 1 1

1 1 1 0 1 0 1 0 1 0 1 0

0 1 1 0 0 1 1 0 0 1 1 0

1 undetected

1 detected

2 undetected

1 corrected

2 corrected

2 detected

Sent	Received	
1 1 1 0	1 0 1 0	1 undetected
1 1 1 0 1 1 1 0	1 1 1 1 1 1 1 0	1 detected
1 1 1 0 1 1 1 0	0 1 1 0 0 1 1 0	2 undetected
1 1 1 0 1 1 1 0 1 1 1 0	1 1 1 0 1 1 0 0 1 1 1 0	1 corrected
1 1 1 0 1 1 1 0 1 1 1 0	1 0 1 0 1 1 1 0 1 1 1 1	2 corrected
1 1 1 0 1 1 1 0 1 1 1 0	1 1 1 0 1 0 1 0 1 0 1 0	2 detected
1 1 1 0 1 1 1 0 1 1 1 0	0 1 1 0 0 1 1 0 0 1 1 0	3 undetected

$$x_1 x_2 x_3 x_4 \mapsto x_1 x_2 x_3 x_4 p_1 p_2 p_3$$

$$p_1 = x_1 \oplus x_2 \oplus x_4$$

$$p_2 = x_1 \oplus x_3 \oplus x_4$$

$$p_3 = x_2 \oplus x_3 \oplus x_4$$

$$x_1 x_2 x_3 x_4 \mapsto x_1 x_2 x_3 x_4 p_1 p_2 p_3$$

$$p_1 = x_1 + x_2 + x_4 \bmod 2$$

$$p_2 = x_1 + x_3 + x_4 \bmod 2$$

$$p_3 = x_2 + x_3 + x_4 \bmod 2$$

$$x_1 x_2 x_3 x_4 \mapsto x_1 x_2 x_3 x_4 p_1 p_2 p_3$$

$$x_1 + x_2 + x_4 + p_1 = 0 \bmod 2$$

$$x_1 + x_3 + x_4 + p_2 = 0 \bmod 2$$

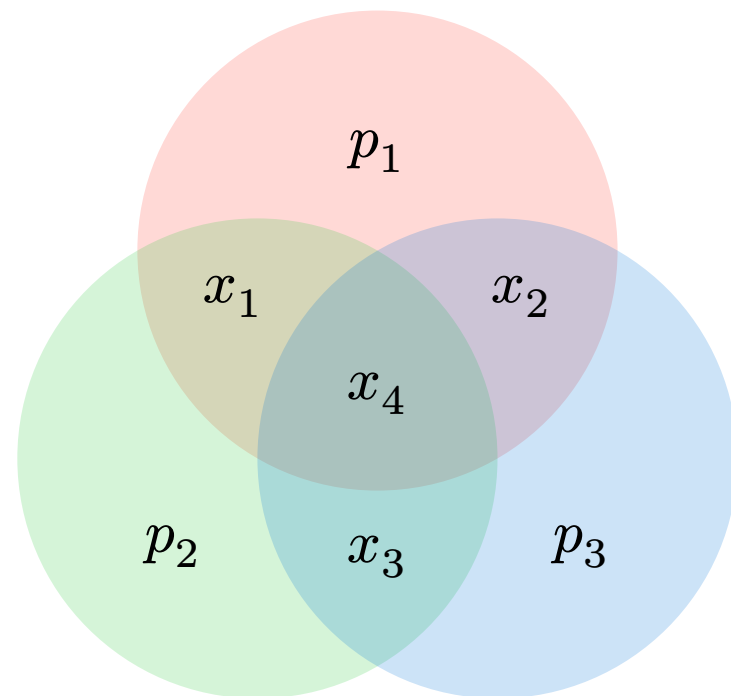
$$x_2 + x_3 + x_4 + p_3 = 0 \bmod 2$$

$$x_1 x_2 x_3 x_4 \mapsto x_1 x_2 x_3 x_4 p_1 p_2 p_3$$

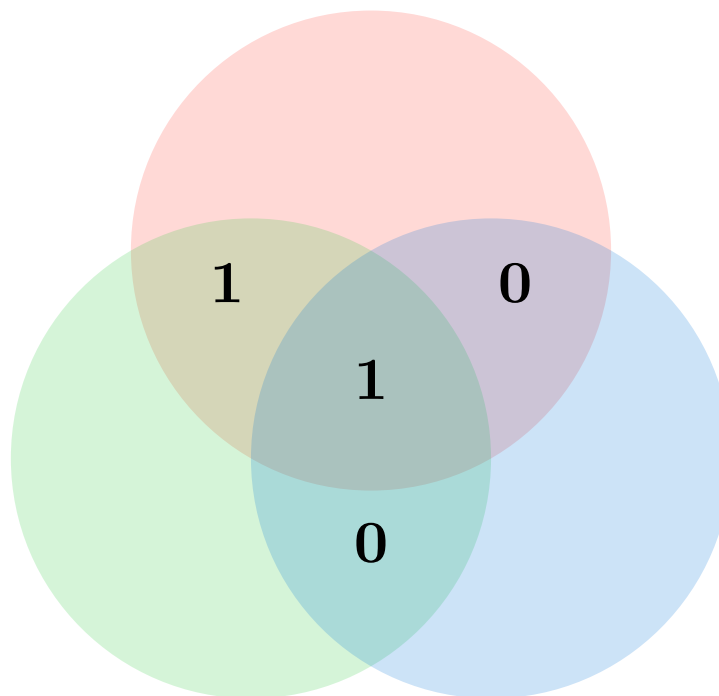
$$x_1 + x_2 + x_4 + p_1 = 0 \bmod 2$$

$$x_1 + x_3 + x_4 + p_2 = 0 \bmod 2$$

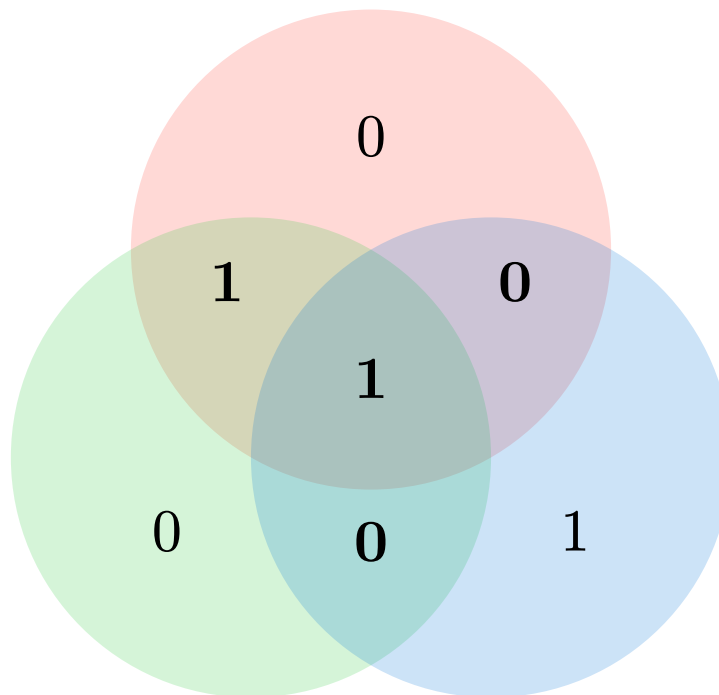
$$x_2 + x_3 + x_4 + p_3 = 0 \bmod 2$$



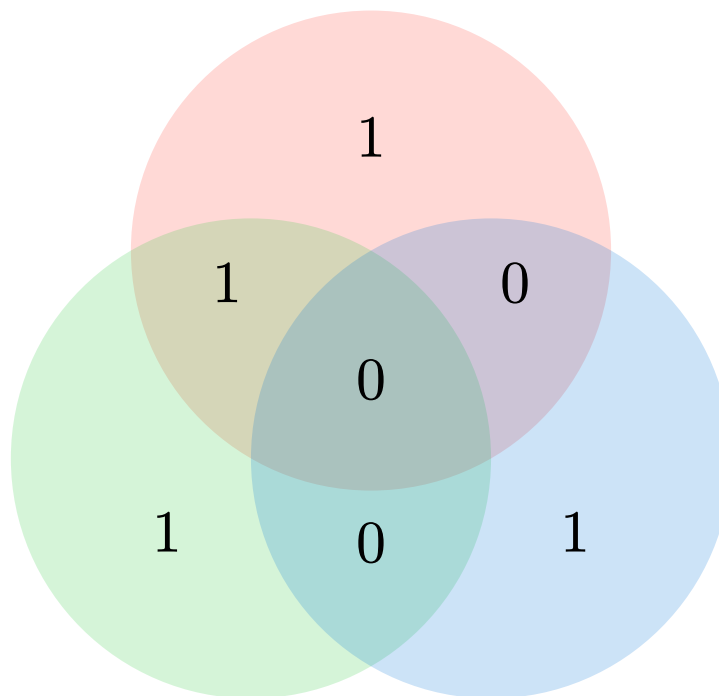
Encoding



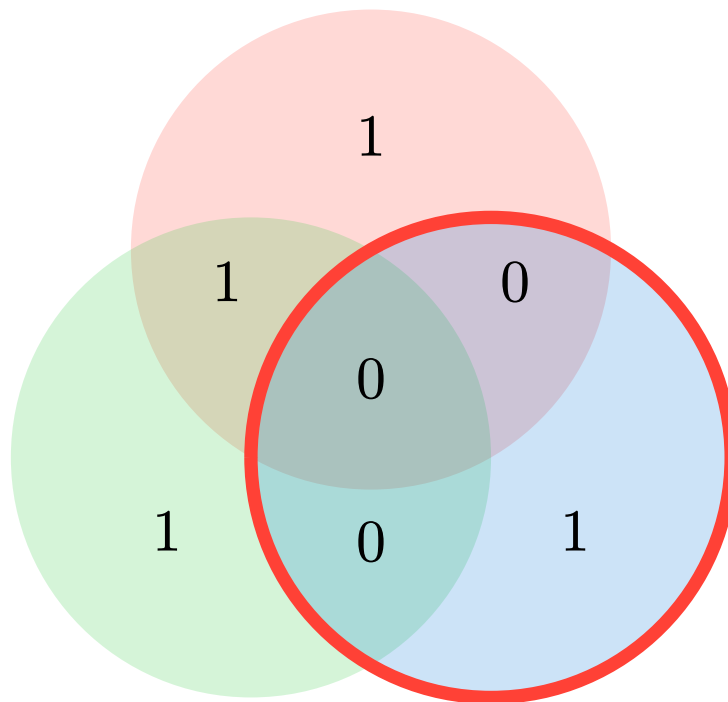
Encoding



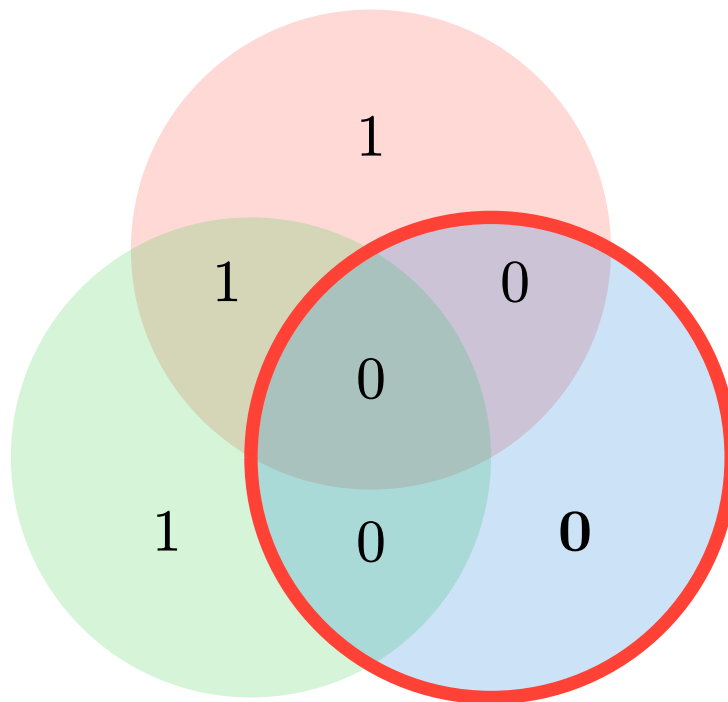
Decoding



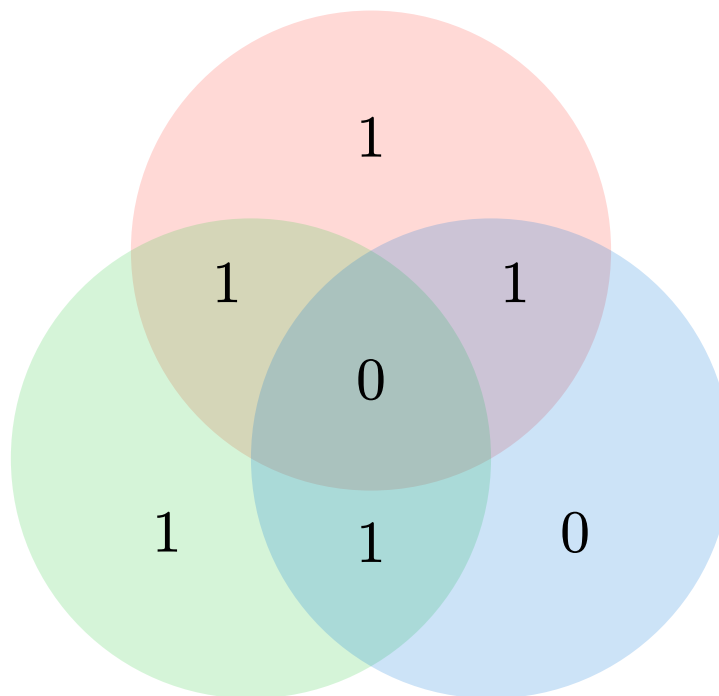
Decoding



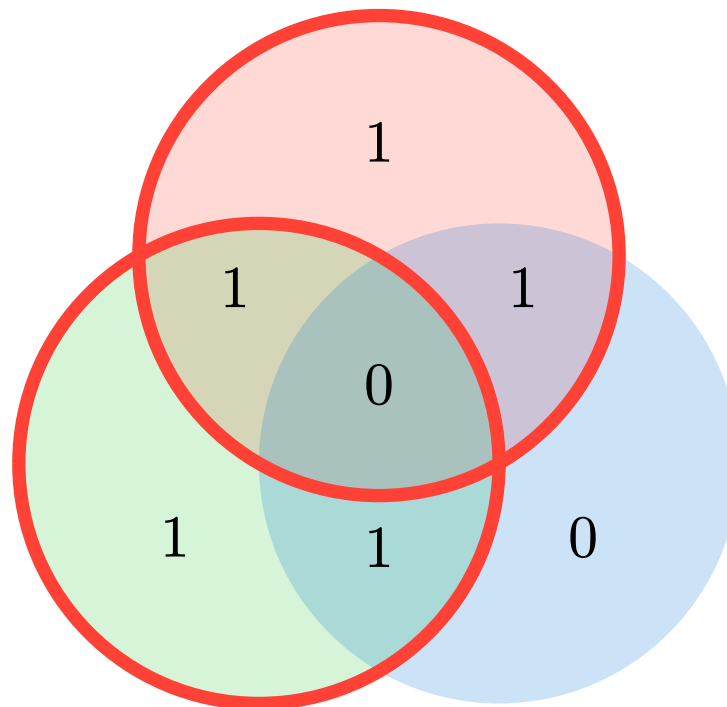
Decoding



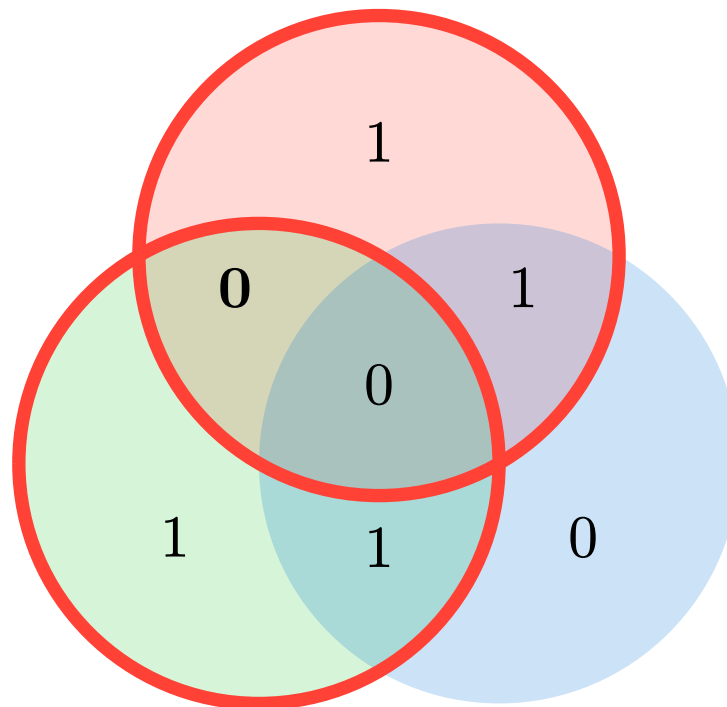
Decoding 2



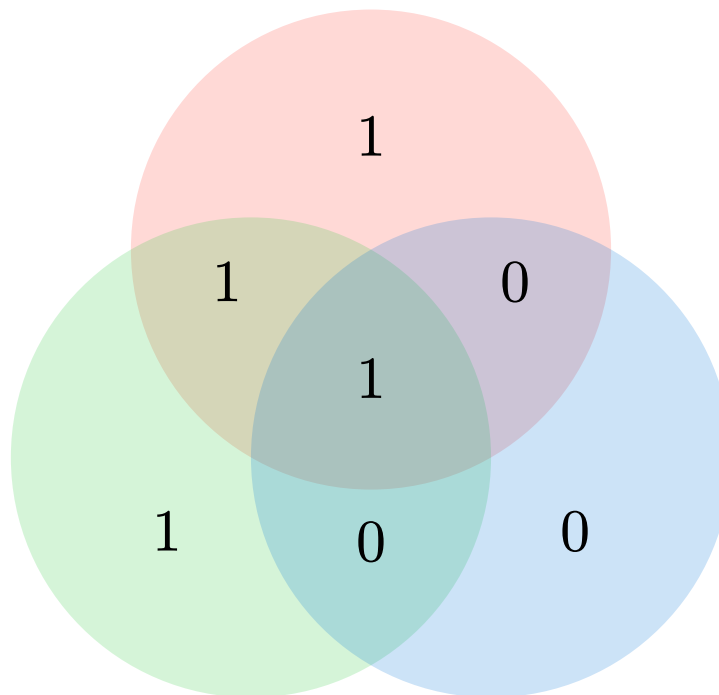
Decoding 2



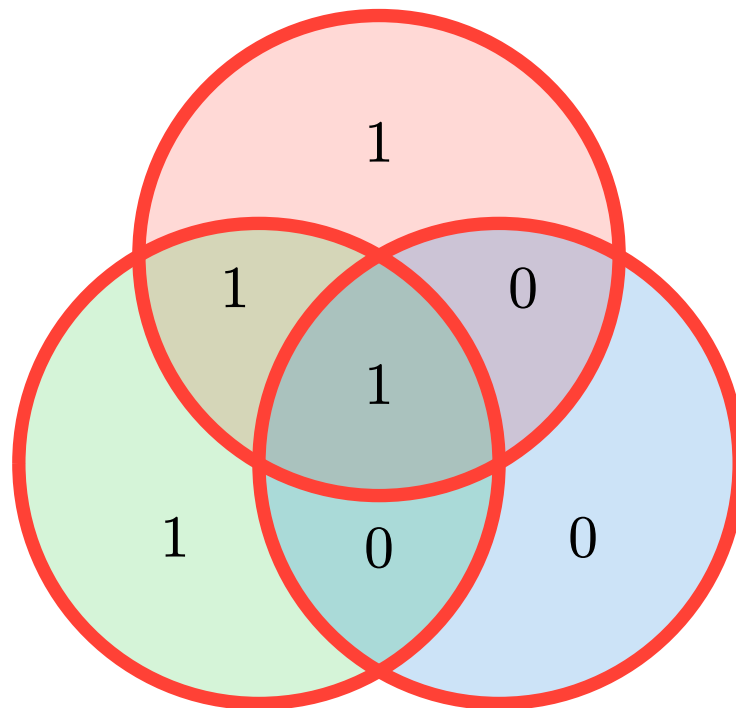
Decoding 2



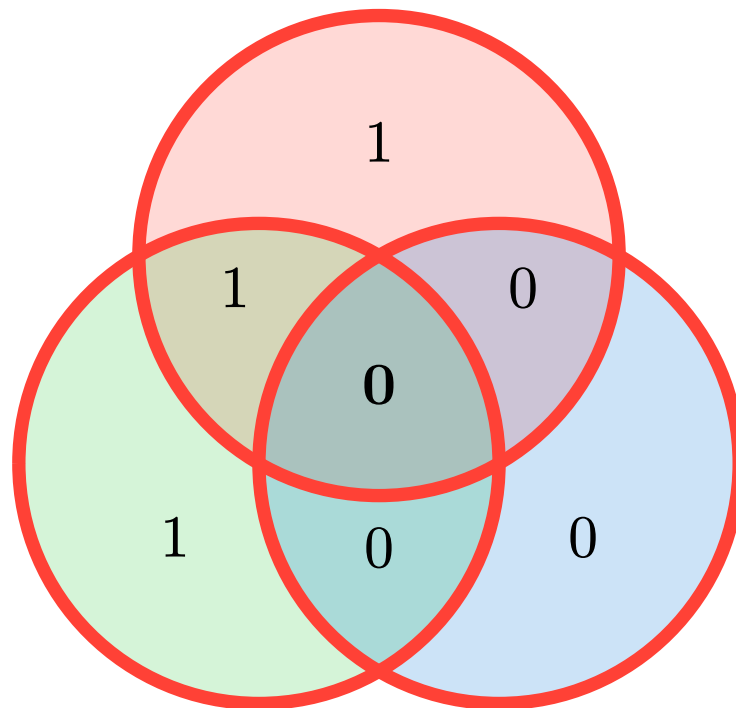
Decoding 3

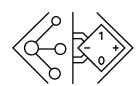


Decoding 3

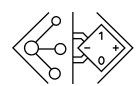


Decoding 3





$$\Delta(101011, 111001) =$$



$$\Delta(101011, 111001) =$$

$$\Delta(101011, 111001) = 2$$

$$\Delta(101011, 111001) = 2$$

$$1100011 \rightarrow 1100\underline{1}11 \rightarrow 11001\underline{0}1 \rightarrow \underline{0}100101$$

$$\Delta(101011, 111001) = 2$$

1100011 $\rightarrow$ 1100111 $\rightarrow$ 1100101 $\rightarrow$ 0100101

10101010 $\rightarrow$ 10101000 $\rightarrow$ 11101000 $\rightarrow$ 11111000 $\rightarrow$ 11110000

$$\Delta(101011, 111001) = 2$$

$$1100011 \rightarrow 1100\underline{1}11 \rightarrow 11001\underline{0}1 \rightarrow \underline{0}100101$$

$$10101010 \rightarrow 101010\underline{00} \rightarrow 1\underline{1}101000 \rightarrow 1111\underline{1}000 \rightarrow 1111\underline{0}000$$

$$11111 \rightarrow 1\underline{0}111 \rightarrow 101\underline{0}1 \rightarrow \underline{00}101 \rightarrow 001\underline{00} \rightarrow 00\underline{000}$$

$$\Delta(101011, 111001) = 2$$

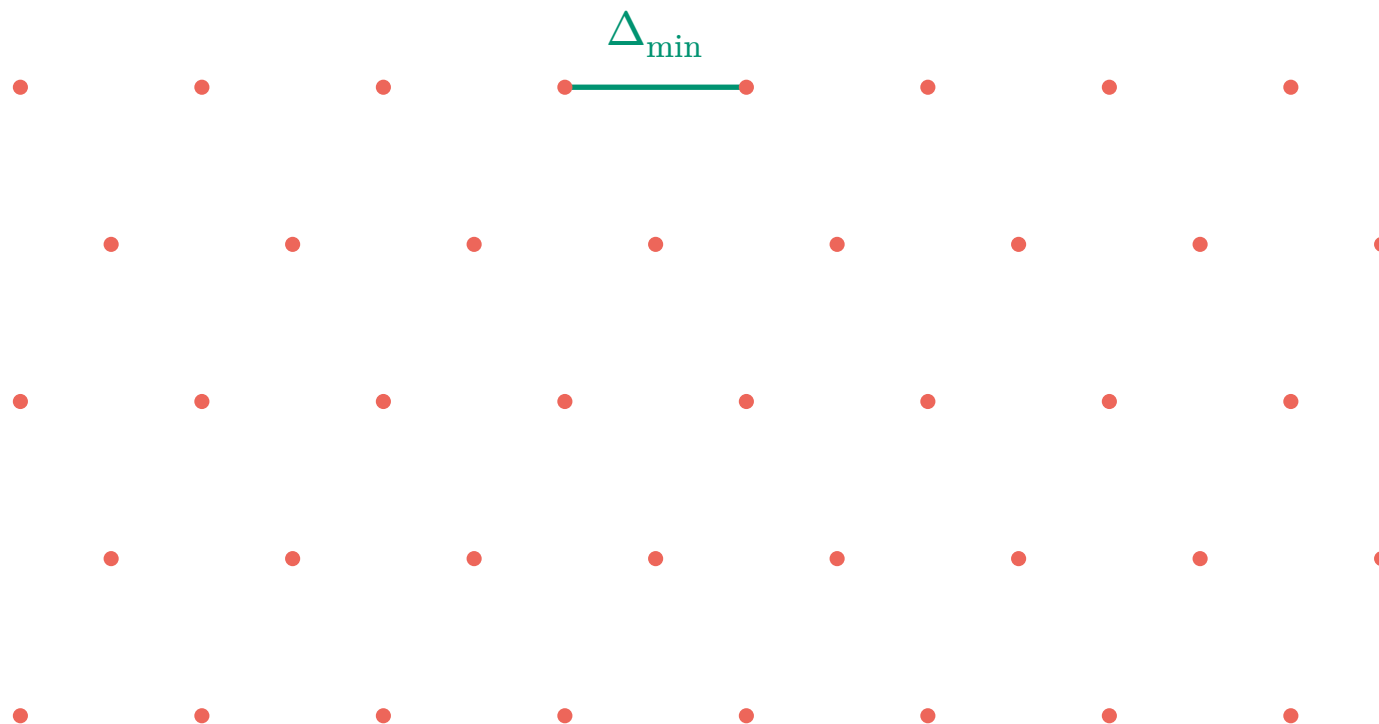
$$1100011 \rightarrow 1100\underline{1}11 \rightarrow 11001\underline{0}1 \rightarrow \underline{0}100101$$

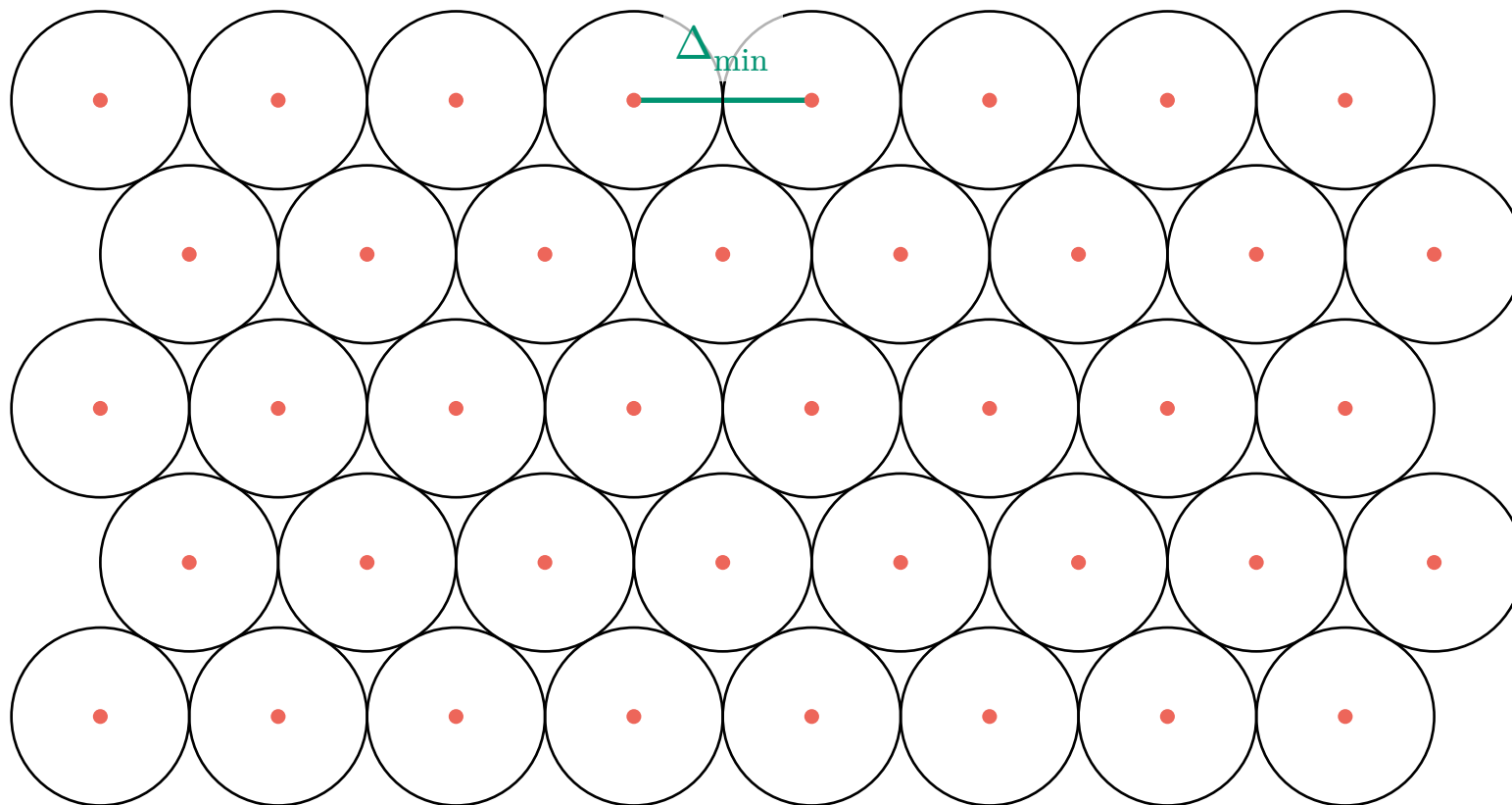
$$10101010 \rightarrow 101010\underline{00} \rightarrow 1\underline{1}101000 \rightarrow 1111\underline{1}000 \rightarrow 1111\underline{0}000$$

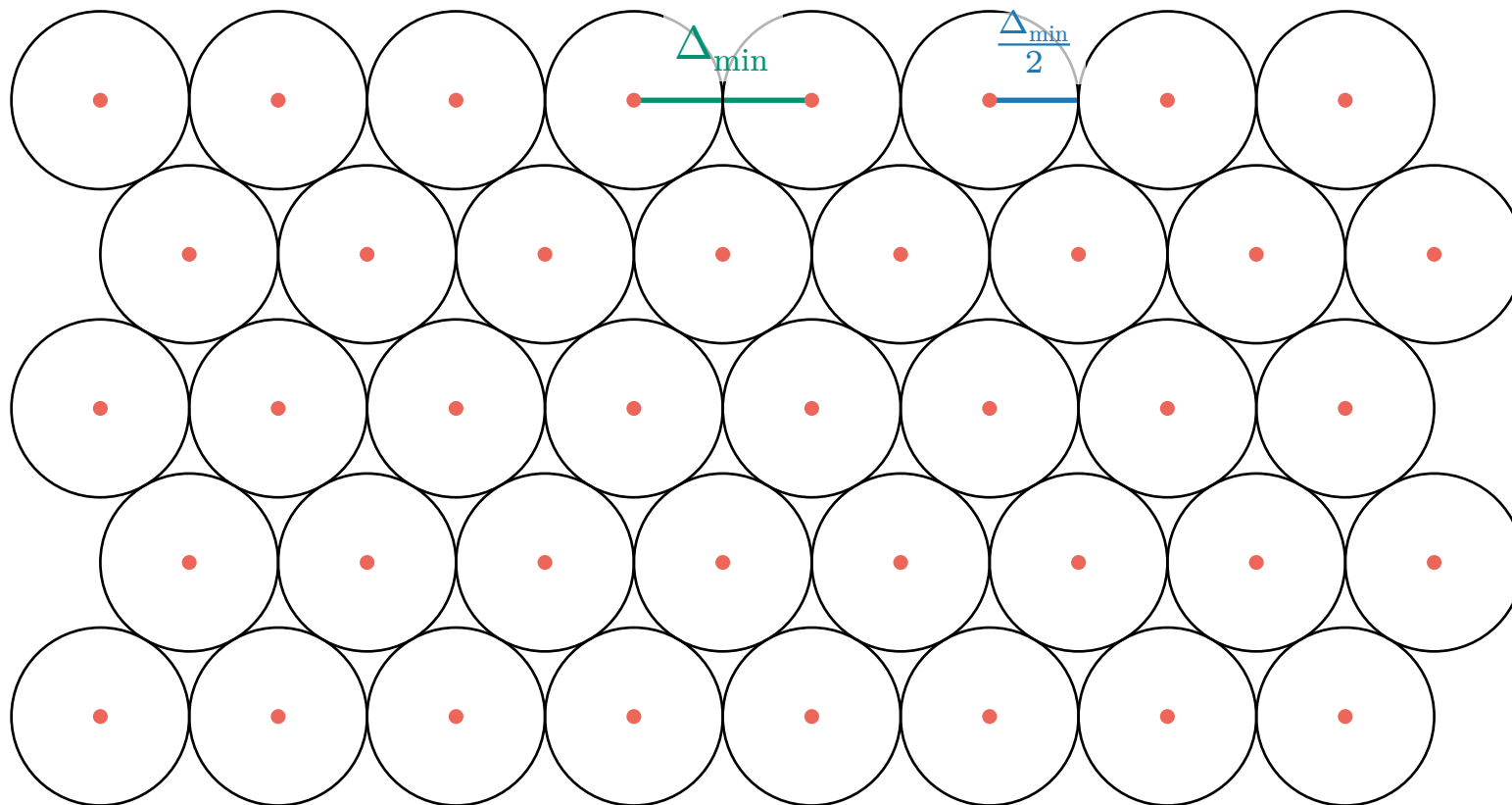
$$11111 \rightarrow 1\underline{0}111 \rightarrow 101\underline{0}1 \rightarrow \underline{00}101 \rightarrow 001\underline{00} \rightarrow 00\underline{000}$$

$$\text{Number of correctable errors} < \frac{\Delta_{\min}}{2}$$









$$g : (x_1, x_2, x_3, x_4) \mapsto (c_1, c_2, c_3, c_4, c_5, c_6, c_7)$$

$$g : (x_1, x_2, x_3, x_4) \mapsto (c_1, c_2, c_3, c_4, c_5, c_6, c_7)$$

$$g : \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} \mapsto \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}}_{G \in \mathbb{F}_2^{n \times k}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} \bmod 2$$

$$h : (c_1, c_2, c_3, c_4, c_5, c_6, c_7) \mapsto \{\text{correct}, \text{error}\}$$

$$h : (c_1, c_2, c_3, c_4, c_5, c_6, c_7) \mapsto \{\text{correct}, \text{error}\}$$

$$h : \underbrace{\begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}}_H \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \\ c_7 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \bmod 2$$

$$Hc = 0$$

$$Hc = 0$$

$$H\tilde{c} =$$

$$H\mathbf{c} = \mathbf{0}$$

$$H\tilde{\mathbf{c}} = H(\mathbf{c} + \mathbf{e}) =$$

$$Hc = 0$$

$$H\tilde{c} = H(c + e) = \underbrace{Hc}_0 + He =$$

$$Hc = 0$$

$$H\tilde{c} = H(c + e) = \underbrace{Hc}_0 + He = He$$

$$Hc = 0$$

$$H\tilde{c} = H(c + e) = \underbrace{Hc}_0 + He = He$$

Syndrome Decoding Problem

Given s and H , find e with $\leq k$ ones, such that

$$He = s.$$

Syndrome Decoding = Error Decoding

$$Hc = 0$$

$$H\tilde{c} = H(c + e) = \underbrace{Hc}_0 + He = He$$

Syndrome Decoding Problem

Given s and H , find e with $\leq k$ ones, such that
 $He = s$.

Syndrome Decoding = Error Decoding

The Syndrome Decoding Problem is NP-hard (in general)



Exercise 1

Syndrome decoding by hand and in Sage

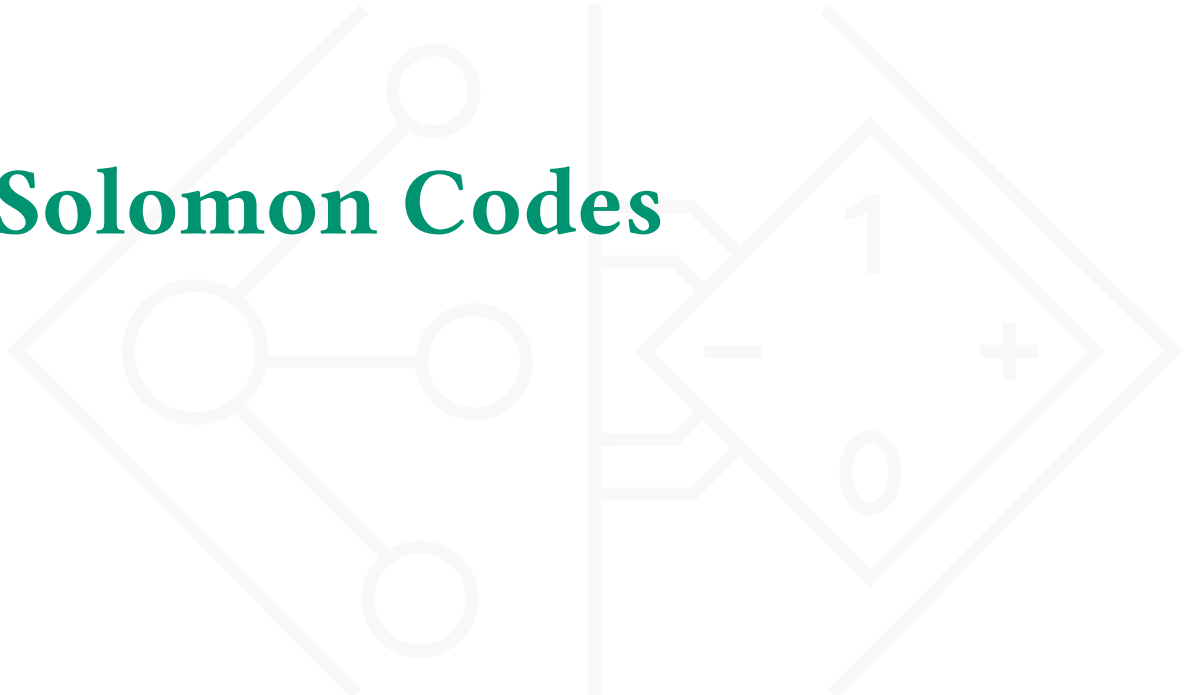
Your task

1. Build the $[7, 4]$ Hamming code in Sage and look at $\mathbf{G}$, $\mathbf{H}$ and the minimum distance
2. Flip one bit of a codeword and compute the syndrome $\mathbf{s} = \mathbf{H}\tilde{\mathbf{c}}$: which column of $\mathbf{H}$ is it? Decode
3. Flip two bits: what happens to the decoded word?

Sage

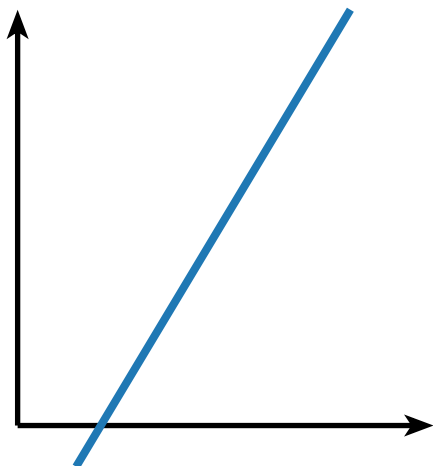
```
C = codes.HammingCode(GF(2), 3)      # [7, 4, 3]
G = C.generator_matrix()
H = C.parity_check_matrix()
c = C.random_element()
e = vector(GF(2), [0, 0, 0, 0, 1, 0, 0])
s = H * (c + e)                       # a column of H?
C.decoder("Syndrome").decode_to_code(c + e)
```

Reed-Solomon Codes

A faint, light gray watermark is visible in the background. It consists of a circuit-like diagram on the left with several circles connected by lines, and a large diamond shape on the right. Inside the diamond are the numbers '1' and '0', and the symbols '-' and '+'. A vertical line separates the two parts of the watermark.

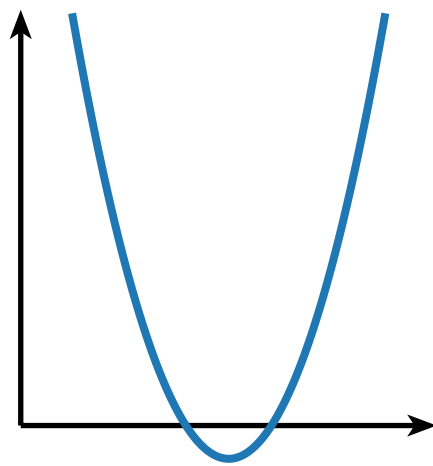
$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_kx^k$$

Degree 1



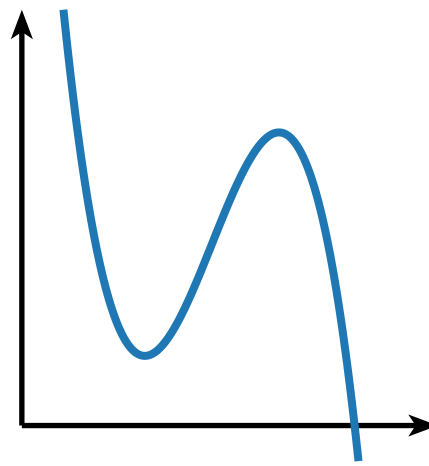
$$a + bx$$

Degree 2



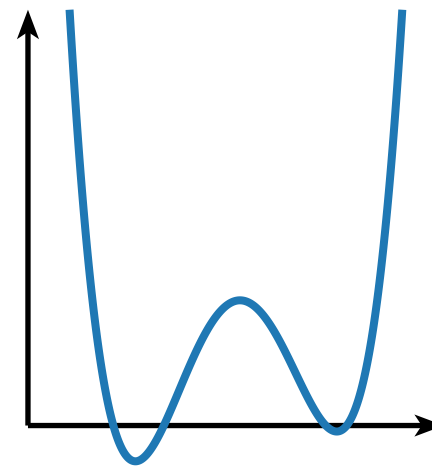
$$a + bx + cx^2$$

Degree 3



$$a + bx + cx^2 + dx^3$$

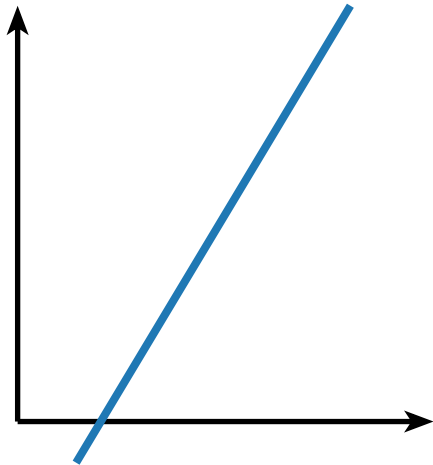
Degree 4



$$a + bx + cx^2 + dx^3 + ex^4$$

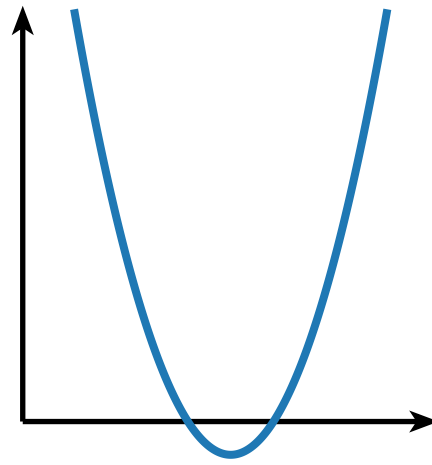
$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_kx^k$$

Degree 1



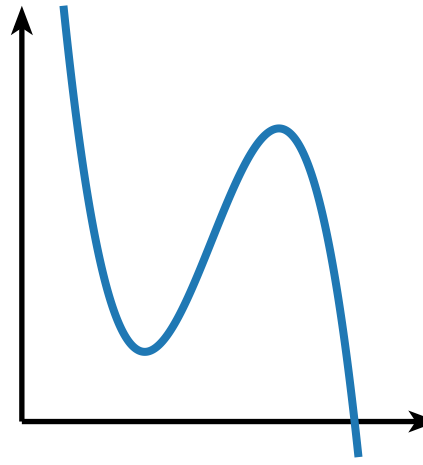
$$a + bx$$

Degree 2



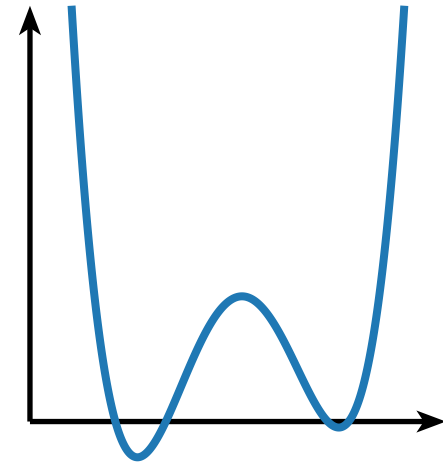
$$a + bx + cx^2$$

Degree 3



$$a + bx + cx^2 + dx^3$$

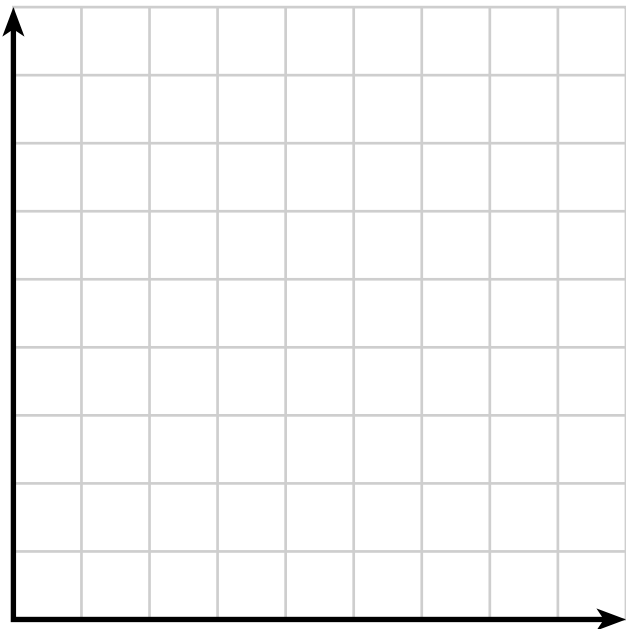
Degree 4



$$a + bx + cx^2 + dx^3 + ex^4$$

Polynomial of degree k has at most k roots

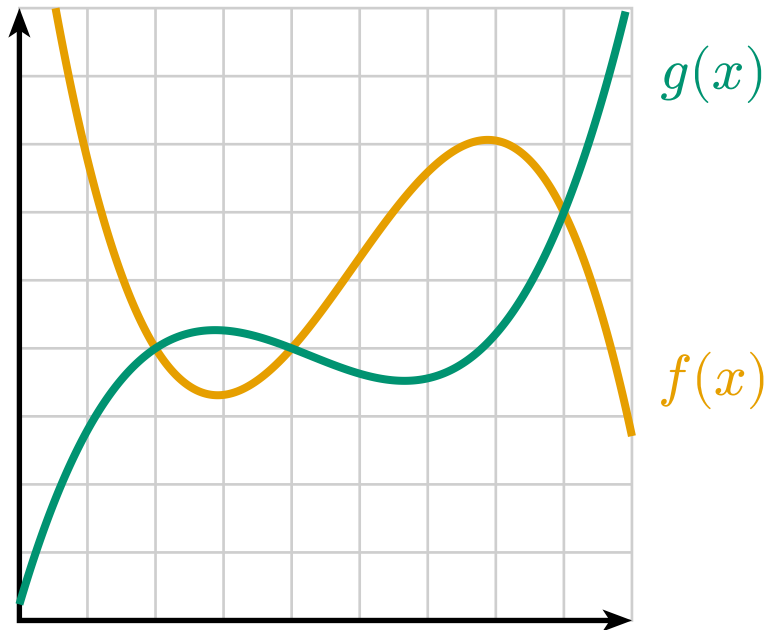
Polynomials of degree k have at most k points in common.



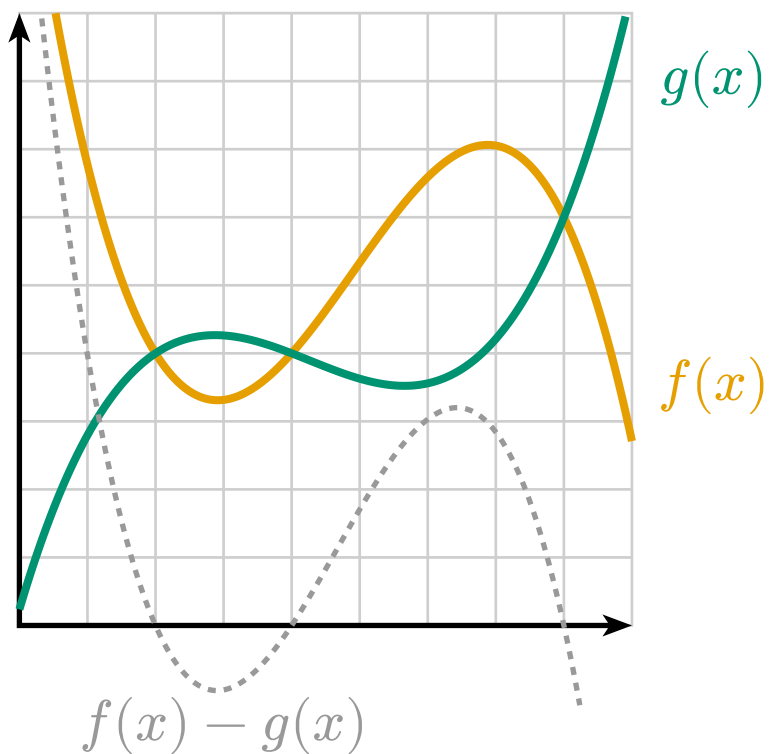
Polynomials of degree k have at most k points in common.



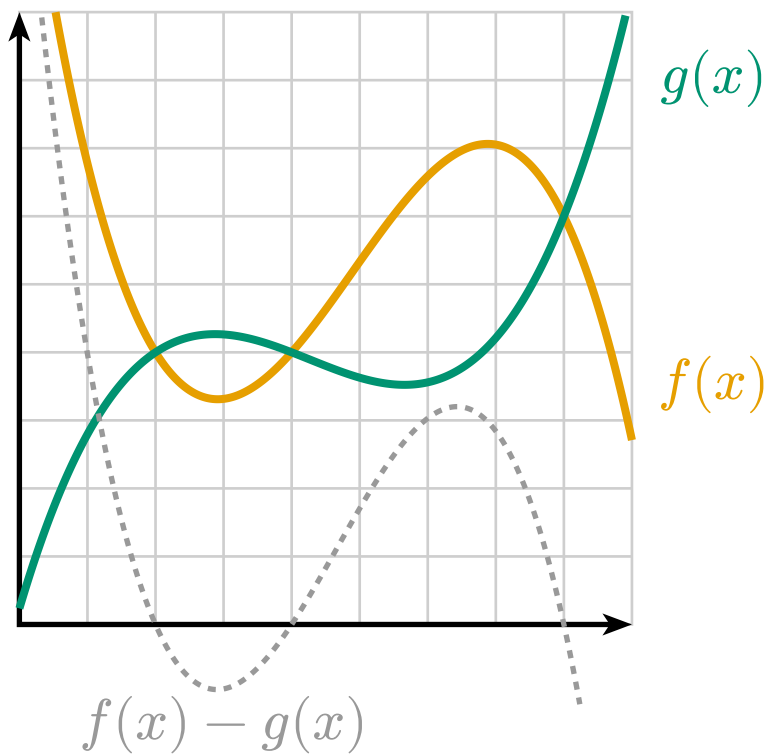
Polynomials of degree k have at most k points in common.



Polynomials of degree k have at most k points in common.

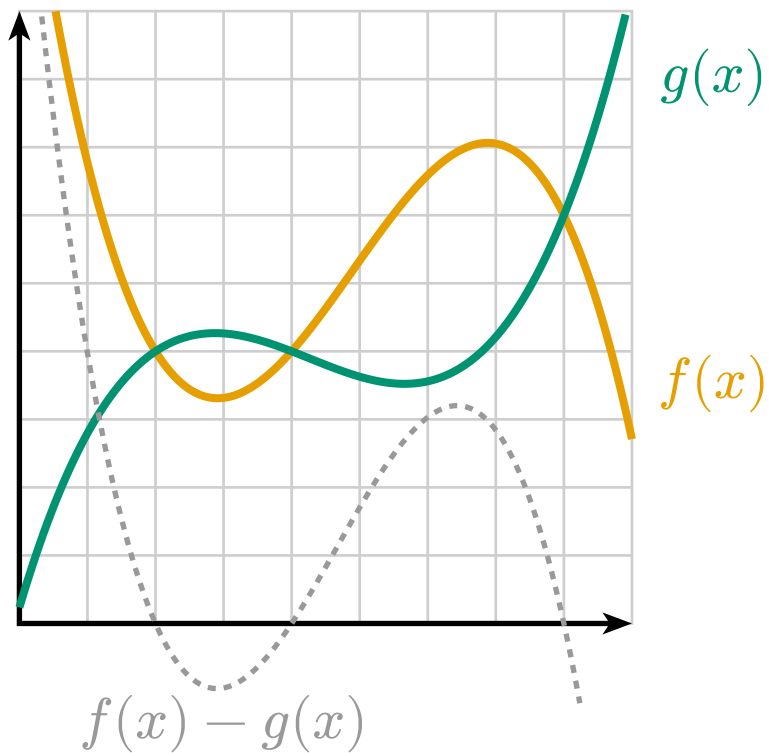


Polynomials of degree k have at most k points in common.



Message: $abcd$

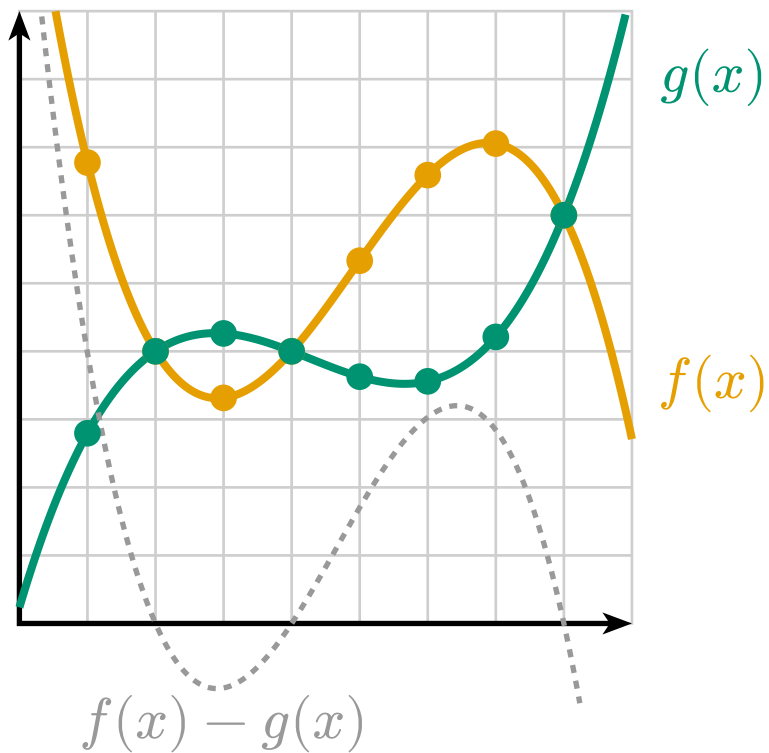
Polynomials of degree k have at most k points in common.



Message: $abcd$

$$f(x) = a + bx + cx^2 + dx^3$$

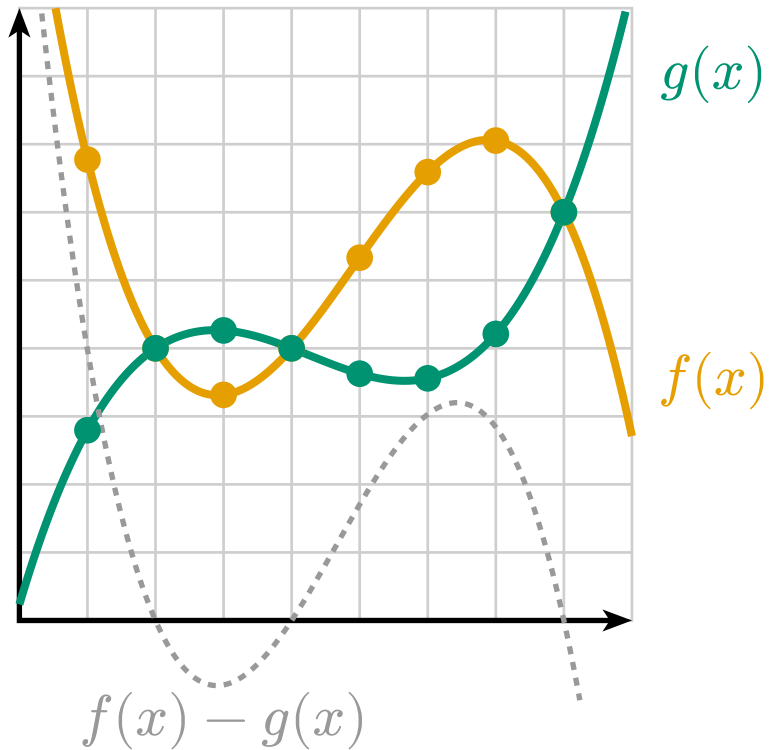
Polynomials of degree k have at most k points in common.



Message: $abcd$

$$f(x) = a + bx + cx^2 + dx^3$$

Polynomials of degree k have at most k points in common.



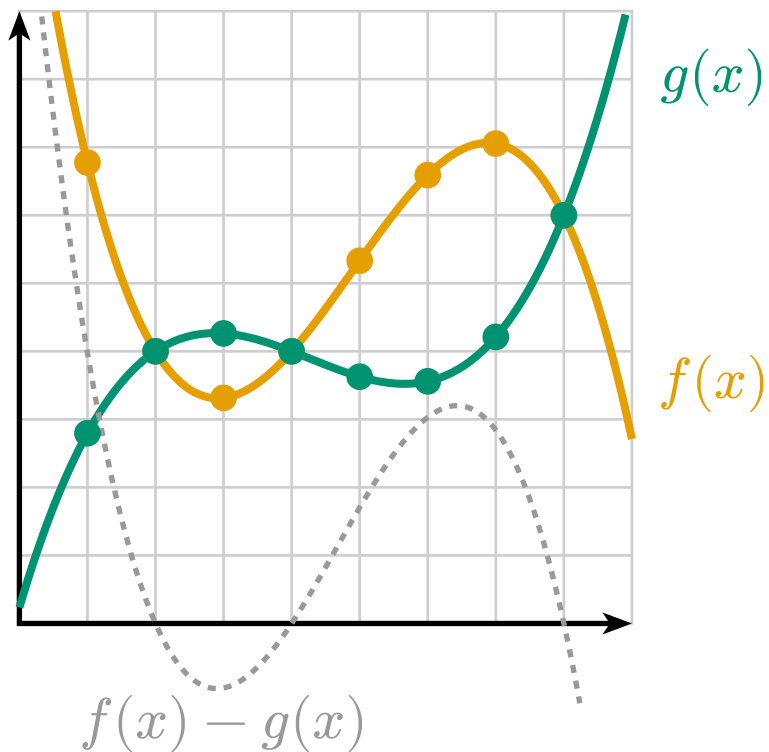
Message: $abcd$

$$f(x) = a + bx + cx^2 + dx^3$$

Send

$$f(1), f(2), \dots, f(7), f(8)$$

Polynomials of degree k have at most k points in common.



Message: $abcd$

$$f(x) = a + bx + cx^2 + dx^3$$

Send

$$f(1), f(2), \dots, f(7), f(8)$$

$$\text{Minimum distance } \Delta_{\min} = 8 - 3 = 5$$

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

$$\text{Codeword} = (f(1), f(2), f(3), \dots, f(n))$$

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

$$\text{Codeword} = (f(1), f(2), f(3), \dots, f(n))$$

$$\text{with } \Delta_{\min} =$$

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

$$\text{Codeword} = (f(1), f(2), f(3), \dots, f(n))$$

$$\text{with } \Delta_{\min} = n - k$$

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

$$\text{Codeword} = (f(1), f(2), f(3), \dots, f(n))$$

$$\text{with } \Delta_{\min} = n - k$$

Finite field

$$a_i, x, f(x) \in \mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$$

Addition and multiplication mod p

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

$$\text{Codeword} = (f(1), f(2), f(3), \dots, f(n))$$

$$\text{with } \Delta_{\min} = n - k$$

Finite field

$$a_i, x, f(x) \in \mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$$

Addition and multiplication mod p

“Basic arithmetic operations” $+, -, \times, \div$

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

$$\text{Codeword} = (f(1), f(2), f(3), \dots, f(n))$$

$$\text{with } \Delta_{\min} = n - k$$

Finite field

$$a_i, x, f(x) \in \mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$$

Addition and multiplication mod p

“Basic arithmetic operations” $+, -, \times, \div$

Commutativity, Associativity, Distributivity

Reed-Solomon Codes

Given message $a_0, a_1, a_2, \dots, a_k$ and block length n :

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_kx^k$$

$$\text{Codeword} = (f(1), f(2), f(3), \dots, f(n))$$

$$\text{with } \Delta_{\min} = n - k$$

Finite field

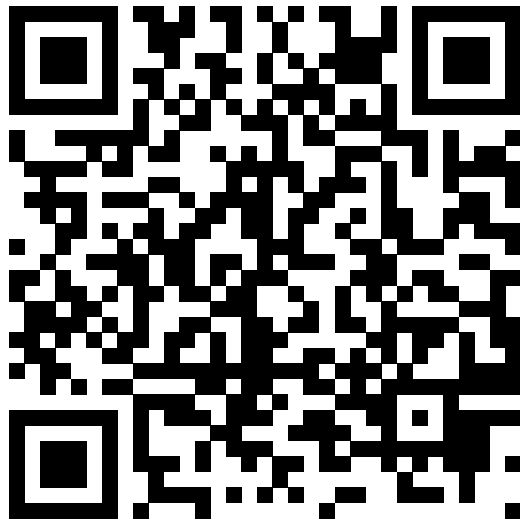
$$a_i, x, f(x) \in \mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$$

Addition and multiplication mod p

“Basic arithmetic operations” $+, -, \times, \div$

Commutativity, Associativity, Distributivity

Polynomials “work” the same over $\mathbb{F}_p$.



RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$G = \begin{pmatrix} 1^0 & 1^1 & 1^2 & 1^3 \\ 2^0 & 2^1 & 2^2 & 2^3 \\ 3^0 & 3^1 & 3^2 & 3^3 \\ 4^0 & 4^1 & 4^2 & 4^3 \\ 5^0 & 5^1 & 5^2 & 5^3 \\ 6^0 & 6^1 & 6^2 & 6^3 \\ 7^0 & 7^1 & 7^2 & 7^3 \end{pmatrix}$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 5 \\ 1 & 4 & 5 & 9 \\ 1 & 5 & 3 & 4 \\ 1 & 6 & 3 & 7 \\ 1 & 7 & 5 & 2 \end{pmatrix}$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 5 \\ 1 & 4 & 5 & 9 \\ 1 & 5 & 3 & 4 \\ 1 & 6 & 3 & 7 \\ 1 & 7 & 5 & 2 \end{pmatrix} \quad \mathbf{H} = \begin{pmatrix} 9 & 1 & 3 & 7 & 3 & 1 & 9 \\ 9 & 2 & 9 & 6 & 4 & 6 & 8 \\ 9 & 4 & 5 & 2 & 9 & 3 & 1 \end{pmatrix}$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 5 \\ 1 & 4 & 5 & 9 \\ 1 & 5 & 3 & 4 \\ 1 & 6 & 3 & 7 \\ 1 & 7 & 5 & 2 \end{pmatrix} \quad H = \begin{pmatrix} 9 & 1 & 3 & 7 & 3 & 1 & 9 \\ 9 & 2 & 9 & 6 & 4 & 6 & 8 \\ 9 & 4 & 5 & 2 & 9 & 3 & 1 \end{pmatrix}$$

$$H^T = \begin{pmatrix} 9 & 9 & 9 \\ 1 & 2 & 4 \\ 3 & 9 & 5 \\ 7 & 6 & 2 \\ 3 & 4 & 9 \\ 1 & 6 & 3 \\ 9 & 8 & 1 \end{pmatrix}$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 5 \\ 1 & 4 & 5 & 9 \\ 1 & 5 & 3 & 4 \\ 1 & 6 & 3 & 7 \\ 1 & 7 & 5 & 2 \end{pmatrix} \quad \mathbf{H} = \begin{pmatrix} 9 & 1 & 3 & 7 & 3 & 1 & 9 \\ 9 & 2 & 9 & 6 & 4 & 6 & 8 \\ 9 & 4 & 5 & 2 & 9 & 3 & 1 \end{pmatrix}$$

$$\mathbf{G}^\perp = \begin{pmatrix} 9 & 9 & 9 \\ 1 & 2 & 4 \\ 3 & 9 & 5 \\ 7 & 6 & 2 \\ 3 & 4 & 9 \\ 1 & 6 & 3 \\ 9 & 8 & 1 \end{pmatrix}$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 5 \\ 1 & 4 & 5 & 9 \\ 1 & 5 & 3 & 4 \\ 1 & 6 & 3 & 7 \\ 1 & 7 & 5 & 2 \end{pmatrix} \quad H = \begin{pmatrix} 9 & 1 & 3 & 7 & 3 & 1 & 9 \\ 9 & 2 & 9 & 6 & 4 & 6 & 8 \\ 9 & 4 & 5 & 2 & 9 & 3 & 1 \end{pmatrix}$$

(almost) RS code over $\mathbb{F}_{11}$ ($k = 2, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2$$

$$(9f(1), 1f(2), 3f(3), 7f(4), 3f(5), 1f(6), 9f(7))$$

$$G^\perp = \begin{pmatrix} 9 & 9 & 9 \\ 1 & 2 & 4 \\ 3 & 9 & 5 \\ 7 & 6 & 2 \\ 3 & 4 & 9 \\ 1 & 6 & 3 \\ 9 & 8 & 1 \end{pmatrix}$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 5 \\ 1 & 4 & 5 & 9 \\ 1 & 5 & 3 & 4 \\ 1 & 6 & 3 & 7 \\ 1 & 7 & 5 & 2 \end{pmatrix} \quad H = \begin{pmatrix} 9 & 1 & 3 & 7 & 3 & 1 & 9 \\ 9 & 2 & 9 & 6 & 4 & 6 & 8 \\ 9 & 4 & 5 & 2 & 9 & 3 & 1 \end{pmatrix}$$

(almost) RS code over $\mathbb{F}_{11}$ ($k = 2, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2$$

$$(9f(1), 1f(2), 3f(3), 7f(4), 3f(5), 1f(6), 9f(7))$$

$$G^\perp = \begin{pmatrix} 9 & 9 & 9 \\ 1 & 2 & 4 \\ 3 & 9 & 5 \\ 7 & 6 & 2 \\ 3 & 4 & 9 \\ 1 & 6 & 3 \\ 9 & 8 & 1 \end{pmatrix} \quad H^\perp = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 4 & 9 & 5 & 3 & 3 & 5 \\ 1 & 8 & 5 & 9 & 4 & 7 & 2 \end{pmatrix}$$

RS code over $\mathbb{F}_{11}$ ($k = 3, n = 7$)

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3$$

$$(f(1), f(2), f(3), f(4), f(5), f(6), f(7))$$

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 5 \\ 1 & 4 & 5 & 9 \\ 1 & 5 & 3 & 4 \\ 1 & 6 & 3 & 7 \\ 1 & 7 & 5 & 2 \end{pmatrix} \quad H = \begin{pmatrix} 9 & 1 & 3 & 7 & 3 & 1 & 9 \\ 9 & 2 & 9 & 6 & 4 & 6 & 8 \\ 9 & 4 & 5 & 2 & 9 & 3 & 1 \end{pmatrix}$$

(almost) RS code over $\mathbb{F}_{11}$ ($k = 2, n = 7$)

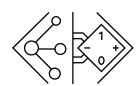
$$f(x) = a_0x^0 + a_1x^1 + a_2x^2$$

$$(9f(1), 1f(2), 3f(3), 7f(4), 3f(5), 1f(6), 9f(7))$$

$$G^\perp = \begin{pmatrix} 9 & 9 & 9 \\ 1 & 2 & 4 \\ 3 & 9 & 5 \\ 7 & 6 & 2 \\ 3 & 4 & 9 \\ 1 & 6 & 3 \\ 9 & 8 & 1 \end{pmatrix} \quad H^\perp = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 4 & 9 & 5 & 3 & 3 & 5 \\ 1 & 8 & 5 & 9 & 4 & 7 & 2 \end{pmatrix}$$

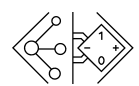
G^T of an RS code = H of another RS code

H^T of an RS code = G of another RS code



Decoding RS codes according to Berlekamp-Welsh

$$f(x) = 2 + 1x$$



Decoding RS codes according to Berlekamp-Welsh

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6))$$

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

Define $E(x) = (x - 4)(x - 6)$,

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

$$\text{Define } E(x) = (x - 4)(x - 6), \quad Q(x) = f(x)E(x)$$

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

Define $E(x) = (x - 4)(x - 6)$, $Q(x) = f(x)E(x)$. Then, $f := \frac{Q}{E}$

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

Define $E(x) = (x - 4)(x - 6)$, $Q(x) = f(x)E(x)$. Then, $f := \frac{Q}{E}$

Observation: $Q(i) = \tilde{c}_i E(i)$ for $i \in \{1, \dots, 6\}$, since $\overbrace{\tilde{c}_i = f(i)}^{\text{correct}}$ or $\overbrace{E(i) = 0}^{\text{error}}$.

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

Define $E(x) = (x - 4)(x - 6)$, $Q(x) = f(x)E(x)$. Then, $f := \frac{Q}{E}$

Observation: $Q(i) = \tilde{c}_i E(i)$ for $i \in \{1, \dots, 6\}$, since $\overbrace{\tilde{c}_i = f(i)}^{\text{correct}}$ or $\overbrace{E(i) = 0}^{\text{error}}$.

All $\mathcal{E}$ and Q with $Q(i) = \tilde{c}_i \mathcal{E}(i)$ work: $f = \frac{Q}{E} = \frac{Q}{\mathcal{E}}$

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

Define $E(x) = (x - 4)(x - 6)$, $Q(x) = f(x)E(x)$. Then, $f := \frac{Q}{E}$

Observation: $Q(i) = \tilde{c}_i E(i)$ for $i \in \{1, \dots, 6\}$, since $\tilde{c}_i = \overbrace{f(i)}^{\text{correct}}$ or $\overbrace{E(i)}^{\text{error}} = 0$.

All $\mathcal{E}$ and Q with $Q(i) = \tilde{c}_i \mathcal{E}(i)$ work: $f = \frac{Q}{E} = \frac{Q}{\mathcal{E}}$

Proof: Define $R(x) = \underbrace{Q(x)}_{\text{degree 5}} \underbrace{\mathcal{E}(x)}_{\text{degree 3+2}} - \underbrace{Q(x)}_{\text{degree 3+2}} \underbrace{E(x)}_{\text{degree 3+2}}$.
Then, $R(i) = \tilde{c}_i E(i) \mathcal{E}(i) - \tilde{c}_i \mathcal{E}(i) E(i) = 0$ for $i \in \{1, \dots, 6\}$
6 roots at degree 5 $\Rightarrow R = 0 \Rightarrow Q\mathcal{E} - Q\mathcal{E} = 0 \Rightarrow \frac{Q}{E} = \frac{Q}{\mathcal{E}}$

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

Define $E(x) = (x - 4)(x - 6)$, $Q(x) = f(x)E(x)$. Then, $f := \frac{Q}{E}$

Observation: $Q(i) = \tilde{c}_i E(i)$ for $i \in \{1, \dots, 6\}$, since $\tilde{c}_i = \overbrace{f(i)}^{\text{correct}}$ or $\overbrace{E(i)}^{\text{error}} = 0$.

All $\mathcal{E}$ and Q with $Q(i) = \tilde{c}_i \mathcal{E}(i)$ work: $f = \frac{Q}{E} = \frac{Q}{\mathcal{E}}$

Proof: Define $R(x) = \underbrace{Q(x)}_{\text{degree 5}} \underbrace{\mathcal{E}(x)}_{\text{degree 3+2}} - \underbrace{Q(x)}_{\text{degree 3+2}} \underbrace{E(x)}_{\text{degree 3+2}}$.

Then, $R(i) = \tilde{c}_i E(i) \mathcal{E}(i) - \tilde{c}_i \mathcal{E}(i) E(i) = 0$ for $i \in \{1, \dots, 6\}$

6 roots at degree 5 $\Rightarrow R = 0 \Rightarrow Q\mathcal{E} - Q\mathcal{E} = 0 \Rightarrow \frac{Q}{E} = \frac{Q}{\mathcal{E}}$

Find $\mathcal{E}$ and Q via linear system of equations with variables $e_0, e_1, e_2, q_0, q_1, q_2$:

$$\forall i \in \{1, \dots, 6\} : q_0 + q_1 i + q_2 i^2 + i^3 = \tilde{c}_i (e_0 + e_1 i + e_2 i^2)$$

$$f(x) = 2 + 1x \longrightarrow c = (f(1), f(2), \dots, f(6)) \rightsquigarrow \tilde{c} = \left(\overbrace{3}^{f(1)}, \overbrace{4}^{f(2)}, \overbrace{5}^{f(3)}, \overbrace{1}^{f(4)}, \overbrace{7}^{f(5)}, \overbrace{7}^{f(6)} \right)$$

Define $E(x) = (x - 4)(x - 6)$, $Q(x) = f(x)E(x)$. Then, $f := \frac{Q}{E}$

Observation: $Q(i) = \tilde{c}_i E(i) f(i) = \tilde{c}_i \underbrace{f(i)}_{\text{correct}} \text{ or } \underbrace{E(i)}_{\text{error}} = 0$.

All $\mathcal{E} = \frac{Q}{\tilde{c}}$

Reed-Solomon codes can be decoded efficiently

$$\begin{aligned} \underbrace{Q(x)}_{\text{degree 5}} &= \underbrace{\tilde{c}_i E(i)}_{\text{degree 3+2}} \underbrace{f(i)}_{\text{degree 3+2}} \\ \text{Then, } R(i) &= \tilde{c}_i E(i) \mathcal{E}(i) - \tilde{c}_i \mathcal{E}(i) E(i) = 0 \text{ for } i \in \{1, \dots, 6\} \\ 6 \text{ roots at degree 5} &\Rightarrow R = 0 \Rightarrow Q\mathcal{E} - Q\mathcal{E} = 0 \Rightarrow \frac{Q}{E} = \frac{Q}{\mathcal{E}} \end{aligned}$$

Find $\mathcal{E}$ and Q via linear system of equations with variables $e_0, e_1, e_2, q_0, q_1, q_2$:

$$\forall i \in \{1, \dots, 6\} : q_0 + q_1 i + q_2 i^2 + i^3 = \tilde{c}_i (e_0 + e_1 i + e_2 i^2)$$



Exercise 2

Interpolation is decoding: see how many errors an RS code tolerates

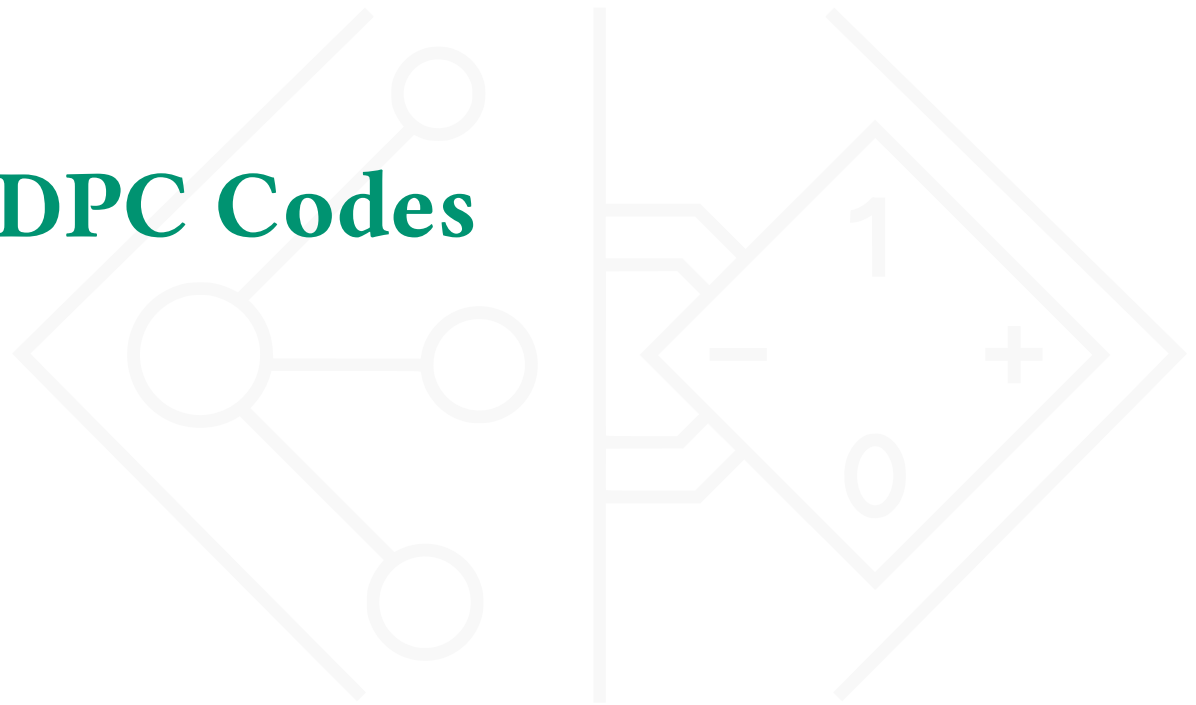
Your task

1. Encode the polynomial $2 + 5x + x^2$ with an RS code over $\mathbb{F}_7$ ($k = 3, n = 6$)
2. Add one, two, three errors and decode: when does Berlekamp-Welch fail?
3. Lower k to 2: how many errors can be corrected now?

Sage

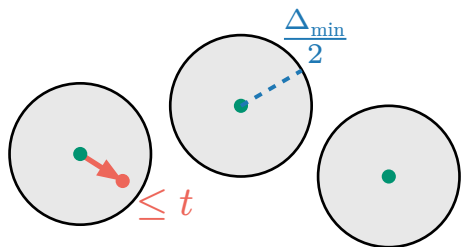
```
F = GF(7)
C = codes.GeneralizedReedSolomonCode(
    [F(i) for i in range(1, 7)], 3) # n = 6, k = 3
c = C.encode(vector(F, [2, 5, 1])) # 2 + 5x + x^2
c_tilde = vector(F, list(c))
c_tilde[0] += 3
C.decoder("BerlekampWelch").decode_to_code(c_tilde)
```

LDPC Codes



Worst case: bounded distance

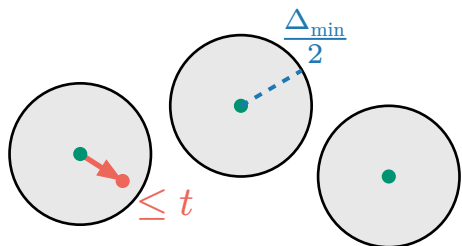
- errors are chosen adversarially
- every pattern of $\leq t$ errors is corrected
- possible iff $t < \frac{\Delta_{\min}}{2}$



decoding spheres of radius t

Worst case: bounded distance

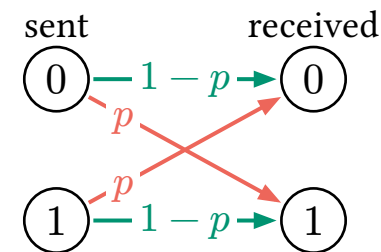
- errors are chosen adversarially
- every pattern of $\leq t$ errors is corrected
- possible iff $t < \frac{\Delta_{\min}}{2}$



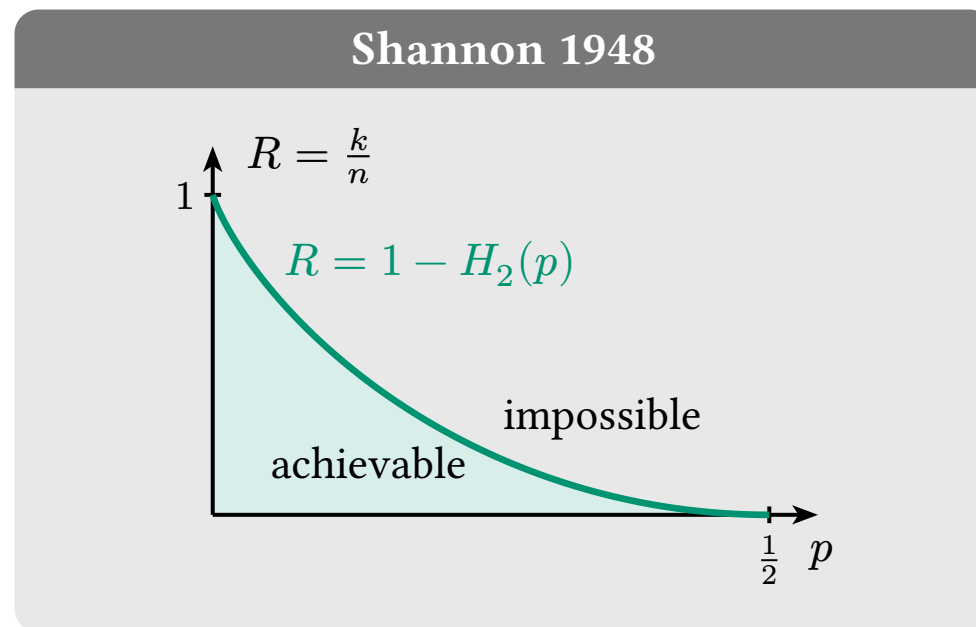
decoding spheres of radius t

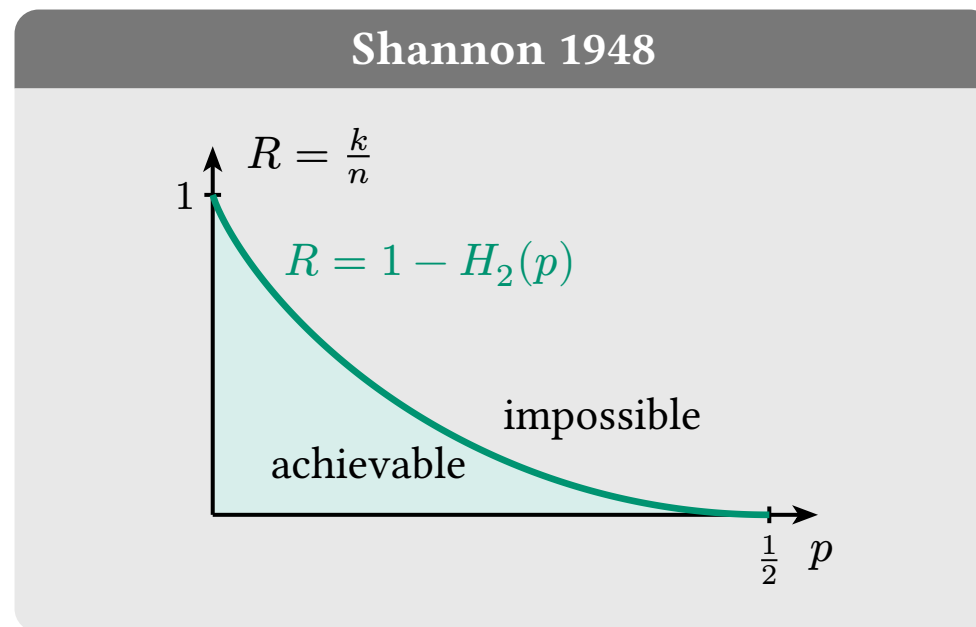
Average case: random errors

- bit-flip with probability p
- decoding succeeds with probability $1 - \varepsilon$
- no hard bound

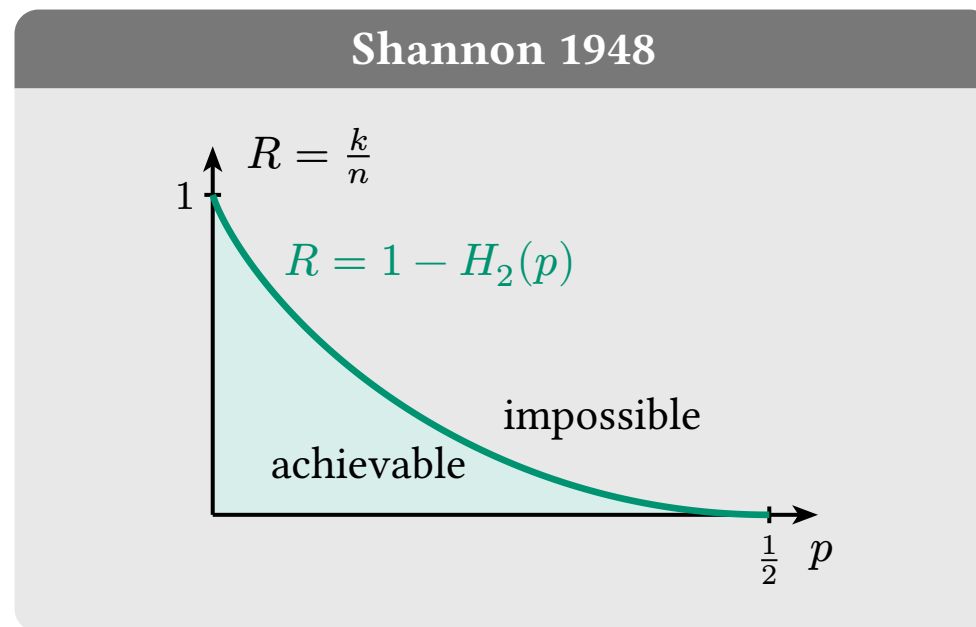


binary symmetric channel





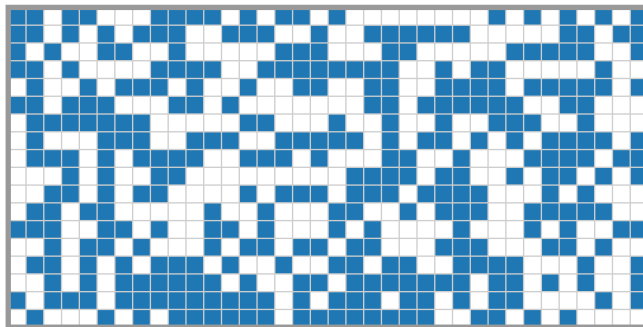
Code with uniformly random $\mathbf{H} \in \mathbb{F}_2^{m \times n}$ meets this bound (w.h.p.)



Code with uniformly random $\mathbf{H} \in \mathbb{F}_2^{m \times n}$ meets this bound (w.h.p.)

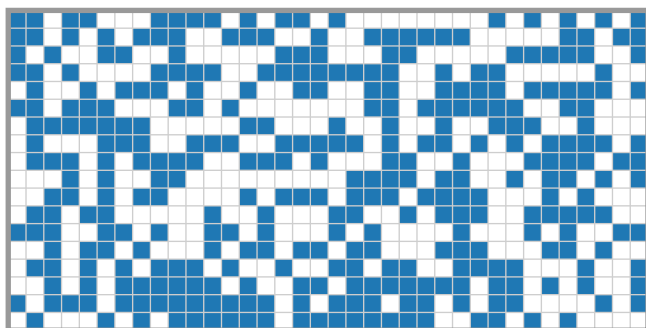
No efficient decoder (NP-hard)

dense random H



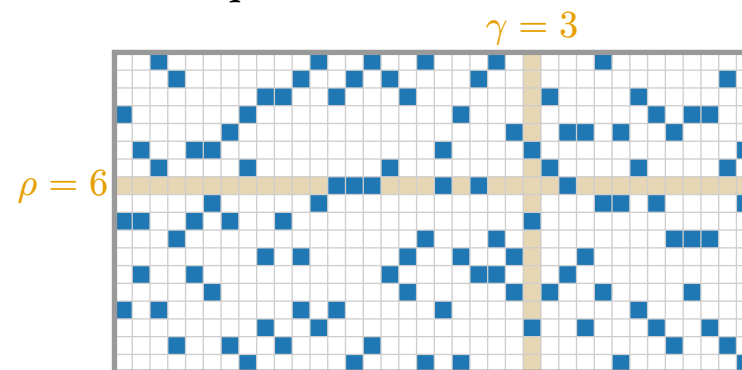
341 ones $\approx \frac{mn}{2}$

dense random H



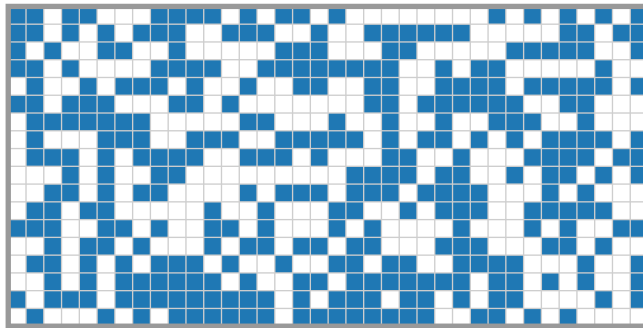
341 ones $\approx \frac{mn}{2}$

sparse random H



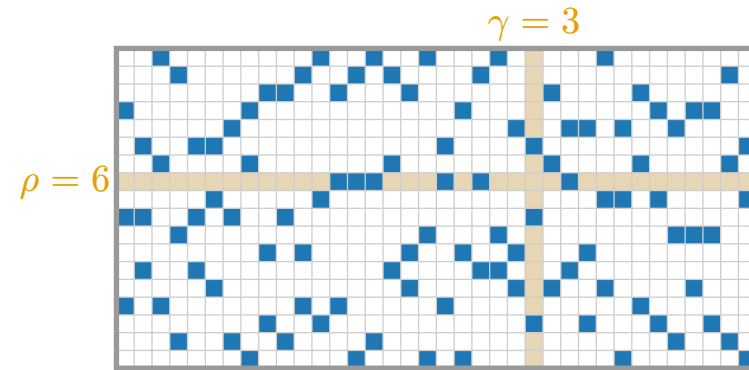
108 ones $= \gamma n$

dense random H



341 ones $\approx \frac{mn}{2}$

sparse random H



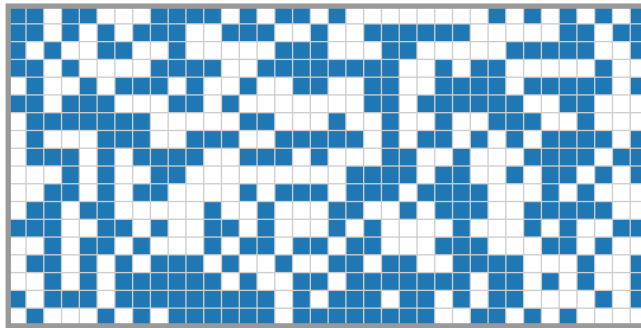
108 ones $= \gamma n$

LDPC code (Gallager 1963)

Linear code with sparse parity-check matrix H .

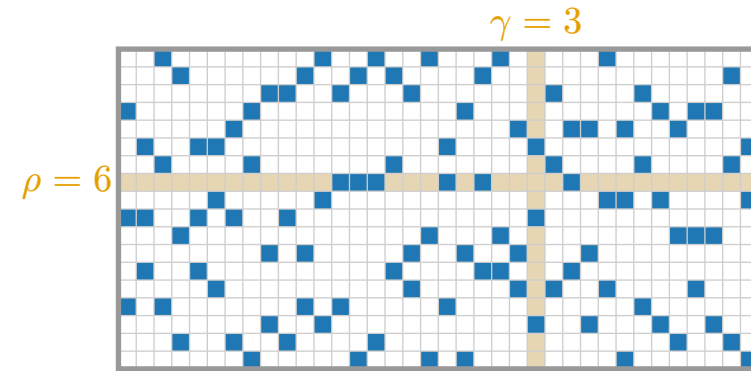
Regular (ρ, γ) : exactly ρ ones per row, γ ones per column $\Rightarrow R \geq 1 - \frac{\gamma}{\rho}$

dense random H



341 ones $\approx \frac{mn}{2}$

sparse random H



$\rho = 6$

$\gamma = 3$

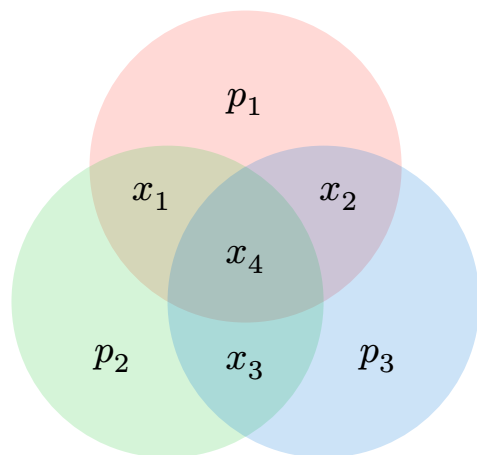
108 ones $= \gamma n$

LDPC code (Gallager 1963)

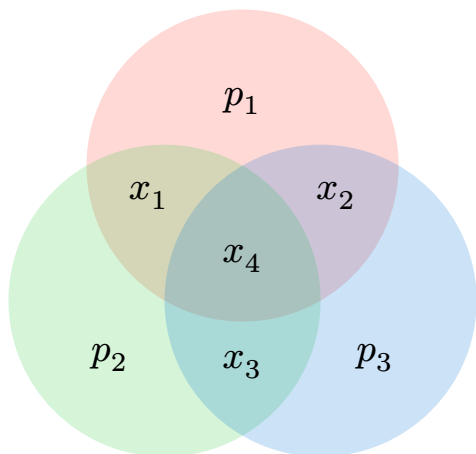
Linear code with sparse parity-check matrix H .

Regular (ρ, γ) : exactly ρ ones per row, γ ones per column $\Rightarrow R \geq 1 - \frac{\gamma}{\rho}$

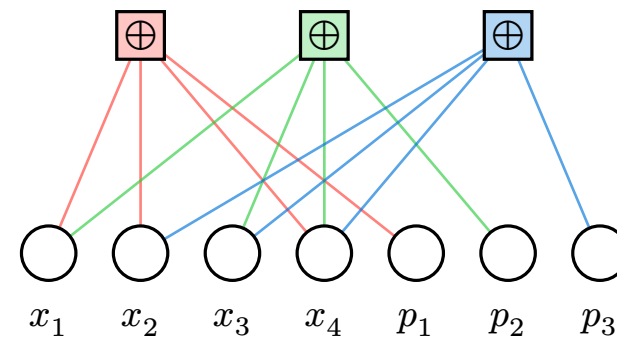
- LDPC codes approach Shannon limit
- **efficient decoders exist**

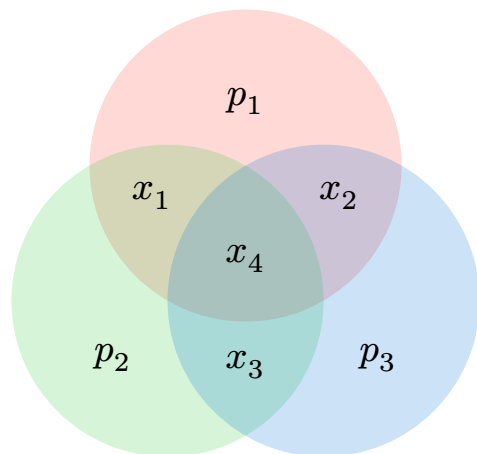


$$\mathbf{H} = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

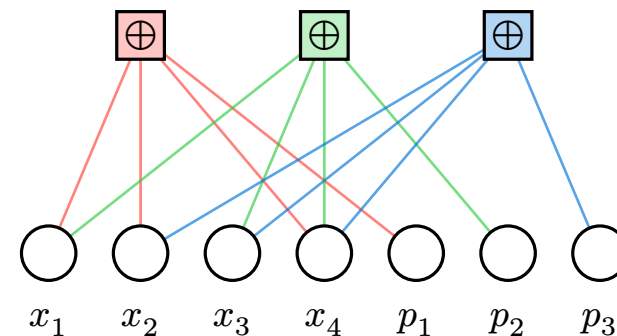


$$H = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

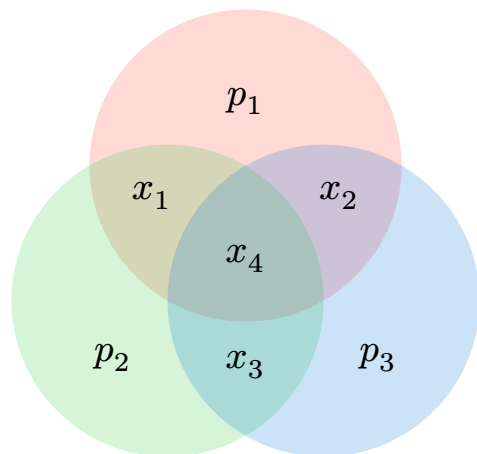




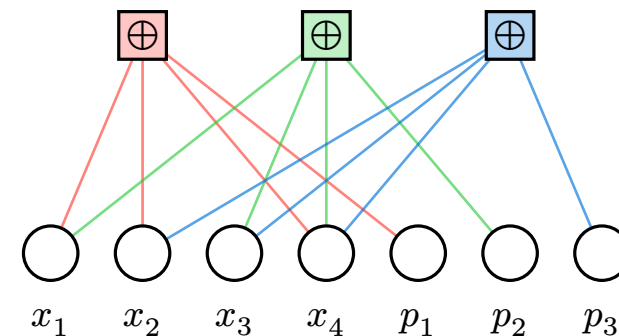
$$\mathbf{H} = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$



Bipartite graph:
variable nodes = code bits,
check nodes = rows of $\mathbf{H}$



$$\mathbf{H} = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$



Bipartite graph:

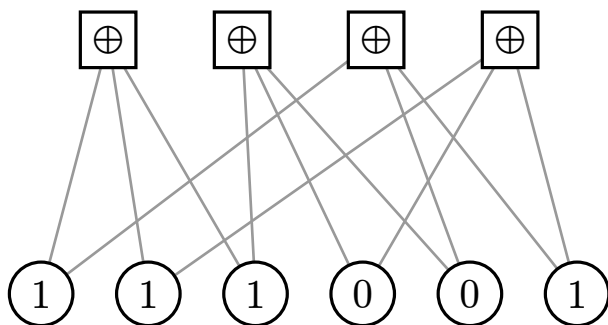
variable nodes = code bits,

check nodes = rows of $\mathbf{H}$

ρ = check degree, γ = variable degree

Pass around beliefs (log-likelihood ratios):

$$L_v = \log \left(\frac{\Pr[c_v = 0 \mid \tilde{c}_v]}{\Pr[c_v = 1 \mid \tilde{c}_v]} \right)$$

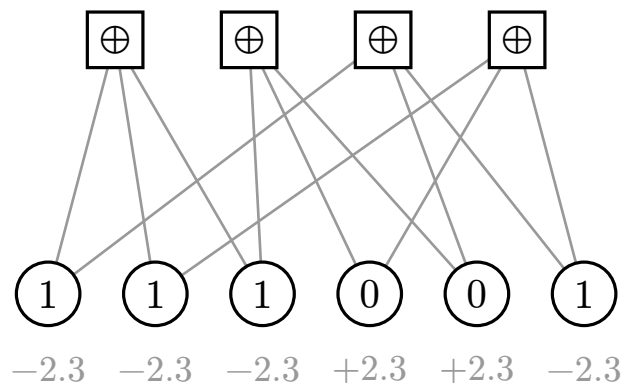


Initialize L_v values:

Pass around beliefs (log-likelihood ratios):

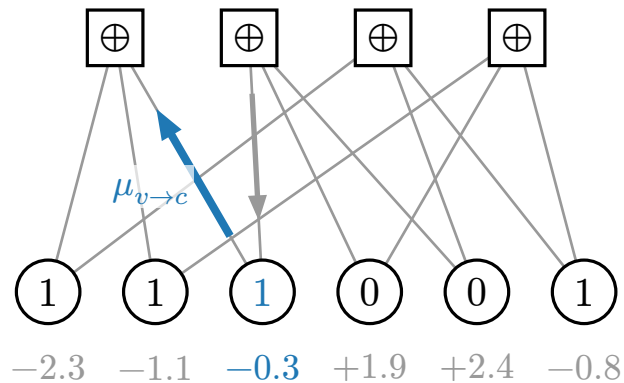
$$L_v = \log \left(\frac{\Pr[c_v = 0 \mid \tilde{c}_v]}{\Pr[c_v = 1 \mid \tilde{c}_v]} \right)$$

$$L_v = (-1)^{\tilde{c}_v} \log \left(\frac{1-p}{p} \right)$$



Pass around beliefs (log-likelihood ratios):

$$L_v = \log \left(\frac{\Pr[c_v = 0 \mid \tilde{c}_v]}{\Pr[c_v = 1 \mid \tilde{c}_v]} \right)$$



Initialize L_v values:

$$L_v = (-1)^{\tilde{c}_v} \log \left(\frac{1-p}{p} \right)$$

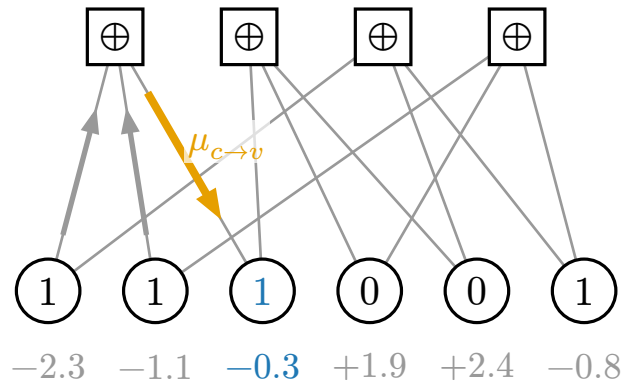
variable $\rightarrow$ check:

what I believe, ignoring what you told me

$$\mu_{v \rightarrow c} = L_v + \sum_{c' \in N(v) \setminus c} \mu_{c' \rightarrow v}$$

Pass around beliefs (log-likelihood ratios):

$$L_v = \log \left(\frac{\Pr[c_v = 0 \mid \tilde{c}_v]}{\Pr[c_v = 1 \mid \tilde{c}_v]} \right)$$



Initialize L_v values:

$$L_v = (-1)^{\tilde{c}_v} \log \left(\frac{1-p}{p} \right)$$

variable $\rightarrow$ check:

what I believe, ignoring what you told me

$$\mu_{v \rightarrow c} = L_v + \sum_{c' \in N(v) \setminus c} \mu_{c' \rightarrow v}$$

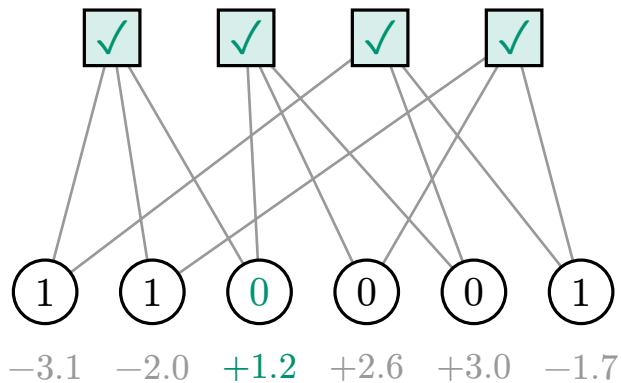
check $\rightarrow$ variable:

what my other bits imply for you

$$\tanh \left(\frac{\mu_{c \rightarrow v}}{2} \right) = \prod_{v' \in N(c) \setminus v} \tanh \left(\frac{\mu_{v' \rightarrow c}}{2} \right)$$

Pass around beliefs (log-likelihood ratios):

$$L_v = \log \left(\frac{\Pr[c_v = 0 \mid \tilde{c}_v]}{\Pr[c_v = 1 \mid \tilde{c}_v]} \right)$$



Initialize L_v values:

$$L_v = (-1)^{\tilde{c}_v} \log \left(\frac{1-p}{p} \right)$$

variable $\rightarrow$ check:

what I believe, ignoring what you told me

$$\mu_{v \rightarrow c} = L_v + \sum_{c' \in N(v) \setminus c} \mu_{c' \rightarrow v}$$

check $\rightarrow$ variable:

what my other bits imply for you

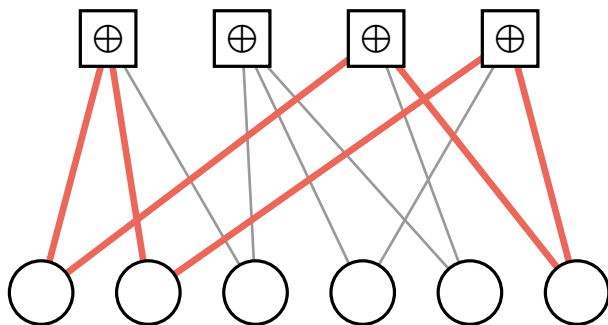
$$\tanh \left(\frac{\mu_{c \rightarrow v}}{2} \right) = \prod_{v' \in N(c) \setminus v} \tanh \left(\frac{\mu_{v' \rightarrow c}}{2} \right)$$

Iterate, stop once $\mathbf{H}\hat{\mathbf{c}} = \mathbf{0}$,

decide by the sign of $L_v + \sum_{c \in N(v)} \mu_{c \rightarrow v}$

If the Tanner graph is a tree

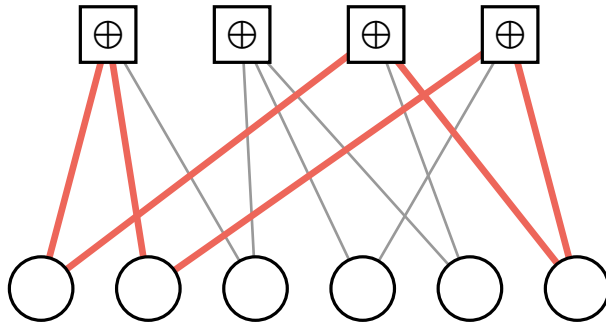
BP works perfectly, but the code is weak



shortest cycle here: length 6

If the Tanner graph is a tree

BP works perfectly, but the code is weak



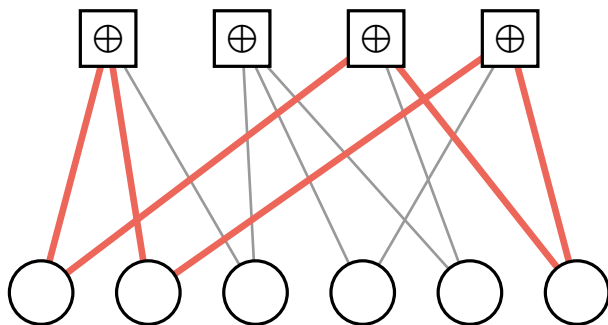
shortest cycle here: length 6

Sparse random graphs

Short cycles are rare

⇒ locally the graph looks like a tree

⇒ the heuristic stays close to optimal



shortest cycle here: length 6

If the Tanner graph is a tree

BP works perfectly, but the code is weak

Sparse random graphs

Short cycles are rare

⇒ locally the graph looks like a tree

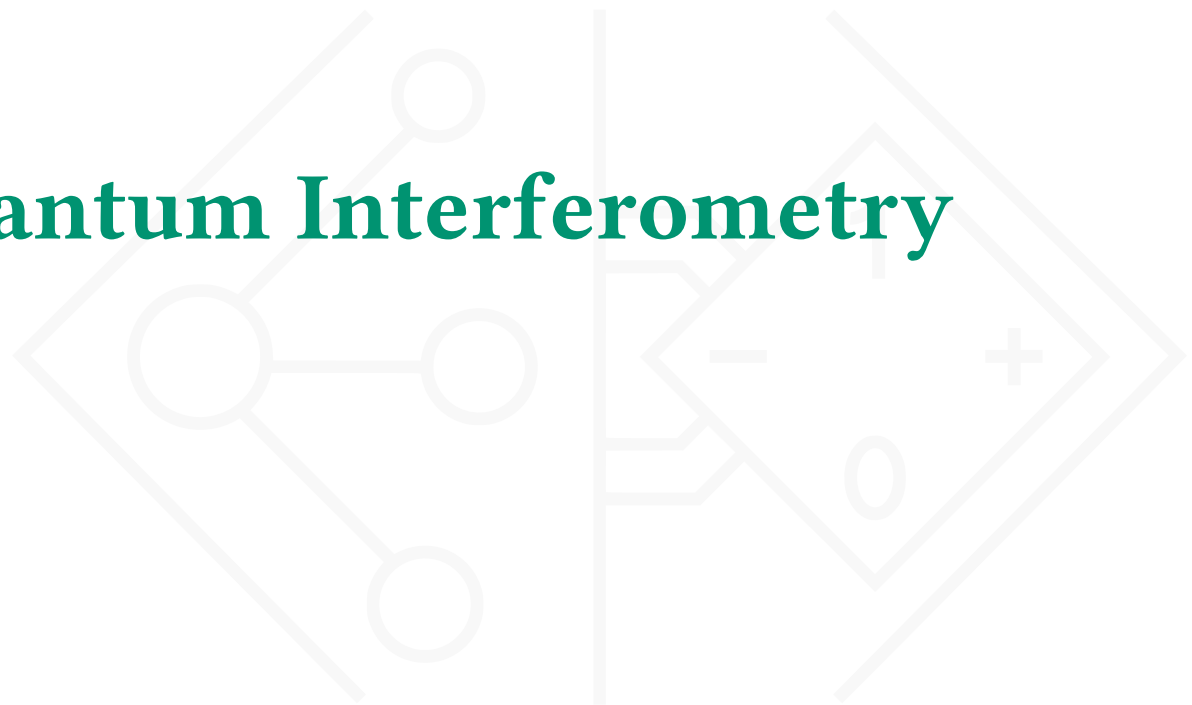
⇒ the heuristic stays close to optimal

Denser $\mathbf{H}$ (large ρ, γ)

⇒ more short cycles

⇒ BP performs worse

Decoded Quantum Interferometry



Max-LINSAT

m linear equations over n variables modulo p

Max-LINSAT

m linear equations over n variables modulo p

Goal: Satisfy as many equations as possible

Max-LINSAT

m linear equations over n variables modulo p

Goal: Satisfy as many equations as possible

Example:

$$2x_1 + 3x_2 + 4x_3 + 1x_4 = 1 \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 = 4 \bmod 5$$

$$1x_1 - 4x_2 = 4 \bmod 5$$

$$3x_2 - 2x_3 = 3 \bmod 5$$

$$x_3 - x_4 = 0 \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 = 2 \bmod 5$$

$$x_1, x_2, x_3, x_4 \in \{0, 1, 2, 3, 4\}$$

$$2x_1 + 3x_2 + 4x_3 + 1x_4 = 1 \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 = 4 \bmod 5$$

$$1x_1 - 4x_2 = 4 \bmod 5$$

$$3x_2 - 2x_3 = 3 \bmod 5$$

$$x_3 - x_4 = 0 \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 = 2 \bmod 5$$

$$2x_1 + 3x_2 + 4x_3 + 1x_4 = 1 \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 = 4 \bmod 5$$

$$1x_1 - 4x_2 = 4 \bmod 5$$

$$3x_2 - 2x_3 = 3 \bmod 5$$

$$x_3 - x_4 = 0 \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 = 2 \bmod 5$$

$$\begin{pmatrix} 2 & 3 & 4 & 5 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 3 & 3 & 0 \\ 0 & 0 & 1 & 4 \\ 1 & 2 & 3 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} \stackrel{\text{max}}{=} \begin{pmatrix} 1 \\ 4 \\ 4 \\ 3 \\ 0 \\ 2 \end{pmatrix} \bmod 5$$

$$2x_1 + 3x_2 + 4x_3 + 1x_4 = 1 \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 = 4 \bmod 5$$

$$1x_1 - 4x_2 = 4 \bmod 5$$

$$3x_2 - 2x_3 = 3 \bmod 5$$

$$x_3 - x_4 = 0 \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 = 2 \bmod 5$$

$$\begin{pmatrix} 2 & 3 & 4 & 5 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 3 & 3 & 0 \\ 0 & 0 & 1 & 4 \\ 1 & 2 & 3 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} \stackrel{\max}{=} \begin{pmatrix} 1 \\ 4 \\ 4 \\ 3 \\ 0 \\ 2 \end{pmatrix} \bmod 5$$

$$\Downarrow \\ Bx \stackrel{\max}{=} v$$

$$2x_1 + 3x_2 + 4x_3 + 1x_4 = 1 \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 = 4 \bmod 5$$

$$1x_1 - 4x_2 = 4 \bmod 5$$

$$3x_2 - 2x_3 = 3 \bmod 5$$

$$x_3 - x_4 = 0 \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 = 2 \bmod 5$$

$$\begin{pmatrix} 2 & 3 & 4 & 5 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 3 & 3 & 0 \\ 0 & 0 & 1 & 4 \\ 1 & 2 & 3 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} \stackrel{\text{max}}{=} \begin{pmatrix} 1 \\ 4 \\ 4 \\ 3 \\ 0 \\ 2 \end{pmatrix} \bmod 5$$

$$\Downarrow \\ Bx \stackrel{\text{max}}{=} v$$

n : number of variables, m : number of equations

$$2x_1 + 3x_2 + 4x_3 + 1x_4 = 1 \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 = 4 \bmod 5$$

$$1x_1 - 4x_2 = 4 \bmod 5$$

$$3x_2 - 2x_3 = 3 \bmod 5$$

$$x_3 - x_4 = 0 \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 = 2 \bmod 5$$

$$\begin{pmatrix} 2 & 3 & 4 & 5 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 3 & 3 & 0 \\ 0 & 0 & 1 & 4 \\ 1 & 2 & 3 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} \stackrel{\text{max}}{=} \begin{pmatrix} 1 \\ 4 \\ 4 \\ 3 \\ 0 \\ 2 \end{pmatrix} \bmod 5$$

$$\Downarrow \\ Bx \stackrel{\text{max}}{=} v$$

n : number of variables, m : number of equations $m > n$

$$2x_1 + 3x_2 + 4x_3 + 1x_4 = 1 \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 = 4 \bmod 5$$

$$1x_1 - 4x_2 = 4 \bmod 5$$

$$3x_2 - 2x_3 = 3 \bmod 5$$

$$x_3 - x_4 = 0 \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 = 2 \bmod 5$$

$$Bx \stackrel{\max}{=} v$$

$$2x_1 + 3x_2 + 4x_3 + 1x_4 \in \{1, 2, 3\} \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 \in \{2, 3, 4\} \bmod 5$$

$$1x_1 - 4x_2 \in \{0, 2, 4\} \bmod 5$$

$$3x_2 - 2x_3 \in \{0, 3, 4\} \bmod 5$$

$$x_3 - x_4 \in \{1, 2, 4\} \bmod 5$$

$$1x_1 + 2x_2 + 3x_3 + 4x_4 \in \underbrace{\{1, 2, 3\}}_{F_i} \bmod 5$$

$$Bx \stackrel{\max}{\in} F$$

$$2x_1 + 3x_2 + 4x_3 + 1x_4 \in \{1, 2, 3\} \bmod 5$$

$$x_1 + x_2 + x_3 + x_4 \in \{2, 3, 4\} \bmod 5$$

$$1x_1 - 4x_2 \in \{0, 2, 4\} \bmod 5$$

$$3x_2 - 2x_3 \in \{0, 3, 4\} \bmod 5$$

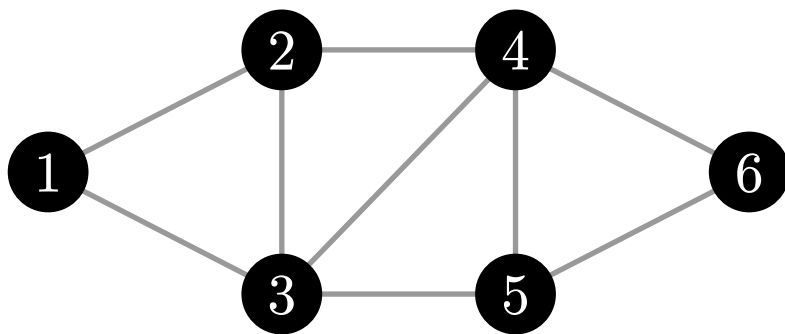
$$x_3 - x_4 \in \{1, 2, 4\} \bmod 5$$

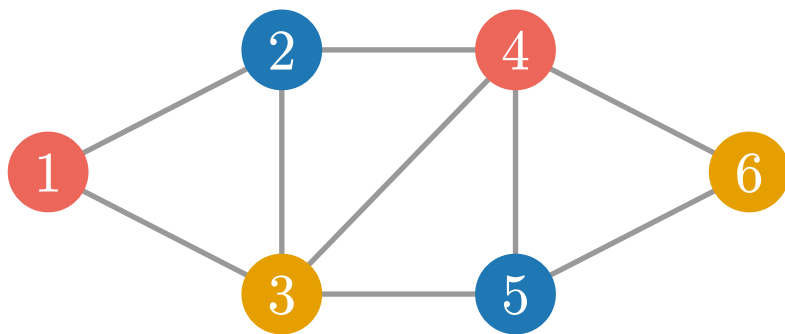
$$1x_1 + 2x_2 + 3x_3 + 4x_4 \in \underbrace{\{1, 2, 3\}}_{F_i} \bmod 5$$

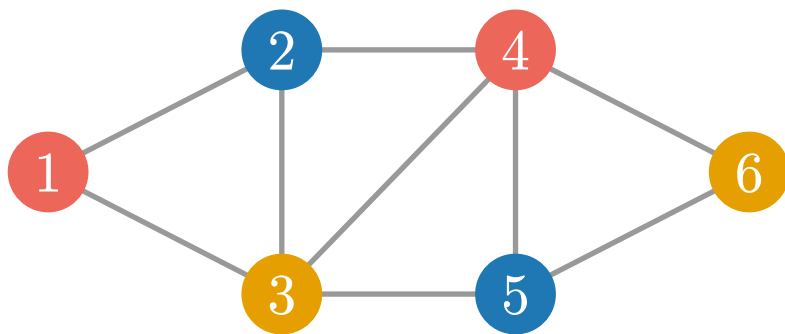
$$Bx \stackrel{\max}{\in} F$$

All F_i have the same size

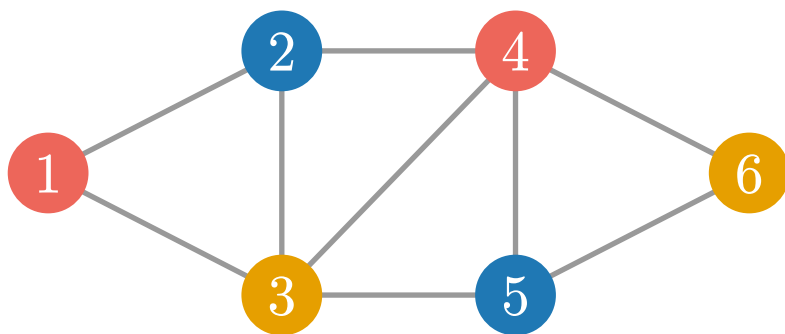
$$|F_1| = |F_2| = \dots = |F_m|$$





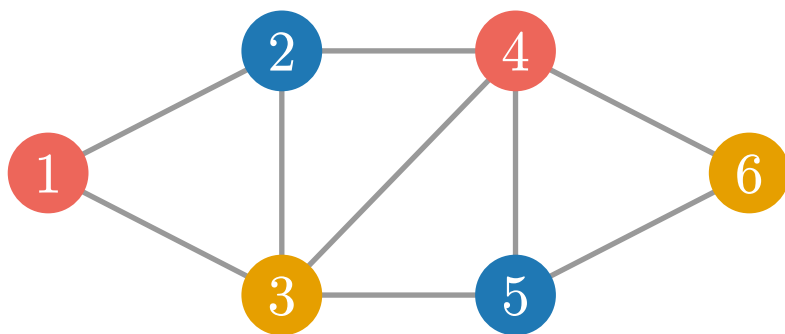


3-COLOR is NP-complete



3-COLOR is NP-complete

$$x_1, x_2, x_3, x_4, x_5, x_6 \in \{\textcolor{red}{0}, \textcolor{blue}{1}, \textcolor{yellow}{2}\}$$



3-COLOR is NP-complete

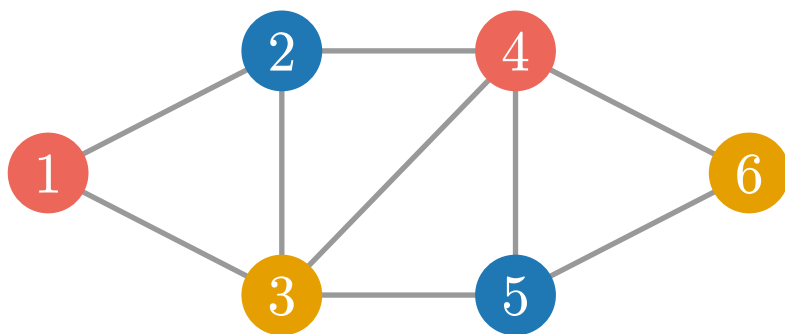
$$x_1, x_2, x_3, x_4, x_5, x_6 \in \{0, 1, 2\}$$

$$x_1 \neq x_2 \bmod 3$$

$$x_1 \neq x_3 \bmod 3$$

$$\vdots$$

$$x_5 \neq x_6 \bmod 3$$



3-COLOR is NP-complete

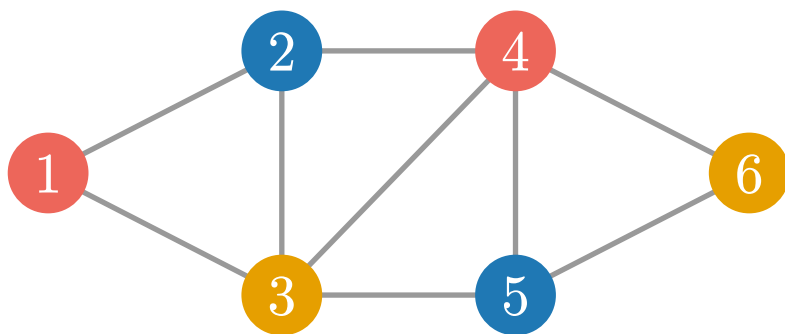
$$x_1, x_2, x_3, x_4, x_5, x_6 \in \{0, 1, 2\}$$

$$x_1 - x_2 \neq 0 \bmod 3$$

$$x_1 - x_3 \neq 0 \bmod 3$$

$$\vdots$$

$$x_5 - x_6 \neq 0 \bmod 3$$



3-COLOR is NP-complete

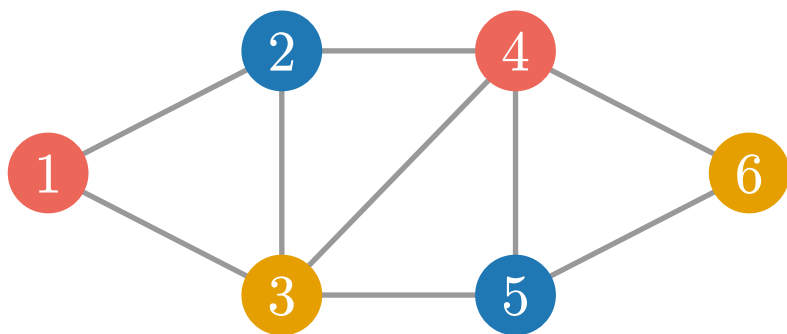
$$x_1, x_2, x_3, x_4, x_5, x_6 \in \{0, 1, 2\}$$

$$x_1 - x_2 \in \{1, 2\} \bmod 3$$

$$x_1 - x_3 \in \{1, 2\} \bmod 3$$

$$\vdots$$

$$x_5 - x_6 \in \{1, 2\} \bmod 3$$



3-COLOR is NP-complete

$$x_1, x_2, x_3, x_4, x_5, x_6 \in \{0, 1, 2\}$$

$$x_1 + 2x_2 \in \{1, 2\} \bmod 3$$

$$x_1 + 2x_3 \in \{1, 2\} \bmod 3$$

$$\vdots$$

$$x_5 + 2x_6 \in \{1, 2\} \bmod 3$$

$$\max_{Bx \in F}$$

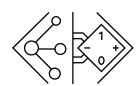
$$f_i(x) = \begin{cases} +1 & \text{if } b_i \cdot x \in F_i \\ -1 & \text{if } b_i \cdot x \notin F_i \end{cases}$$

$$f(x) = \sum_{i=1}^m f_i(x)$$

$$\max_{Bx \in F}$$

$$f_i(x) = \begin{cases} +1 & \text{if } b_i \cdot x \in F_i \\ -1 & \text{if } b_i \cdot x \notin F_i \end{cases}$$

$$f(x) = \sum_{i=1}^m f_i(x) = \# \text{ satisfied} - \# \text{ not satisfied}$$



Decoded Quantum Interferometry

Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

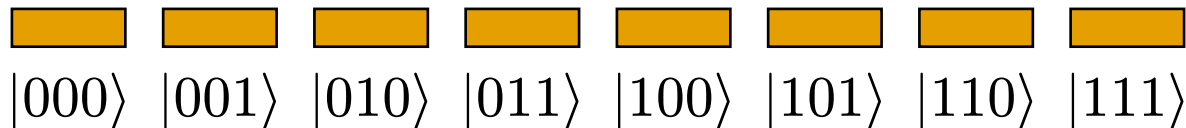
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x |x\rangle$$



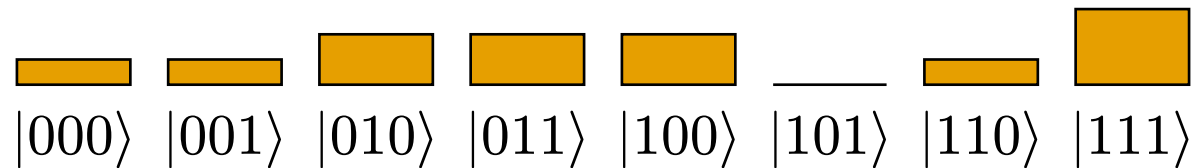
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x) |x\rangle$$



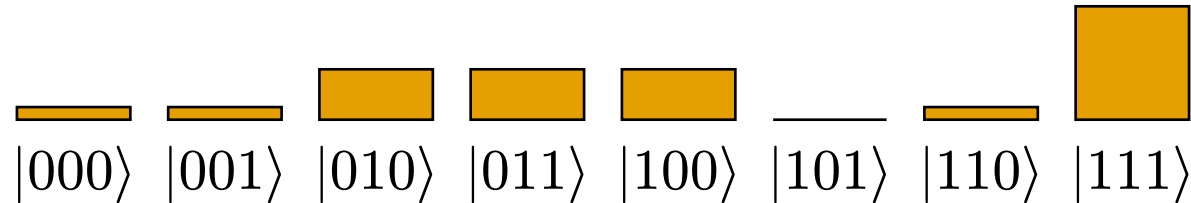
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^2 |x\rangle$$



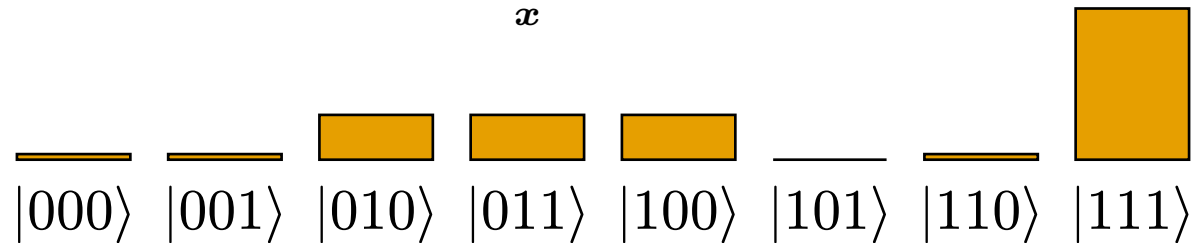
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^3 |x\rangle$$



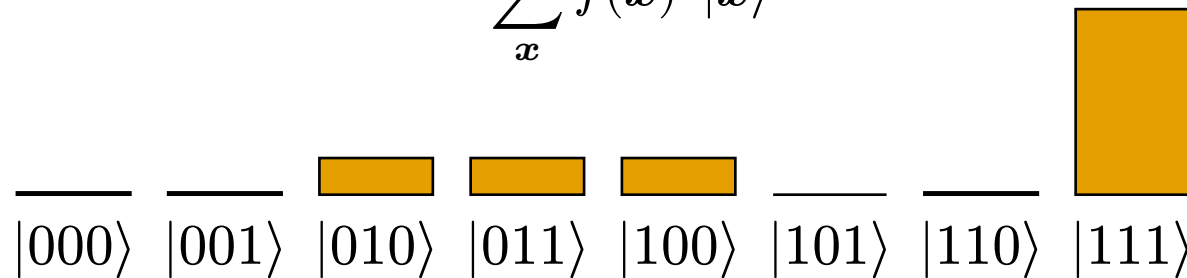
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^4 |x\rangle$$

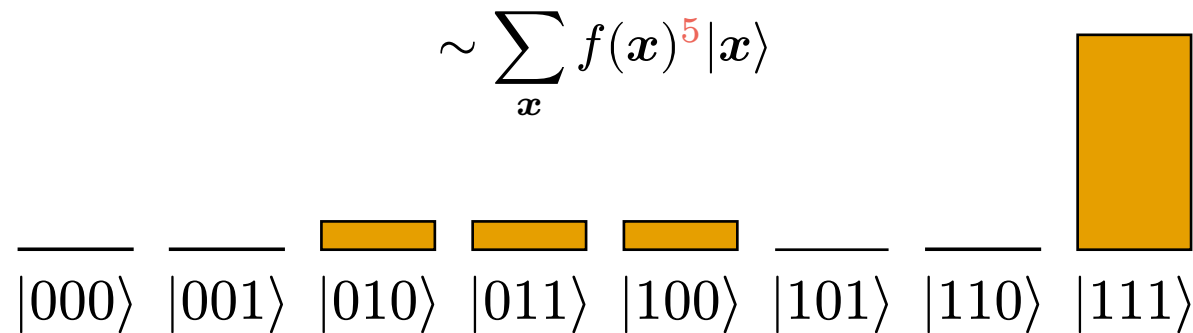


Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution



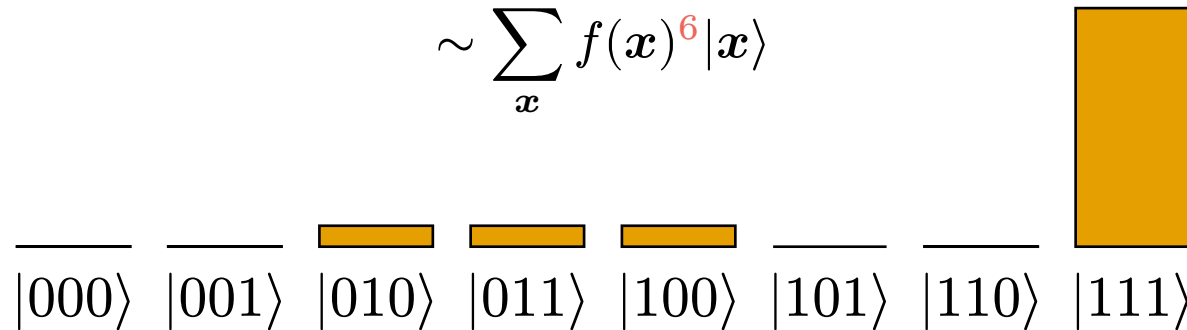
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^6 |x\rangle$$



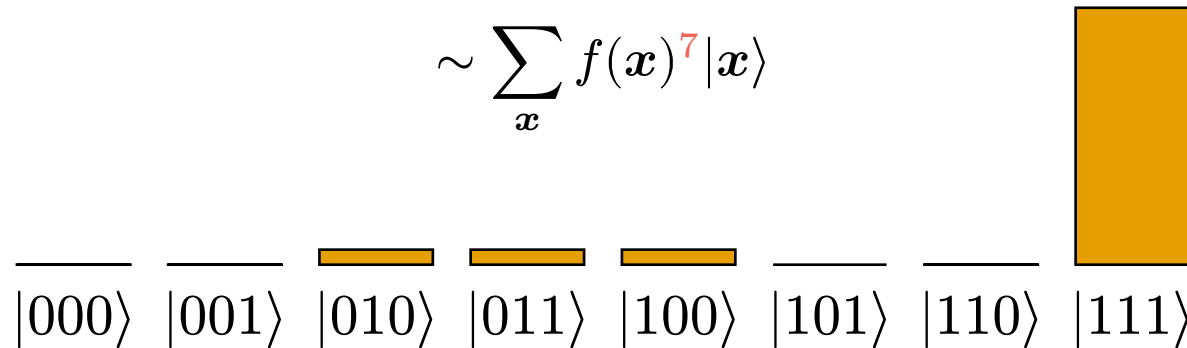
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^7 |x\rangle$$



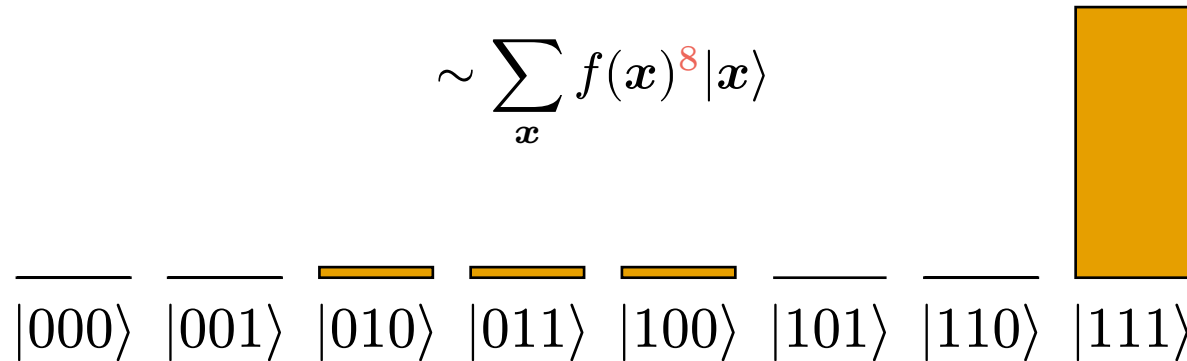
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^8 |x\rangle$$



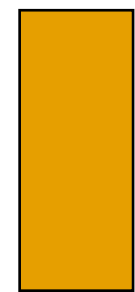
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^9 |x\rangle$$

							
$ 000\rangle$	$ 001\rangle$	$ 010\rangle$	$ 011\rangle$	$ 100\rangle$	$ 101\rangle$	$ 110\rangle$	$ 111\rangle$

Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x f(x)^{10} |x\rangle$$

$|000\rangle$ $|001\rangle$ $|010\rangle$ $|011\rangle$ $|100\rangle$ $|101\rangle$ $|110\rangle$ $|111\rangle$

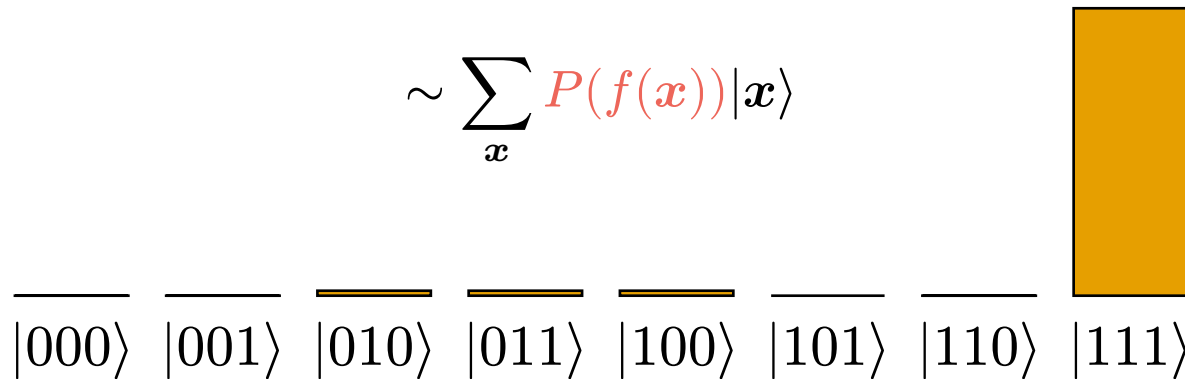
Decoded Quantum Interferometry

Idea:

Create superposition over all solutions,
better solution gets higher amplitude

Measurement: $\Rightarrow$ Likely a good solution

$$\sim \sum_x P(f(x)) |x\rangle$$



Goal: prepare $|\psi\rangle \sim \sum_{x \in \mathbb{F}_p^n} P(f(x)) |x\rangle$ for Max-LINSAT

Problem

1

$$Bx = 0$$

binary, $v = 0$



6

$$Bx = v$$

phases $(-1)^{v \cdot y}$



7

$$\mathbb{F}_2 \rightarrow \mathbb{F}_p$$



8

$$(Bx)_i \in F_i$$

Amplitude

2

$$f(x)$$



3

$$f(x)^2$$



4

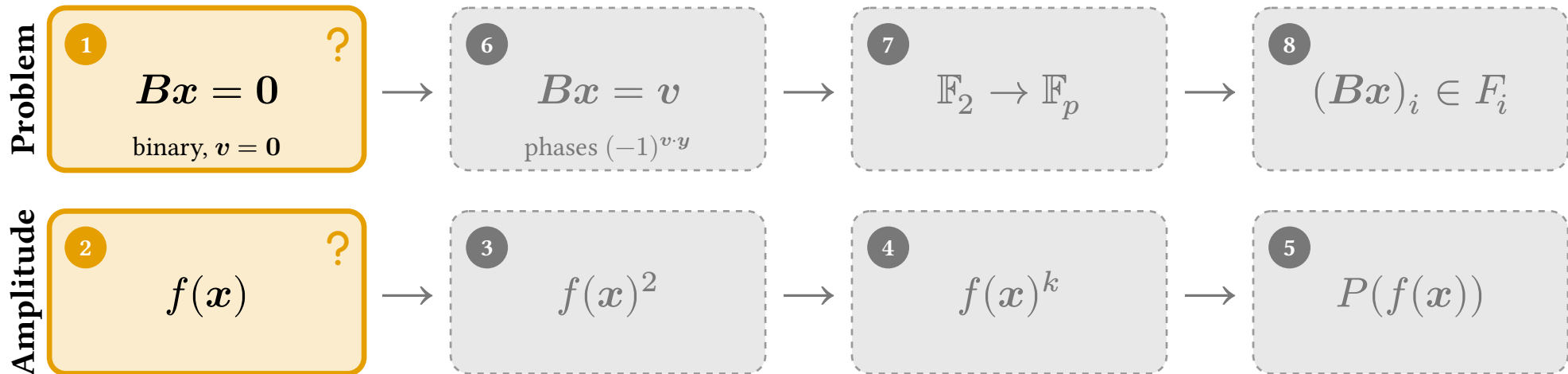
$$f(x)^k$$



5

$$P(f(x))$$

Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT



Back to basics: Hadamard gate on n -qubit systems

$$|0, 1\rangle \xrightarrow{H_1}$$

The Hadamard gate

$$H : \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

Back to basics: Hadamard gate on n -qubit systems

$$|0, 1\rangle \xrightarrow{H_1} \frac{1}{\sqrt{2}}|0, 1\rangle + \frac{1}{\sqrt{2}}|1, 1\rangle$$

The Hadamard gate

$$H: \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

Back to basics: Hadamard gate on n -qubit systems

$$\begin{aligned} |0, 1\rangle &\xrightarrow{H_1} \frac{1}{\sqrt{2}}|0, 1\rangle + \frac{1}{\sqrt{2}}|1, 1\rangle \\ &\xrightarrow{H_2} \end{aligned}$$

The Hadamard gate

$$H: \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

Back to basics: Hadamard gate on n -qubit systems

$$\begin{aligned} |0, 1\rangle &\xrightarrow{H_1} \frac{1}{\sqrt{2}}|0, 1\rangle + \frac{1}{\sqrt{2}}|1, 1\rangle \\ &\xrightarrow{H_2} \frac{1}{2}|0, 0\rangle - \frac{1}{2}|0, 1\rangle + \frac{1}{2}|1, 0\rangle - \frac{1}{2}|1, 1\rangle \end{aligned}$$

The Hadamard gate

$$H: \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

$$\begin{aligned} |0, 1\rangle &\xrightarrow{H_1} \frac{1}{\sqrt{2}}|0, 1\rangle + \frac{1}{\sqrt{2}}|1, 1\rangle \\ &\xrightarrow{H_2} \frac{1}{2}|0, 0\rangle - \frac{1}{2}|0, 1\rangle + \frac{1}{2}|1, 0\rangle - \frac{1}{2}|1, 1\rangle \end{aligned}$$

The Hadamard gate

$$H: \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

2-Qubit Hadamard transform

$$|a_1, a_2\rangle \xrightarrow{H_1 H_2}$$

$$\begin{aligned}
 |0, 1\rangle &\xrightarrow{H_1} \frac{1}{\sqrt{2}}|0, 1\rangle + \frac{1}{\sqrt{2}}|1, 1\rangle \\
 &\xrightarrow{H_2} \frac{1}{2}|0, 0\rangle - \frac{1}{2}|0, 1\rangle + \frac{1}{2}|1, 0\rangle - \frac{1}{2}|1, 1\rangle
 \end{aligned}$$

The Hadamard gate

$$H: \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

2-Qubit Hadamard transform

$$|a_1, a_2\rangle \xrightarrow{H_1 H_2} \frac{1}{2} \left(|0, 0\rangle + (-1)^{a_2} |0, 1\rangle + (-1)^{a_1} |1, 0\rangle + (-1)^{a_1+a_2} |1, 1\rangle \right)$$

The Hadamard gate

$$H: \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

$$\begin{aligned} |0, 1\rangle &\xrightarrow{H_1} \frac{1}{\sqrt{2}}|0, 1\rangle + \frac{1}{\sqrt{2}}|1, 1\rangle \\ &\xrightarrow{H_2} \frac{1}{2}|0, 0\rangle - \frac{1}{2}|0, 1\rangle + \frac{1}{2}|1, 0\rangle - \frac{1}{2}|1, 1\rangle \end{aligned}$$

2-Qubit Hadamard transform

$$|a_1, a_2\rangle \xrightarrow{H_1 H_2} \frac{1}{2} \left(|0, 0\rangle + (-1)^{a_2} |0, 1\rangle + (-1)^{a_1} |1, 0\rangle + (-1)^{a_1+a_2} |1, 1\rangle \right)$$

n -Qubit Hadamard transform

$$\underbrace{|a_1, a_2, \dots, a_n\rangle}_{|\mathbf{a}\rangle} \xrightarrow{H_1 H_2 \dots H_n}$$

$$\begin{aligned}
 |0, 1\rangle &\xrightarrow{H_1} \frac{1}{\sqrt{2}}|0, 1\rangle + \frac{1}{\sqrt{2}}|1, 1\rangle \\
 &\xrightarrow{H_2} \frac{1}{2}|0, 0\rangle - \frac{1}{2}|0, 1\rangle + \frac{1}{2}|1, 0\rangle - \frac{1}{2}|1, 1\rangle
 \end{aligned}$$

The Hadamard gate

$$H: \begin{cases} |0\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ |1\rangle \mapsto \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{cases}$$

2-Qubit Hadamard transform

$$|a_1, a_2\rangle \xrightarrow{H_1 H_2} \frac{1}{2} \left(|0, 0\rangle + (-1)^{a_2} |0, 1\rangle + (-1)^{a_1} |1, 0\rangle + (-1)^{a_1+a_2} |1, 1\rangle \right)$$

n -Qubit Hadamard transform

$$\underbrace{|a_1, a_2, \dots, a_n\rangle}_{|\mathbf{a}\rangle} \xrightarrow{H_1 H_2 \dots H_n} \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x} \in \{0,1\}^n} (-1)^{\overbrace{\mathbf{a} \cdot \mathbf{x}}^{\text{inner product}}} |\mathbf{x}\rangle$$

$$\mathbf{B} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

b_i is i -th column of $\mathbf{B}$

i -th equation: $b_i \cdot x = 0 \bmod 2$

$$\mathbf{B} = \begin{pmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \vdots \\ \mathbf{b}_m \end{pmatrix}$$

$\mathbf{b}_i$ is i -th column of $\mathbf{B}$

i -th equation: $\mathbf{b}_i \cdot \mathbf{x} = 0 \bmod 2$

$$f(\mathbf{x}) = \sum_{i=1}^m \overbrace{(-1)^{\mathbf{b}_i \cdot \mathbf{x}}}^{\begin{cases} +1 & \text{if } \mathbf{b}_i \cdot \mathbf{x} = 0 \\ -1 & \text{if } \mathbf{b}_i \cdot \mathbf{x} = 1 \end{cases}}$$

$$B = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

b_i is i -th column of B

i -th equation: $b_i \cdot x = 0 \bmod 2$

$$f(x) = \sum_{i=1}^m \overbrace{(-1)^{b_i \cdot x}}^{= \begin{cases} +1 & \text{if } b_i \cdot x = 0 \\ -1 & \text{if } b_i \cdot x = 1 \end{cases}}$$

$$= \# \text{ satisfied} - \# \text{ not satisfied}$$

$$\mathbf{B} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

b_i is i -th column of $\mathbf{B}$

i -th equation: $b_i \cdot \mathbf{x} = 0 \bmod 2$

$$|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x}) |\mathbf{x}\rangle$$

$$f(\mathbf{x}) = \sum_{i=1}^m \overbrace{(-1)^{b_i \cdot \mathbf{x}}}^{\begin{cases} +1 & \text{if } b_i \cdot \mathbf{x} = 0 \\ -1 & \text{if } b_i \cdot \mathbf{x} = 1 \end{cases}}$$

$$= \# \text{ satisfied} - \# \text{ not satisfied}$$

$$\mathbf{B} = \begin{pmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \vdots \\ \mathbf{b}_m \end{pmatrix}$$

$\mathbf{b}_i$ is i -th column of $\mathbf{B}$

i -th equation: $\mathbf{b}_i \cdot \mathbf{x} = 0 \bmod 2$

$$f(\mathbf{x}) = \sum_{i=1}^m \overbrace{(-1)^{\mathbf{b}_i \cdot \mathbf{x}}}^{\begin{cases} +1 & \text{if } \mathbf{b}_i \cdot \mathbf{x} = 0 \\ -1 & \text{if } \mathbf{b}_i \cdot \mathbf{x} = 1 \end{cases}}$$

$$= \# \text{ satisfied} - \# \text{ not satisfied}$$

$$|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x}) |\mathbf{x}\rangle = \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} |\mathbf{x}\rangle$$

$$B = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

b_i is i -th column of B

i -th equation: $b_i \cdot x = 0 \bmod 2$

$$f(x) = \sum_{i=1}^m \overbrace{(-1)^{b_i \cdot x}}^{\begin{cases} +1 & \text{if } b_i \cdot x = 0 \\ -1 & \text{if } b_i \cdot x = 1 \end{cases}}$$

= # satisfied – # not satisfied

$$\begin{aligned} |\psi\rangle &\sim \sum_{x \in \mathbb{F}_2^n} f(x) |x\rangle = \sum_{x \in \mathbb{F}_2^n} \sum_{i=1}^m (-1)^{b_i \cdot x} |x\rangle \\ &= \sum_{i=1}^m \sum_{x \in \mathbb{F}_2^n} (-1)^{b_i \cdot x} |x\rangle \end{aligned}$$

$$B = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

b_i is i -th column of B

i -th equation: $b_i \cdot x = 0 \bmod 2$

$$f(x) = \sum_{i=1}^m \overbrace{(-1)^{b_i \cdot x}}^{= \begin{cases} +1 & \text{if } b_i \cdot x = 0 \\ -1 & \text{if } b_i \cdot x = 1 \end{cases}}$$

= # satisfied – # not satisfied

$$\begin{aligned} |\psi\rangle &\sim \sum_{x \in \mathbb{F}_2^n} f(x) |x\rangle = \sum_{x \in \mathbb{F}_2^n} \sum_{i=1}^m (-1)^{b_i \cdot x} |x\rangle \\ &= \sum_{i=1}^m \sum_{x \in \mathbb{F}_n^n} (-1)^{b_i \cdot x} |x\rangle \xleftarrow{H_1 H_2 \dots H_n} \sum_{i=1}^m |b_i\rangle \end{aligned}$$

Goal: prepare $|\psi\rangle \sim \sum_{x \in \mathbb{F}_p^n} P(f(x))|x\rangle$ for Max-LINSAT

Problem

1

$$Bx = 0$$

binary, $v = 0$



6

$$Bx = v$$

phases $(-1)^{v \cdot y}$



7

$$\mathbb{F}_2 \rightarrow \mathbb{F}_p$$



8

$$(Bx)_i \in F_i$$

Amplitude

2

$$f(x)$$



3

$$f(x)^2$$



4

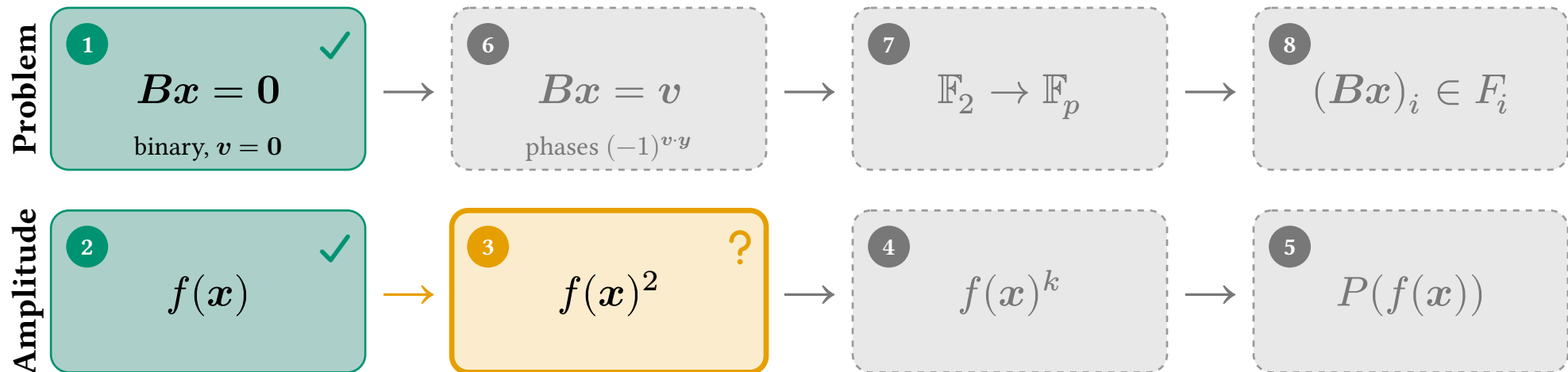
$$f(x)^k$$



5

$$P(f(x))$$

Goal: prepare $|\psi\rangle \sim \sum_{x \in \mathbb{F}_p^n} P(f(x)) |x\rangle$ for Max-LINSAT



$$|\psi\rangle \sim \sum_{x \in \mathbb{F}_2^n} f(x)^2 |x\rangle$$

$$\begin{aligned} |\psi\rangle &\sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^2 |\mathbf{x}\rangle \\ &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \left(\sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} \right) \left(\sum_{j=1}^m (-1)^{\mathbf{b}_j \cdot \mathbf{x}} \right) |\mathbf{x}\rangle \end{aligned}$$

$$\begin{aligned} |\psi\rangle &\sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^2 |\mathbf{x}\rangle \\ &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \left(\sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} \right) \left(\sum_{j=1}^m (-1)^{\mathbf{b}_j \cdot \mathbf{x}} \right) |\mathbf{x}\rangle \\ &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} |\mathbf{x}\rangle \end{aligned}$$

$$\begin{aligned}
 |\psi\rangle &\sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^2 |\mathbf{x}\rangle \\
 &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \left(\sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} \right) \left(\sum_{j=1}^m (-1)^{\mathbf{b}_j \cdot \mathbf{x}} \right) |\mathbf{x}\rangle \\
 &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} |\mathbf{x}\rangle
 \end{aligned}$$

$$\overbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ \color{red}{1} & \color{red}{1} & \color{red}{0} & \color{red}{0} \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ \color{red}{1} & \color{red}{0} & \color{red}{0} & \color{red}{1} \\ 0 & 1 & 0 & 1 \end{pmatrix}}^B \rightarrow \begin{matrix} \mathbf{b}_2 \\ \downarrow \\ \color{red}{\begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}} \end{matrix} + \begin{matrix} \mathbf{b}_5 \\ \downarrow \\ \color{red}{\begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}} \end{matrix}$$

$$\begin{aligned}
|\psi\rangle &\sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^2 |\mathbf{x}\rangle \\
&= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \left(\sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} \right) \left(\sum_{j=1}^m (-1)^{\mathbf{b}_j \cdot \mathbf{x}} \right) |\mathbf{x}\rangle \\
&= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} |\mathbf{x}\rangle \\
&\approx \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} |\mathbf{x}\rangle
\end{aligned}$$

$$\overbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}}^{\mathbf{B}} \rightarrow \begin{matrix} \mathbf{b}_2 \\ \downarrow \\ \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} \end{matrix} + \begin{matrix} \mathbf{b}_5 \\ \downarrow \\ \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} \end{matrix}$$

$$\begin{aligned}
 |\psi\rangle &\sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^2 |\mathbf{x}\rangle \\
 &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \left(\sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} \right) \left(\sum_{j=1}^m (-1)^{\mathbf{b}_j \cdot \mathbf{x}} \right) |\mathbf{x}\rangle \\
 &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} |\mathbf{x}\rangle \\
 &\approx \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} |\mathbf{x}\rangle
 \end{aligned}$$

$$\overbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ \color{red}{1} & \color{red}{1} & \color{red}{0} & \color{red}{0} \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ \color{red}{1} & \color{red}{0} & \color{red}{0} & \color{red}{1} \\ 0 & 1 & 0 & 1 \end{pmatrix}}^{\mathbf{B}} \rightarrow \begin{matrix} \mathbf{b}_2 \downarrow \\ \begin{pmatrix} \color{red}{1} \\ \color{red}{1} \\ \color{red}{0} \\ \color{red}{0} \end{pmatrix} \end{matrix} + \begin{matrix} \mathbf{b}_5 \downarrow \\ \begin{pmatrix} \color{red}{0} \\ \color{red}{1} \\ \color{red}{0} \\ \color{red}{1} \end{pmatrix} \end{matrix}$$

$$\overbrace{\begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}}^{\mathbf{B}^T} \begin{matrix} \mathbf{y} \downarrow \\ \begin{pmatrix} 0 \\ \color{red}{1} \\ 0 \\ 0 \\ \color{red}{1} \\ 0 \end{pmatrix} \end{matrix} = \begin{matrix} \mathbf{b}_2 \downarrow \\ \begin{pmatrix} \color{red}{1} \\ \color{red}{1} \\ \color{red}{0} \\ \color{red}{0} \end{pmatrix} \end{matrix} + \begin{matrix} \mathbf{b}_5 \downarrow \\ \begin{pmatrix} \color{red}{0} \\ \color{red}{1} \\ \color{red}{0} \\ \color{red}{1} \end{pmatrix} \end{matrix}$$

$$\begin{aligned}
|\psi\rangle &\sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^2 |\mathbf{x}\rangle \\
&= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \left(\sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} \right) \left(\sum_{j=1}^m (-1)^{\mathbf{b}_j \cdot \mathbf{x}} \right) |\mathbf{x}\rangle \\
&= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} |\mathbf{x}\rangle \\
&\approx \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} |\mathbf{x}\rangle \\
&= \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} \sum_{\mathbf{x} \in \mathbb{F}_2^n} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} |\mathbf{x}\rangle
\end{aligned}$$

$$\overbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}}^{\mathbf{B}} \rightarrow \begin{matrix} \mathbf{b}_2 \downarrow \\ \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} \end{matrix} + \begin{matrix} \mathbf{b}_5 \downarrow \\ \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} \end{matrix}$$

$$\overbrace{\begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}}^{\mathbf{B}^T} \begin{matrix} \mathbf{y} \downarrow \\ \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \end{matrix} = \begin{matrix} \mathbf{b}_2 \downarrow \\ \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} \end{matrix} + \begin{matrix} \mathbf{b}_5 \downarrow \\ \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} \end{matrix}$$

$$\begin{aligned}
 |\psi\rangle &\sim \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^2 |\mathbf{x}\rangle \\
 &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \left(\sum_{i=1}^m (-1)^{\mathbf{b}_i \cdot \mathbf{x}} \right) \left(\sum_{j=1}^m (-1)^{\mathbf{b}_j \cdot \mathbf{x}} \right) |\mathbf{x}\rangle \\
 &= \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} |\mathbf{x}\rangle \\
 &\approx \sum_{\mathbf{x} \in \mathbb{F}_2^n} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} |\mathbf{x}\rangle \\
 &= \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} \sum_{\mathbf{x} \in \mathbb{F}_2^n} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} |\mathbf{x}\rangle \xleftarrow{H_1 \dots H_n} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} |\mathbf{B}^T \mathbf{y}\rangle
 \end{aligned}$$

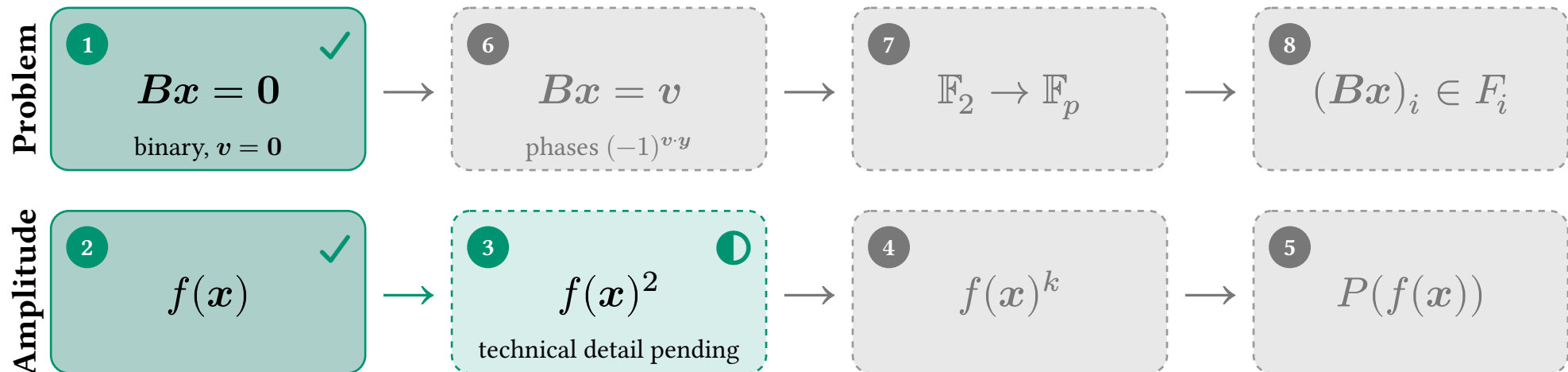
$$\overbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}}^B \rightarrow \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

$\downarrow \mathbf{b}_2$ $\downarrow \mathbf{b}_5$

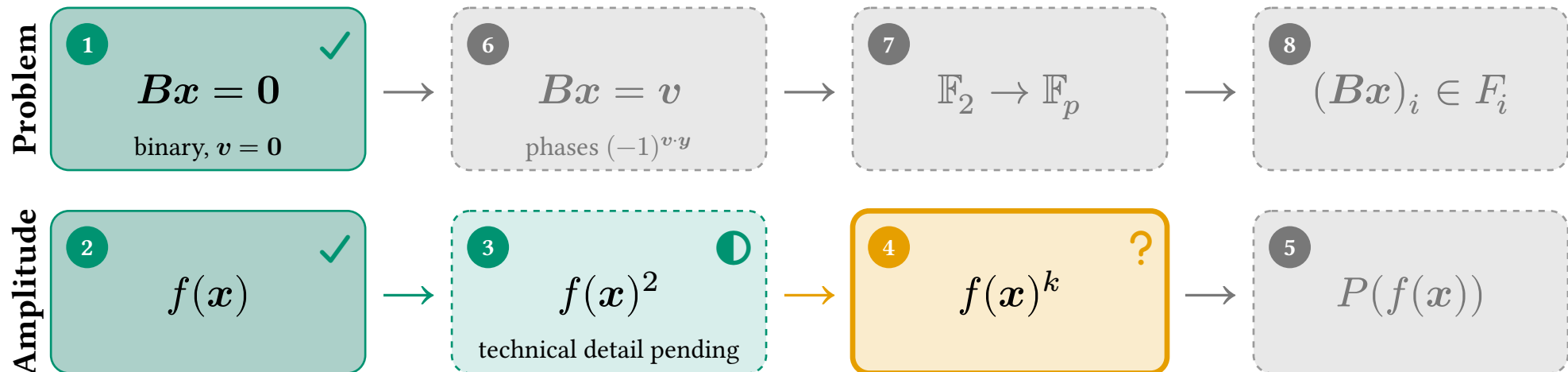
$$\overbrace{\begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}}^{B^T} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

$\downarrow \mathbf{y}$ $\downarrow \mathbf{b}_2$ $\downarrow \mathbf{b}_5$

Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT



Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT



$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$\longrightarrow \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \longrightarrow \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$\longrightarrow \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \longrightarrow \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$|0\rangle \longrightarrow \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \longrightarrow \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$|0\rangle \xrightarrow{\text{possible}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \longrightarrow \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$|0\rangle \xrightarrow{\text{possible}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \xrightarrow{?} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$|0\rangle \xrightarrow{\text{possible}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \xrightarrow{?} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$\overbrace{|\mathbf{y}\rangle}^m \xrightleftharpoons{\text{invertible}} \overbrace{|B^T \mathbf{y}\rangle}^n$$

$$|0\rangle \xrightarrow{\text{possible}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \xrightarrow{?} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$\overbrace{|\mathbf{y}\rangle}^m \xrightleftharpoons{\text{invertible}} \overbrace{|B^T \mathbf{y}\rangle}^n$$

Only possible if unique $\mathbf{y}$ with k ones exists.

$$|0\rangle \xrightarrow{\text{possible}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \xrightarrow{?} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

$$\overbrace{|\mathbf{y}\rangle}^m \xrightleftharpoons{\text{invertible}} \overbrace{|B^T \mathbf{y}\rangle}^n$$

Only possible if unique $\mathbf{y}$ with k ones exists.

Syndrome Decoding Problem

Find $\mathbf{y}$ with $\leq k$ ones and $\mathbf{H}\mathbf{y} = \mathbf{z}$

$$|0\rangle \xrightarrow{\text{possible}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle \xrightarrow{?} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{y}\rangle \xrightarrow{H_1 H_2 \dots H_n} \sum_{\mathbf{x}} f(\mathbf{x})^k |\mathbf{x}\rangle$$

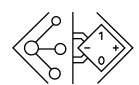
$$\overbrace{|\mathbf{y}\rangle}^m \xleftrightarrow{\text{invertible}} \overbrace{|B^T \mathbf{y}\rangle}^n$$

Only possible if unique $\mathbf{y}$ with k ones exists.

Syndrome Decoding Problem

Find $\mathbf{y}$ with $\leq k$ ones and $\mathbf{H}\mathbf{y} = \mathbf{z}$

Only possible if B^T is parity-check matrix of code that can correct k errors!



$$\overbrace{|\mathbf{0}\rangle}^m \overbrace{|\mathbf{0}\rangle}^n$$

$$\overbrace{|\mathbf{0}\rangle}^m \overbrace{|\mathbf{0}\rangle}^n \xrightarrow{\text{Dicke state}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |\mathbf{0}\rangle$$

$$\begin{aligned} \overbrace{|\mathbf{0}\rangle}^m \overbrace{|\mathbf{0}\rangle}^n &\xrightarrow{\text{Dicke state}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |\mathbf{0}\rangle \\ &\xrightarrow{\text{matrix multip.}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle \end{aligned}$$

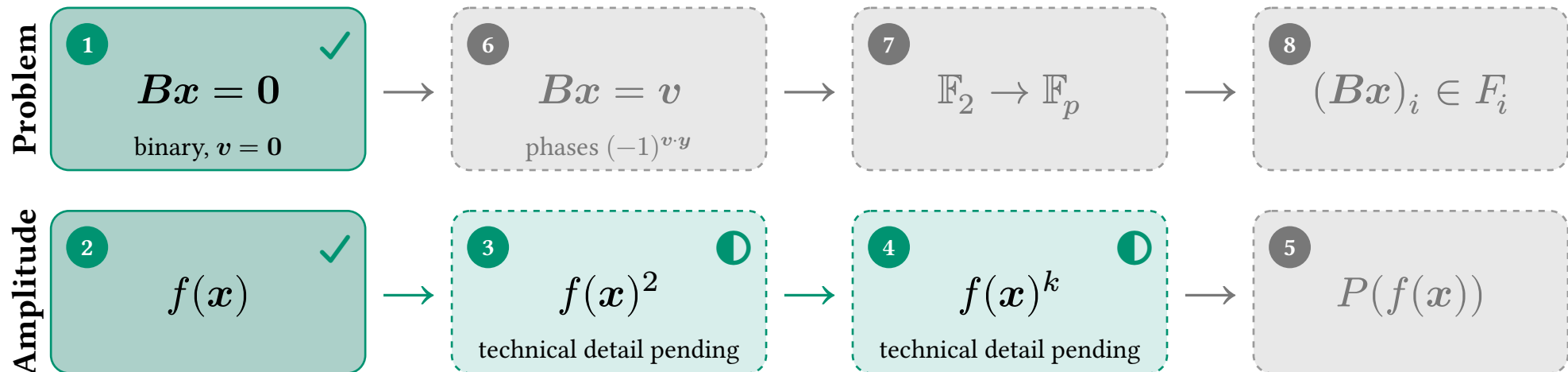
$$\overbrace{|\mathbf{0}\rangle}^m \overbrace{|\mathbf{0}\rangle}^n \xrightarrow{\text{Dicke state}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |\mathbf{0}\rangle$$

$$\xrightarrow{\text{matrix multip.}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle$$

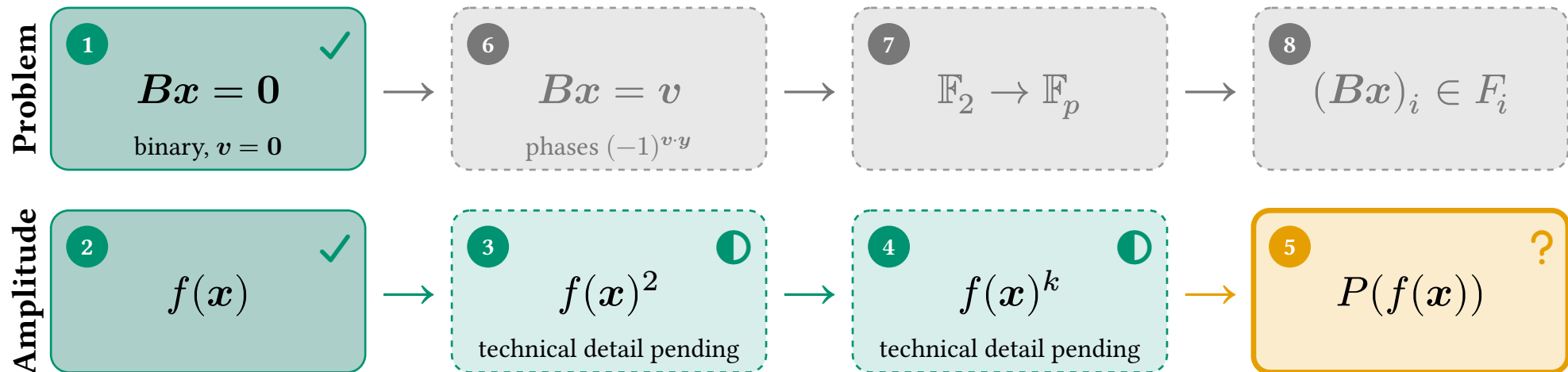
$$\xrightarrow{\text{Syndr.-Decod.}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{0}\rangle |B^T \mathbf{y}\rangle$$

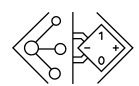
$$\begin{aligned}
 &\overbrace{|\mathbf{0}\rangle}^m \overbrace{|\mathbf{0}\rangle}^n \xrightarrow{\text{Dicke state}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |\mathbf{0}\rangle \\
 &\xrightarrow{\text{matrix multip.}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle \\
 &\xrightarrow{\text{Syndr.-Decod.}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{0}\rangle |B^T \mathbf{y}\rangle \\
 &\xrightarrow{H_1 \cdots H_n} \sum_{\mathbf{x} \in \mathbb{F}_2^n} f(\mathbf{x})^k |\mathbf{0}\rangle |\mathbf{x}\rangle
 \end{aligned}$$

Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT

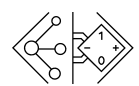


Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT





$$f(\boldsymbol{x})^2 =$$



$$f(\boldsymbol{x})^2 = \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\boldsymbol{b}_i + \boldsymbol{b}_j) \cdot \boldsymbol{x}}$$

$$f(\mathbf{x})^2 = \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}}$$

 $\rightsquigarrow$

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}}$$

$$f(\mathbf{x})^2 = \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}}$$

 $\rightsquigarrow$

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}}$$

$$f(\mathbf{x}) = \sum_{i=1}^m f_i \quad \text{with} \quad f_i = (-1)^{\mathbf{b}_i \cdot \mathbf{x}}$$

$$f(\mathbf{x})^2 = \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}}$$

 $\rightsquigarrow$

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} = f_1 f_2 + f_1 f_3 + \cdots + f_{m-1} f_m$$

$$f(\mathbf{x}) = \sum_{i=1}^m f_i \quad \text{with} \quad f_i = (-1)^{\mathbf{b}_i \cdot \mathbf{x}}$$

$$f(\mathbf{x})^2 = \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} = \underbrace{f_1^2}_1 + \dots + \underbrace{f_m^2}_1 + 2(f_1 f_2 + \dots f_{m-1} f_m)$$

⋈

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} = f_1 f_2 + f_1 f_3 + \dots + f_{m-1} f_m$$

$$f(\mathbf{x}) = \sum_{i=1}^m f_i \quad \text{with} \quad f_i = (-1)^{\mathbf{b}_i \cdot \mathbf{x}}$$

$$f(\mathbf{x})^2 = \sum_{i=1}^m \sum_{j=1}^m (-1)^{(\mathbf{b}_i + \mathbf{b}_j) \cdot \mathbf{x}} = \underbrace{f_1^2}_1 + \dots + \underbrace{f_m^2}_1 + 2(f_1 f_2 + \dots f_{m-1} f_m)$$

$$\frac{f(\mathbf{x})^2 - m}{2} = \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with 2 ones}}} (-1)^{(\mathbf{B}^T \mathbf{y}) \cdot \mathbf{x}} = f_1 f_2 + f_1 f_3 + \dots + f_{m-1} f_m$$

$$f(\mathbf{x}) = \sum_{i=1}^m f_i \quad \text{with} \quad f_i = (-1)^{\mathbf{b}_i \cdot \mathbf{x}}$$

What DQI prepares

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{B}^T \mathbf{x}\rangle \longrightarrow \sum_{\mathbf{x} \in \mathbb{F}_2^n} f^{(k)}(\mathbf{x}) |\mathbf{x}\rangle$$

What DQI prepares

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{x}\rangle \longrightarrow \sum_{\mathbf{x} \in \mathbb{F}_2^n} f^{(k)}(\mathbf{x}) |\mathbf{x}\rangle$$

$$f^{(k)} = \underbrace{f_1 f_2 \cdots f_k + f_1 f_2 \cdots f_{k+1} + \cdots + f_{m-k+1} \cdots f_m}_{\text{all } \binom{n}{k} \text{ combinations}}$$

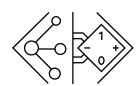
What DQI prepares

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |B^T \mathbf{x}\rangle \longrightarrow \sum_{\mathbf{x} \in \mathbb{F}_2^n} f^{(k)}(\mathbf{x}) |\mathbf{x}\rangle$$

General polynomials

$$P(f(\mathbf{x})) = \sum_{k=0}^m w_k f^{(k)}(\mathbf{x})$$

$$f^{(k)} = \underbrace{f_1 f_2 \cdots f_k + f_1 f_2 \cdots f_{k+1} + \cdots + f_{m-k+1} \cdots f_m}_{\text{all } \binom{n}{k} \text{ combinations}}$$



$$\underbrace{\log(m)}_{|0\rangle} \underbrace{m}_{|0\rangle} \underbrace{n}_{|0\rangle}$$

$$\overbrace{|0\rangle}^{\log(m)} \overbrace{|0\rangle}^m \overbrace{|0\rangle}^n \xrightarrow{\text{State prep.}} \sum_{k=0}^{\ell} w_k |k\rangle |0\rangle |0\rangle$$

$$\begin{array}{c} \log(m) \\ \underbrace{\quad} \\ |0\rangle \end{array} \begin{array}{c} m \\ \underbrace{\quad} \\ |0\rangle \end{array} \begin{array}{c} n \\ \underbrace{\quad} \\ |0\rangle \end{array} \xrightarrow{\text{State prep.}} \sum_{k=0}^{\ell} w_k |k\rangle |0\rangle |0\rangle \xrightarrow{\text{Dicke state}} \sum_{k=0}^{\ell} w_k |k\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle$$

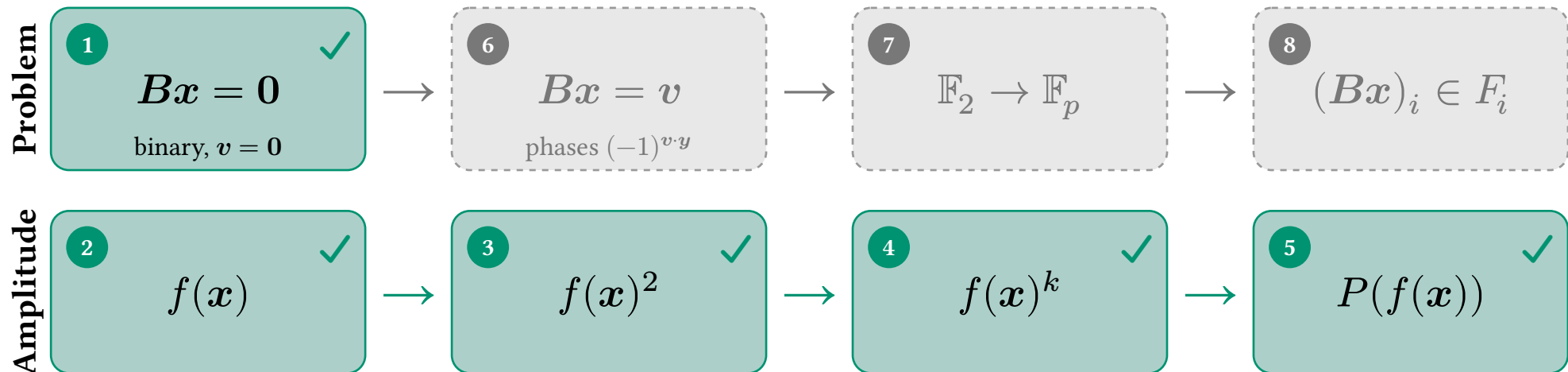
$$\begin{aligned}
 & \overbrace{|0\rangle}^{\log(m)} \overbrace{|0\rangle}^m \overbrace{|0\rangle}^n \xrightarrow{\text{State prep.}} \sum_{k=0}^{\ell} w_k |k\rangle |0\rangle |0\rangle \xrightarrow{\text{Dicke state}} \sum_{k=0}^{\ell} w_k |k\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{uncomp. } |k\rangle} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle
 \end{aligned}$$

$$\begin{aligned}
 & \underbrace{\log(m)}_{|0\rangle} \underbrace{m}_{|0\rangle} \underbrace{n}_{|0\rangle} \xrightarrow{\text{State prep.}} \sum_{k=0}^{\ell} w_k |k\rangle |0\rangle |0\rangle \xrightarrow{\text{Dicke state}} \sum_{k=0}^{\ell} w_k |k\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{uncomp. } |k\rangle} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{matrix multip.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle
 \end{aligned}$$

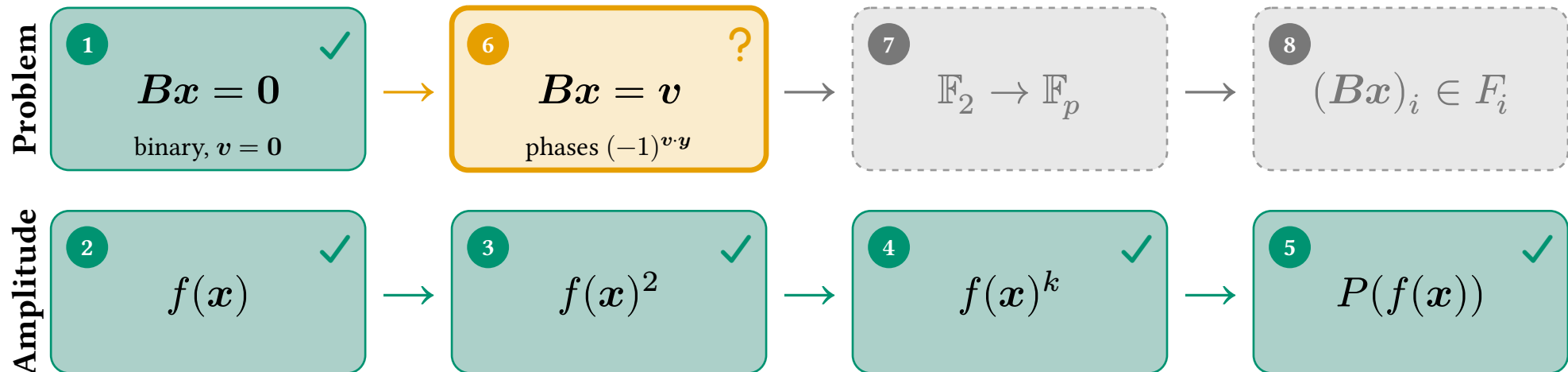
$$\begin{aligned}
 & \overbrace{|0\rangle}^{\log(m)} \overbrace{|0\rangle}^m \overbrace{|0\rangle}^n \xrightarrow{\text{State prep.}} \sum_{k=0}^{\ell} w_k |k\rangle |0\rangle |0\rangle \xrightarrow{\text{Dicke state}} \sum_{k=0}^{\ell} w_k |k\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{uncomp. } |k\rangle} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{matrix multip.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle \xrightarrow{\text{Syndr.-Decod.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |0\rangle |B^T \mathbf{y}\rangle
 \end{aligned}$$

$$\begin{aligned}
 & \underbrace{\log(m)}_{|0\rangle} \underbrace{m}_{|0\rangle} \underbrace{n}_{|0\rangle} \xrightarrow{\text{State prep.}} \sum_{k=0}^{\ell} w_k |k\rangle |0\rangle |0\rangle \xrightarrow{\text{Dicke state}} \sum_{k=0}^{\ell} w_k |k\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\substack{\text{uncomp.} \\ |k\rangle}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{matrix multip.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle \xrightarrow{\text{Syndr.-Decod.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |0\rangle |B^T \mathbf{y}\rangle \\
 & \xrightarrow{H_1 \cdots H_n} \sum_{\mathbf{x} \in \mathbb{F}_2^n} P(f(\mathbf{x})) |0\rangle |0\rangle |\mathbf{x}\rangle
 \end{aligned}$$

Goal: prepare $|\psi\rangle \sim \sum_{x \in \mathbb{F}_p^n} P(f(x)) |x\rangle$ for Max-LINSAT



Goal: prepare $|\psi\rangle \sim \sum_{x \in \mathbb{F}_p^n} P(f(x)) |x\rangle$ for Max-LINSAT

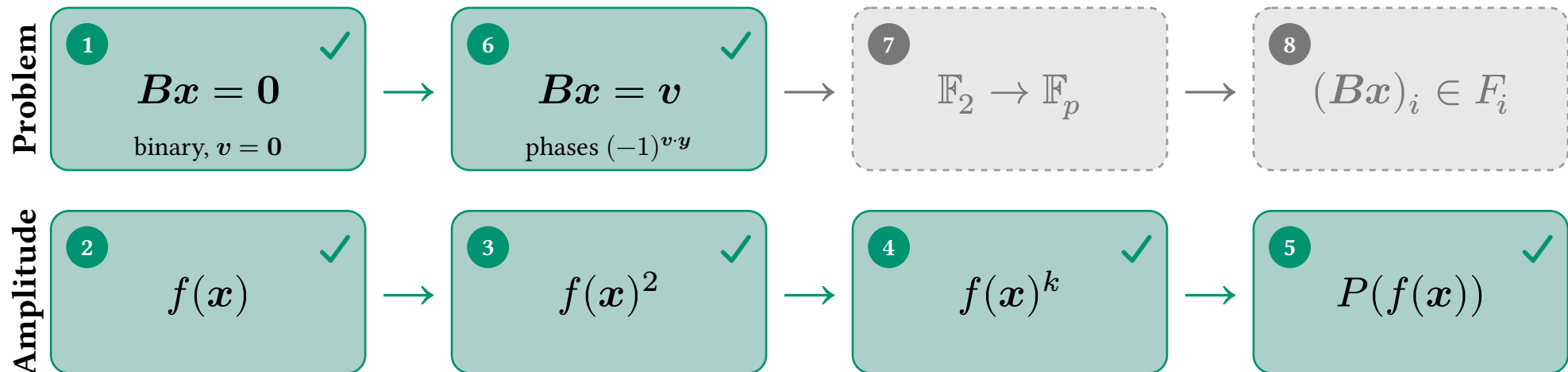


$$Bx = 0$$

$$Bx = v$$

$$\begin{aligned}
 & \overbrace{|0\rangle}^{\log(m)} \overbrace{|0\rangle}^m \overbrace{|0\rangle}^n \xrightarrow{\text{State prep.}} \sum_{k=0}^{\ell} w_k |k\rangle |0\rangle |0\rangle \xrightarrow{\text{Dicke state}} \sum_{k=0}^{\ell} w_k |k\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{uncomp.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |0\rangle \xrightarrow{\substack{Z^{(i)}|\mathbf{y}\rangle \\ \text{for } v_i=1}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} (-1)^{\mathbf{v} \cdot \mathbf{y}} |\mathbf{y}\rangle |0\rangle \\
 & \xrightarrow{\text{matrix multip.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle (-1)^{\mathbf{v} \cdot \mathbf{y}} |B^T \mathbf{y}\rangle \xrightarrow{\text{Syndr.-Decod.}} \sum_{k=0}^{\ell} w_k |0\rangle \sum_{\substack{\mathbf{y} \in \mathbb{F}_2^m \\ \text{with } k \text{ ones}}} |0\rangle (-1)^{\mathbf{v} \cdot \mathbf{y}} |B^T \mathbf{y}\rangle \\
 & \xrightarrow{H_1 \dots H_n} \sum_{\mathbf{x} \in \mathbb{F}_2^n} P(f(\mathbf{x})) |0\rangle |0\rangle |\mathbf{x}\rangle
 \end{aligned}$$

Goal: prepare $|\psi\rangle \sim \sum_{x \in \mathbb{F}_p^n} P(f(x)) |x\rangle$ for Max-LINSAT





Exercise 3

From a graph to a Max-XORSAT instance and through DQI

Your task

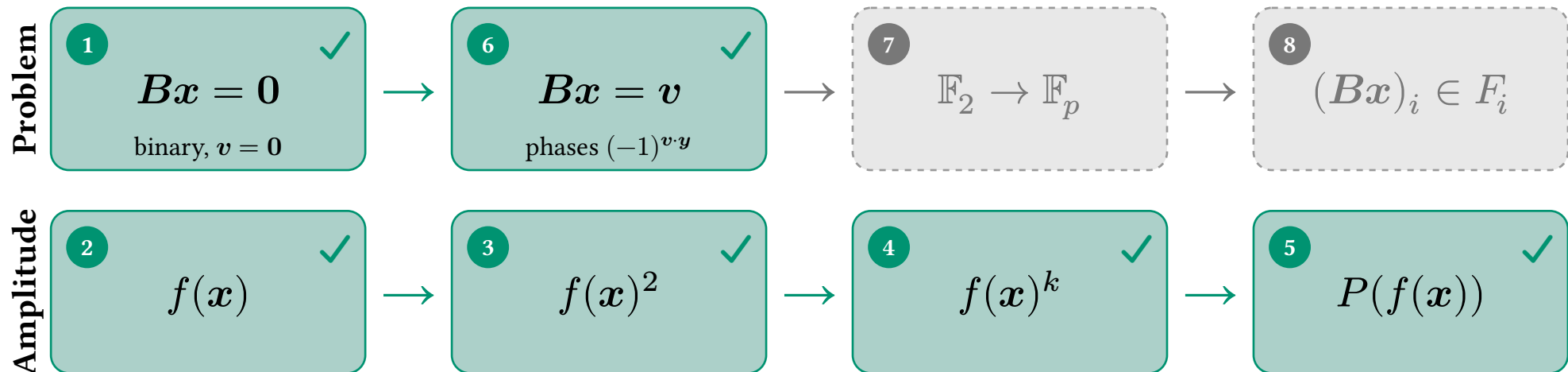
1. Encode the graph from the previous slide as Max-XORSAT: one constraint $x_u + x_v = 1$ per edge. Look at B and v
2. Run DQI with $\ell = 1$ and compare with the maximum cut (brute force) and with random guessing
3. Remove the two edges that close triangles, $(2, 3)$ and $(4, 5)$: how do $\Delta_{\min}$ and the DQI estimate change?

DQI-Kit

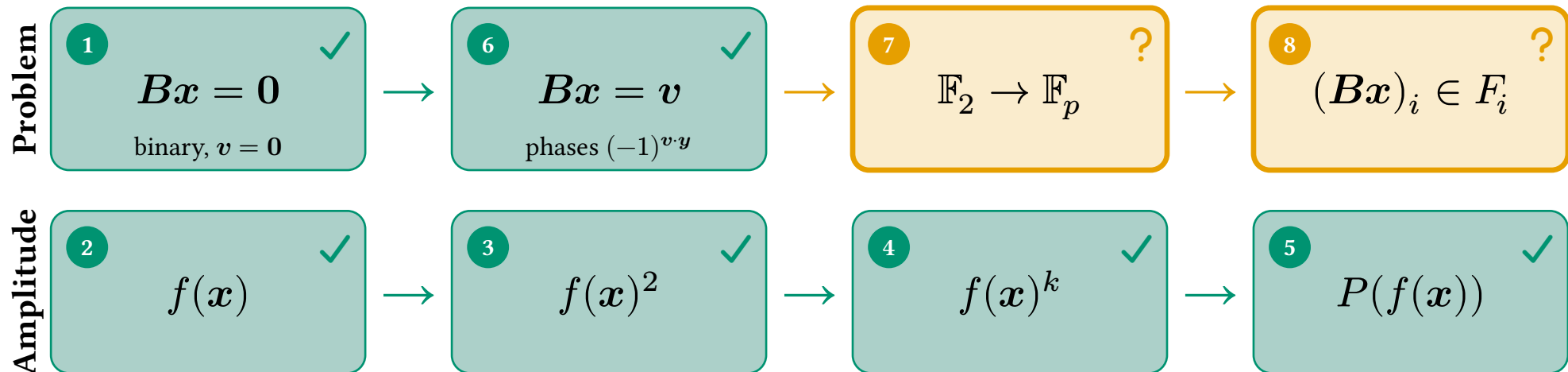
```
G = nx.Graph([(1, 2), (1, 3), (2, 3), ...])
prob = MaxXorSat()
x = {u: prob.new_var(f"x_{u}") for u in G}
for u, v in G.edges:
    prob.add_constraint(x[u] + x[v] == 1)

prob.get_B(), prob.get_F(), prob.get_minimum_distance()
Dqi(prob).estimate_solution_quality(1)
BruteForceSolver(prob).get_solution_quality()
```

Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT



Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT



Max-XORSAT

$$\mathbb{F}_2 = \{0, 1\}$$

$$\underbrace{\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_B \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_x = \underbrace{\begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}}_v$$

Max-XORSAT

$$\mathbb{F}_2 = \{0, 1\}$$

$$\underbrace{\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_B \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_x = \underbrace{\begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}}_v$$

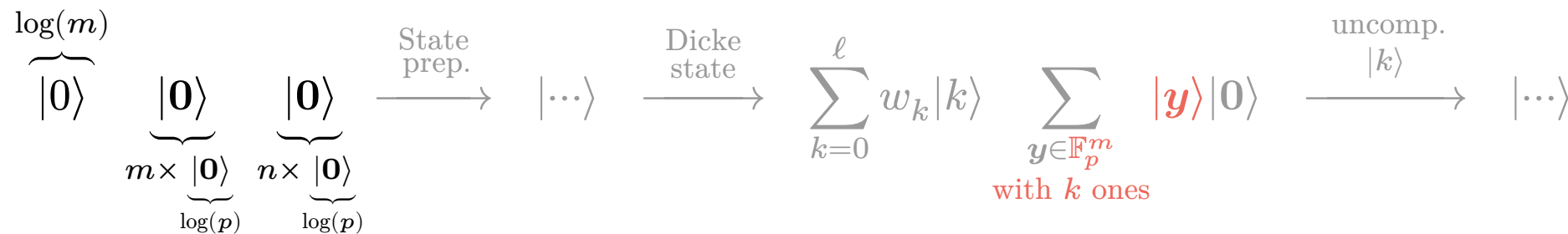
Max-LINSAT

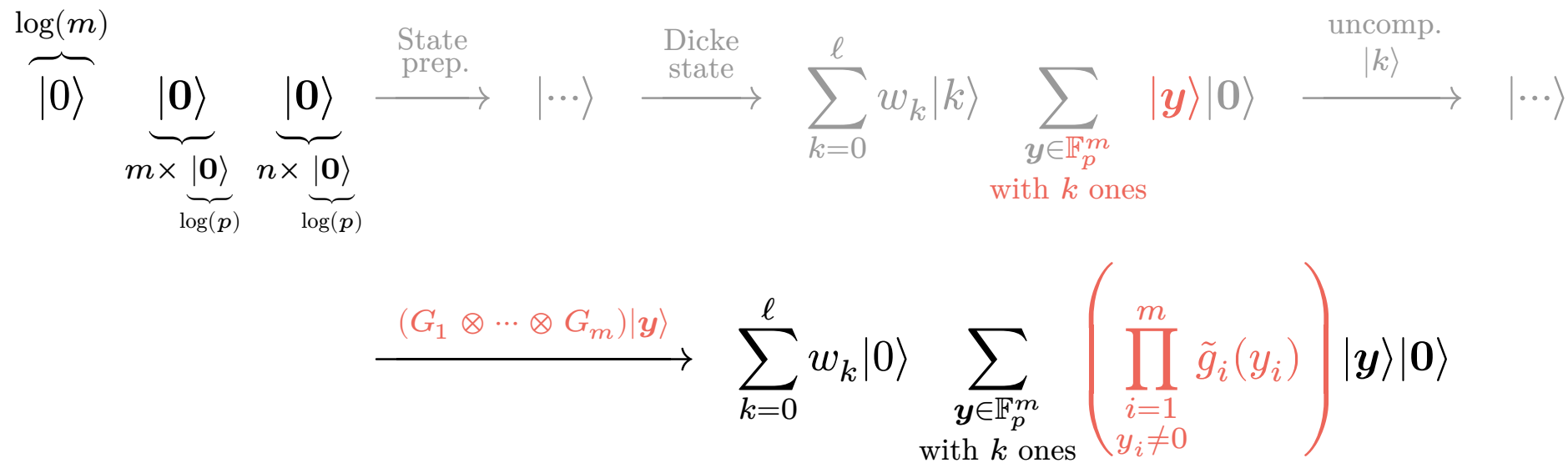
$$\mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$$

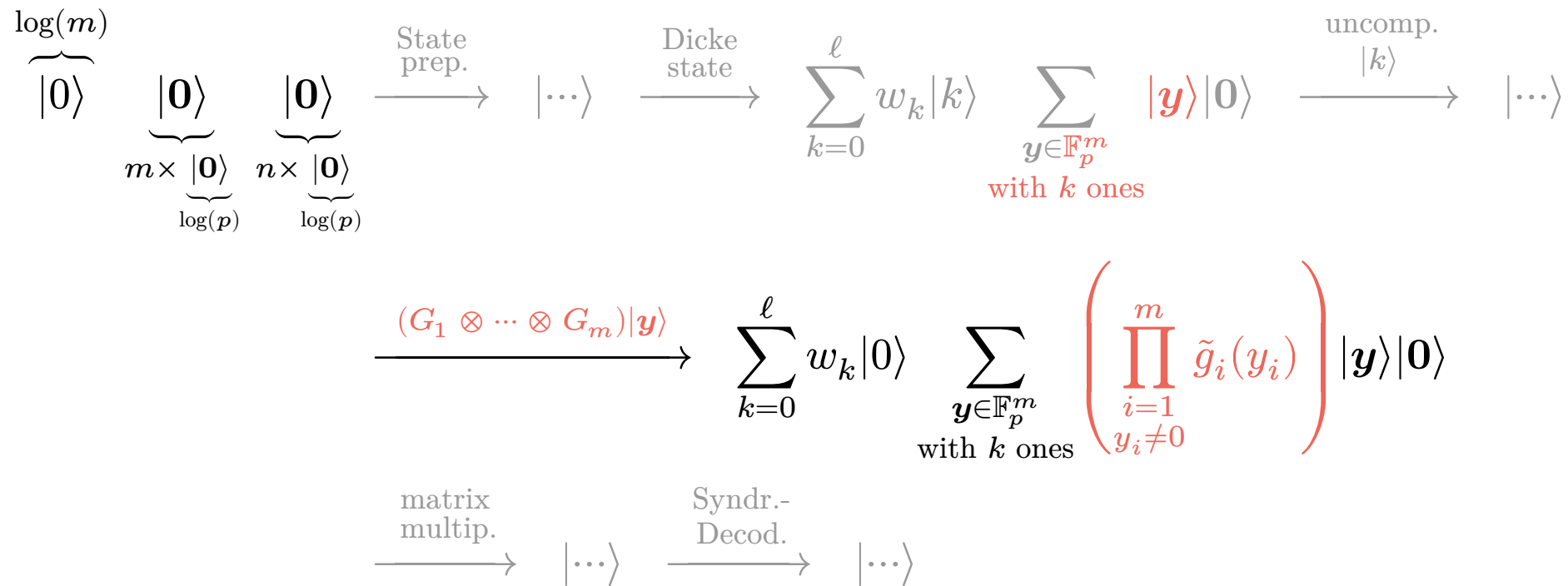
$$\underbrace{\begin{pmatrix} 1 & 2 & 3 \\ -1 & 1 & -1 \\ 0 & -2 & 0 \\ 4 & 0 & 1 \end{pmatrix}}_B \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_x = \underbrace{\begin{pmatrix} \{0, 1, 2, \dots\} \\ \{2, 4, 6, \dots\} \\ \{0, -1, 1, \dots\} \\ \{2, -2, 7, \dots\} \end{pmatrix}}_{F_1, F_2, \dots, F_m}$$

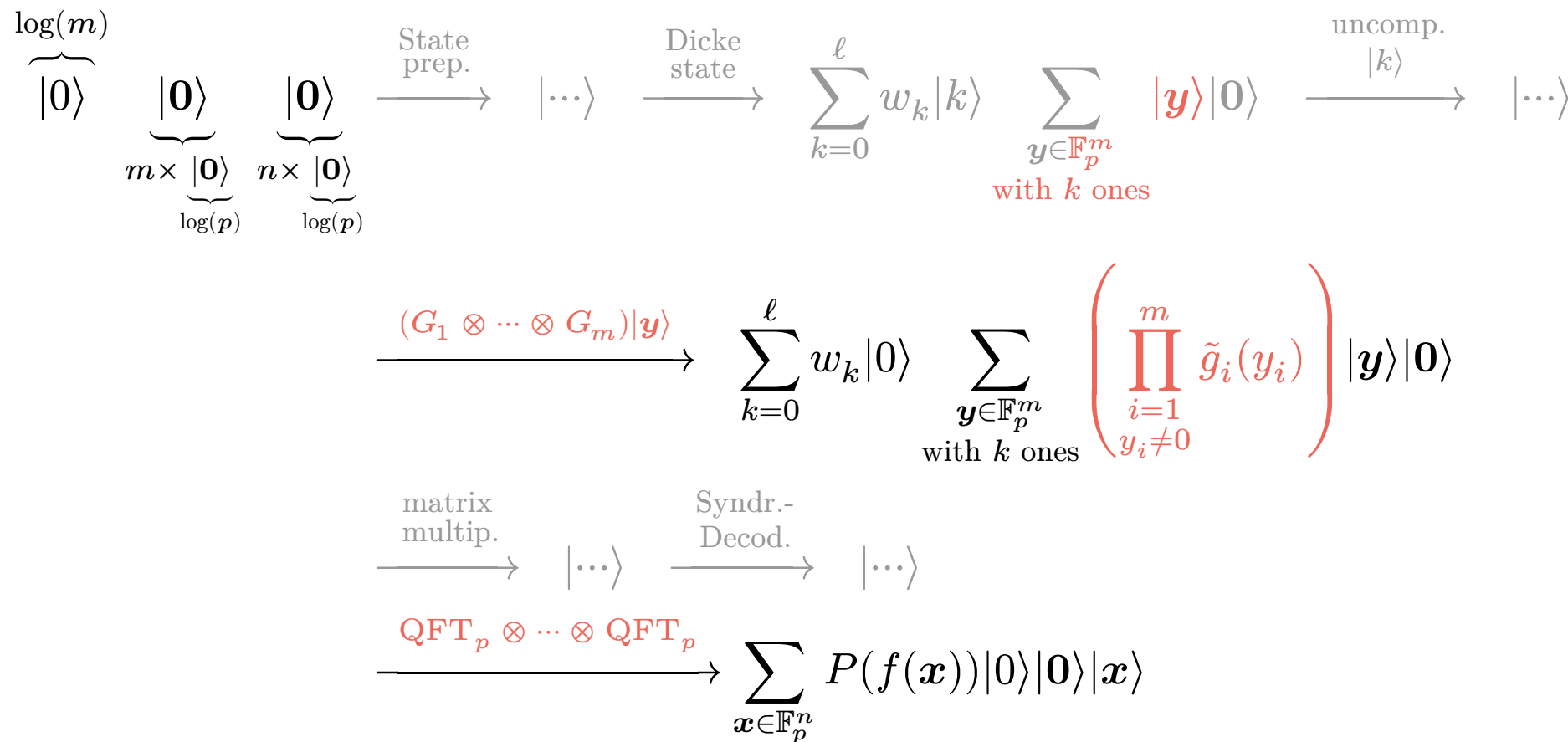
$$|F_1| = |F_2| = \dots = |F_m| = r$$

$$\begin{array}{c}
 \log(m) \\
 \underbrace{\quad} \\
 |0\rangle \\
 \underbrace{\quad} \\
 m \times \underbrace{|0\rangle}_{\log(p)} \quad n \times \underbrace{|0\rangle}_{\log(p)}
 \end{array}$$









$$f_i(y) = \begin{cases} -1 & \text{if } y \notin F_i \\ +1 & \text{if } y \in F_i \end{cases} \xrightarrow[\varphi = \left(\sum_y |f_i(y) - \bar{f}|^2 \right)^{\frac{1}{2}}]{\bar{f} = \frac{1}{p} \sum_y f_i(y)} g_i(y) = \frac{f_i(y) - \bar{f}}{\varphi}$$

$$f_i(y) = \begin{cases} -1 & \text{if } y \notin F_i \\ +1 & \text{if } y \in F_i \end{cases} \xrightarrow[\varphi = \left(\sum_y |f_i(y) - \bar{f}|^2 \right)^{\frac{1}{2}}]{\bar{f} = \frac{1}{p} \sum_y f_i(y)} g_i(y) = \frac{f_i(y) - \bar{f}}{\varphi}$$

$\tilde{g}_i$: Fourier transform of g_i

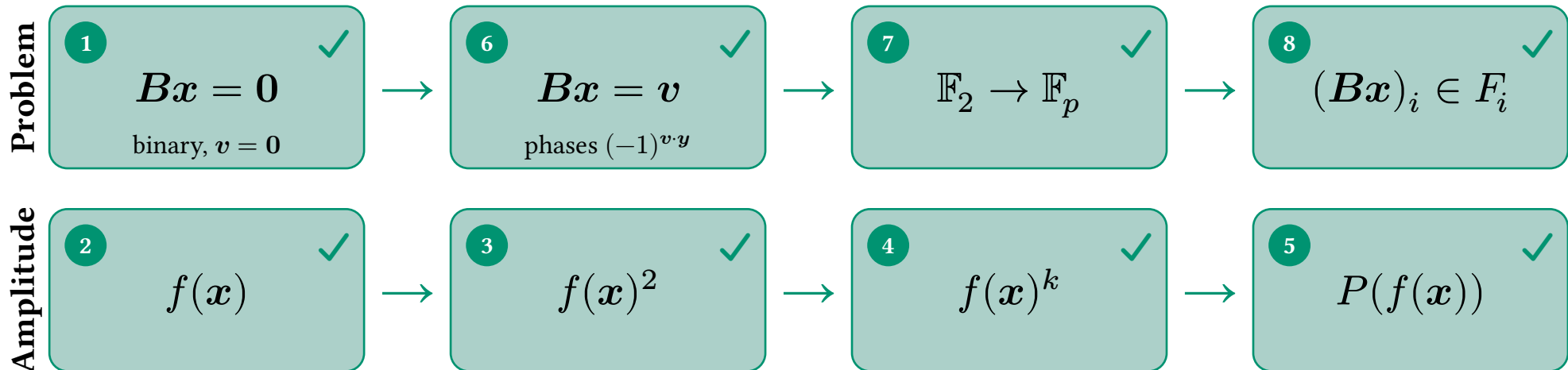
$$f_i(y) = \begin{cases} -1 & \text{if } y \notin F_i \\ +1 & \text{if } y \in F_i \end{cases} \xrightarrow[\varphi = \left(\sum_y |f_i(y) - \bar{f}|^2 \right)^{\frac{1}{2}}]{\bar{f} = \frac{1}{p} \sum_y f_i(y)} g_i(y) = \frac{f_i(y) - \bar{f}}{\varphi}$$

$\tilde{g}_i$: Fourier transform of g_i

Write: $P(f(x)) = \sum_{k=0}^{\ell} g^{(k)}$ with

$$g^{(k)} = \underbrace{g_1 g_2 \cdots g_k + g_1 g_2 \cdots g_{k+1} + \cdots + g_{m-k+1} \cdots g_m}_{\text{all } \binom{n}{k} \text{ combinations}}$$

Goal: prepare $|\psi\rangle \sim \sum_{\mathbf{x} \in \mathbb{F}_p^n} P(f(\mathbf{x})) |\mathbf{x}\rangle$ for Max-LINSAT



Decoded Quantum Interferometry

Given:

m equations of the form

$$(B_{i1}x_1 + B_{i2}x_2 + \dots + B_{in}x_n \bmod p) \in F_i$$

with $F_i \subseteq \mathbb{F}_p$ and $|F_1| = |F_2| = \dots = |F_m|$

Decoded Quantum Interferometry

Given:

m equations of the form

$$(B_{i1}x_1 + B_{i2}x_2 + \dots + B_{in}x_n \bmod p) \in F_i$$

with $F_i \subseteq \mathbb{F}_p$ and $|F_1| = |F_2| = \dots = |F_m|$

as well as polynomial P :

$$P(z) = a_0 + a_1z + a_2z^2 + \dots + a_kz^k.$$

Decoded Quantum Interferometry

Given:

m equations of the form

$$(B_{i1}x_1 + B_{i2}x_2 + \dots + B_{in}x_n \bmod p) \in F_i$$

with $F_i \subseteq \mathbb{F}_p$ and $|F_1| = |F_2| = \dots = |F_m|$
as well as polynomial P :

$$P(z) = a_0 + a_1z + a_2z^2 + \dots + a_kz^k.$$

Create state:

$$\sim \sum_{\mathbf{x} \in \mathbb{F}_p} P(f(\mathbf{x})) |\mathbf{x}\rangle,$$

where $f(\mathbf{x})$ = number of satisfied equations.

Decoded Quantum Interferometry

Given:

m equations of the form

$$(B_{i1}x_1 + B_{i2}x_2 + \dots + B_{in}x_n \bmod p) \in F_i$$

with $F_i \subseteq \mathbb{F}_p$ and $|F_1| = |F_2| = \dots = |F_m|$
as well as polynomial P :

$$P(z) = a_0 + a_1z + a_2z^2 + \dots + a_kz^k.$$

Create state:

$$\sim \sum_{x \in \mathbb{F}_p} P(f(x)) |x\rangle,$$

where $f(x)$ = number of satisfied equations.

Requirements:

The code with parity-check matrix B^T can correct at least k errors.

Decoded Quantum Interferometry

Given:

m equations of the form

$$(B_{i1}x_1 + B_{i2}x_2 + \dots + B_{in}x_n \bmod p) \in F_i$$

with $F_i \subseteq \mathbb{F}_p$ and $|F_1| = |F_2| = \dots = |F_m|$
as well as polynomial P :

$$P(z) = a_0 + a_1z + a_2z^2 + \dots + a_kz^k.$$

Create state:

$$\sim \sum_{x \in \mathbb{F}_p} P(f(x)) |x\rangle,$$

where $f(x)$ = number of satisfied equations.

Requirements:

The code with parity-check matrix B^T can correct at least k errors.

There is an efficient closed form for the solution quality of DQI!



Exercise 4

Same graph, three colours: from Max-XORSAT to Max-LINSAT with set constraints

Your task

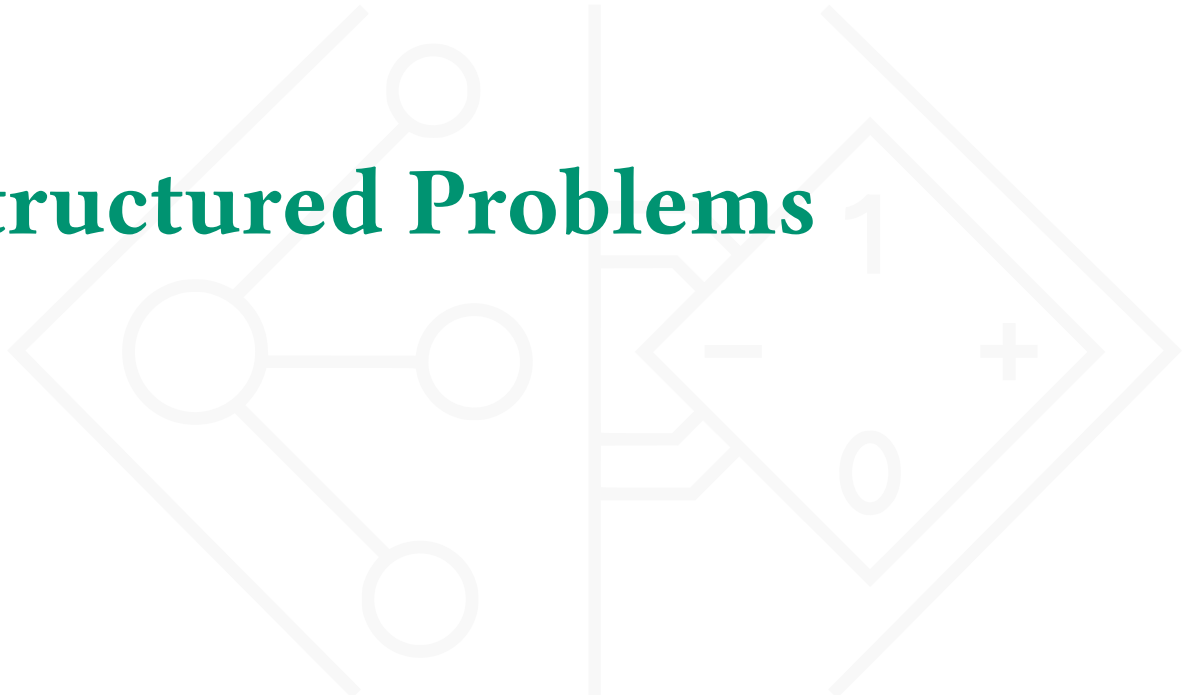
1. Switch your Max-Cut code to $\mathbb{F}_3$ and $\neq$ constraints. Look at the sets F_i
2. Run DQI: how many edges are properly coloured in expectation? Compare with the optimum
3. Add edges until the graph is no longer 3-colourable: what does DQI return now?

DQI-Kit

```
prob = MaxLinSat(GF(3))
x = {u: prob.new_var(f"x_{u}") for u in G}
for u, v in G.edges:
    prob.add_constraint(x[u] != x[v])    # F_i = {1, 2}

prob.get_F()
Dqi(prob).estimate_solution_quality(1)
BruteForceSolver(prob).get_solution_quality()
VertexColor(3, G)    # the built-in version
```


DQI on Structured Problems

A faint, light gray watermark is visible in the background. It consists of a circuit board layout on the left with several circular components and connecting lines, and a diamond-shaped logic symbol on the right. The diamond symbol contains a minus sign (-), a plus sign (+), and the numbers 1 and 0.

DQI approx. ratio (asymptotic behavior)

$$\frac{\langle S \rangle}{m} = \left(\sqrt{\frac{\ell}{m} \left(1 - \frac{r}{p} \right)} + \sqrt{\frac{r}{p} \left(1 - \frac{\ell}{m} \right)} \right)^2$$

for optimal polynomial P in the limit $m \rightarrow \infty$ with fixed $\frac{\ell}{m}$

$\langle S \rangle$ number of satisfied constraints

m total number of constraints

ℓ degree of polynomial

r size of right-hand side sets F_i

p size of field $\mathbb{F}_p$

DQI approx. ratio (asymptotic behavior)

$$\frac{\langle S \rangle}{m} = \left(\sqrt{\frac{\ell}{m} \left(1 - \frac{r}{p} \right)} + \sqrt{\frac{r}{p} \left(1 - \frac{\ell}{m} \right)} \right)^2$$

for optimal polynomial P in the limit $m \rightarrow \infty$ with fixed $\frac{\ell}{m}$

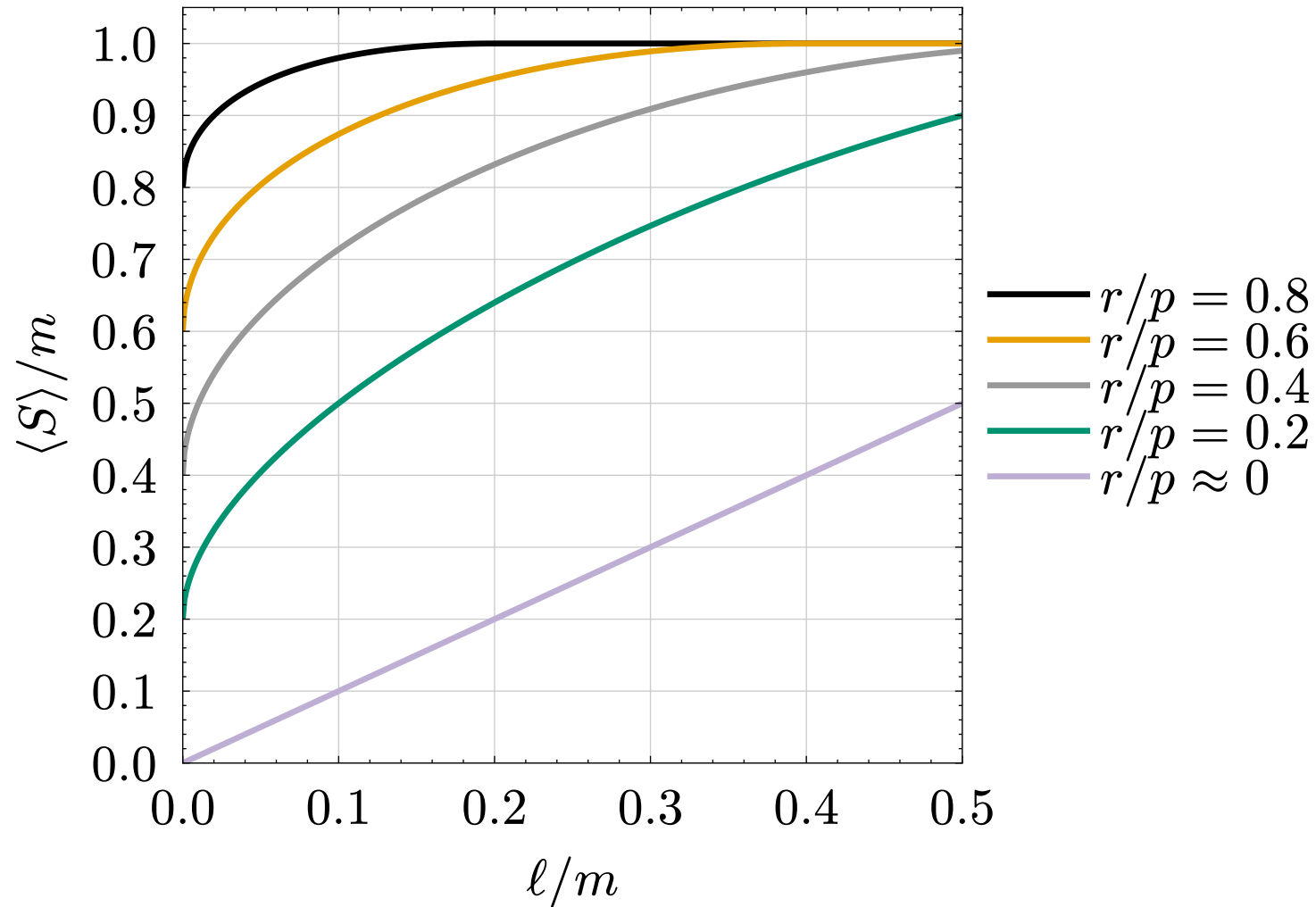
$\langle S \rangle$ number of satisfied constraints

m total number of constraints

ℓ degree of polynomial

r size of right-hand side sets F_i

p size of field $\mathbb{F}_p$



$$p = 2, \quad r = 1$$

Semi-circle law: binary case

$$\frac{\langle S \rangle}{m} = \left(\sqrt{\frac{\ell}{m} \left(1 - \frac{r}{p} \right)} + \sqrt{\frac{r}{p} \left(1 - \frac{\ell}{m} \right)} \right)^2$$

$$p = 2, \quad r = 1$$

Semi-circle law: binary case

$$\frac{\langle S \rangle}{m} = \left(\sqrt{\frac{\ell}{m} \left(1 - \frac{1}{2} \right)} + \sqrt{\frac{1}{2} \left(1 - \frac{\ell}{m} \right)} \right)^2$$

$$p = 2, \quad r = 1$$

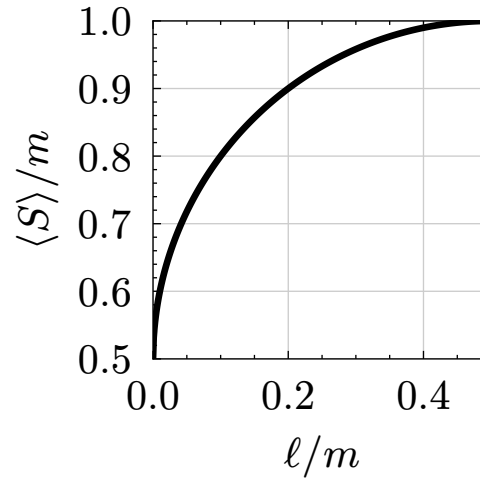
Semi-circle law: binary case

$$\frac{\langle S \rangle}{m} = \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m} \right)}$$

$$p = 2, \quad r = 1$$

Semi-circle law: binary case

$$\frac{\langle S \rangle}{m} = \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m} \right)}$$



Reed-Solomon Codes

$$B = \begin{pmatrix} 1 & 1 & 1^2 & 1^3 & \dots & 1^n \\ 1 & 2 & 2^2 & 2^3 & \dots & 2^n \\ 1 & 3 & 3^2 & 3^3 & \dots & 3^n \\ 1 & 4 & 4^2 & 4^3 & \dots & 4^n \\ 1 & 5 & 5^2 & 5^3 & \dots & 5^n \\ 1 & 6 & 6^2 & 6^3 & \dots & 6^n \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & m & m^2 & m^3 & \dots & m^n \end{pmatrix} \quad B^T = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & 2 & 3 & 4 & 5 & 6 & \dots & m \\ 1^2 & 2^2 & 3^2 & 4^2 & 5^2 & 6^2 & \dots & m^2 \\ 1^3 & 2^3 & 3^3 & 4^3 & 5^3 & 6^3 & \dots & m^3 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1^n & 2^n & 3^n & 4^n & 5^n & 6^n & \dots & m^n \end{pmatrix}$$

Variables: $\mathbf{a} = (a_1, \dots, a_n)$

$$\begin{pmatrix} 1 & 1 & 1^2 & 1^3 & \dots & 1^n \\ 1 & 2 & 2^2 & 2^3 & \dots & 2^n \\ 1 & 3 & 3^2 & 3^3 & \dots & 3^n \\ 1 & 4 & 4^2 & 4^3 & \dots & 4^n \\ 1 & 5 & 5^2 & 5^3 & \dots & 5^n \\ 1 & 6 & 6^2 & 6^3 & \dots & 6^n \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & m & m^2 & m^3 & \dots & m^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_n \end{pmatrix} \stackrel{\max}{\in} \begin{pmatrix} \{1, 2, 3, \dots\} \\ \{0, 4, 9, \dots\} \\ \{2, 6, 7, \dots\} \\ \{0, 1, 8, \dots\} \\ \{4, 5, 6, \dots\} \\ \{3, 5, 7, \dots\} \\ \vdots \\ \{2, 4, 8, \dots\} \end{pmatrix}$$

Variables: $\mathbf{a} = (a_1, \dots, a_n)$

$$a_0 1 + a_1 1 + a_2 1^2 + a_3 1^3 + \dots + a_5 1^n \in \{1, 2, 3, \dots\}$$

$$a_0 1 + a_1 2 + a_2 2^2 + a_3 2^3 + \dots + a_5 2^n \in \{0, 4, 9, \dots\}$$

$$a_0 1 + a_1 3 + a_2 3^2 + a_3 3^3 + \dots + a_5 3^n \in \{2, 6, 7, \dots\}$$

$$a_0 1 + a_1 4 + a_2 4^2 + a_3 4^3 + \dots + a_5 4^n \in \{0, 1, 8, \dots\}$$

$$a_0 1 + a_1 5 + a_2 5^2 + a_3 5^3 + \dots + a_5 5^n \in \{4, 5, 6, \dots\}$$

$$a_0 1 + a_1 6 + a_2 6^2 + a_3 6^3 + \dots + a_5 6^n \in \{3, 5, 7, \dots\}$$

$$\vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots$$

$$a_0 1 + a_1 m + a_2 m^2 + a_3 m^3 + \dots + a_5 m^n \in \{2, 4, 8, \dots\}$$

Variables: $\mathbf{a} = (a_1, \dots, a_n)$

$$a_0 1 + a_1 1 + a_2 1^2 + a_3 1^3 + \dots + a_5 1^n \in \{1, 2, 3, \dots\}$$

$$a_0 1 + a_1 2 + a_2 2^2 + a_3 2^3 + \dots + a_5 2^n \in \{0, 4, 9, \dots\}$$

$$a_0 1 + a_1 3 + a_2 3^2 + a_3 3^3 + \dots + a_5 3^n \in \{2, 6, 7, \dots\}$$

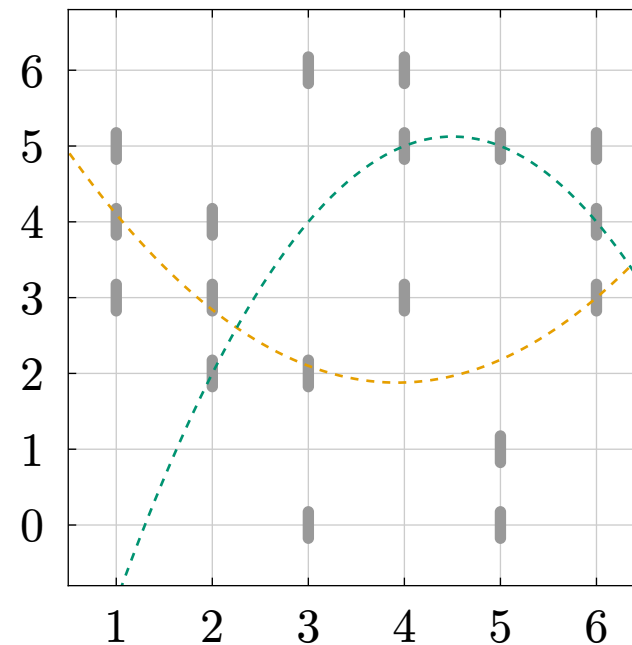
$$a_0 1 + a_1 4 + a_2 4^2 + a_3 4^3 + \dots + a_5 4^n \in \{0, 1, 8, \dots\}$$

$$a_0 1 + a_1 5 + a_2 5^2 + a_3 5^3 + \dots + a_5 5^n \in \{4, 5, 6, \dots\}$$

$$a_0 1 + a_1 6 + a_2 6^2 + a_3 6^3 + \dots + a_5 6^n \in \{3, 5, 7, \dots\}$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$a_0 1 + a_1 m + a_2 m^2 + a_3 m^3 + \dots + a_5 m^n \in \{2, 4, 8, \dots\}$$



Optimal Polynomial Intersection

Variables: $\mathbf{a} = (a_1, \dots, a_n)$

$$a_0 1 + a_1 1 + a_2 1^2 + a_3 1^3 + \dots + a_5 1^n \in \{1, 2, 3, \dots\}$$

$$a_0 1 + a_1 2 + a_2 2^2 + a_3 2^3 + \dots + a_5 2^n \in \{0, 4, 9, \dots\}$$

$$a_0 1 + a_1 3 + a_2 3^2 + a_3 3^3 + \dots + a_5 3^n \in \{2, 6, 7, \dots\}$$

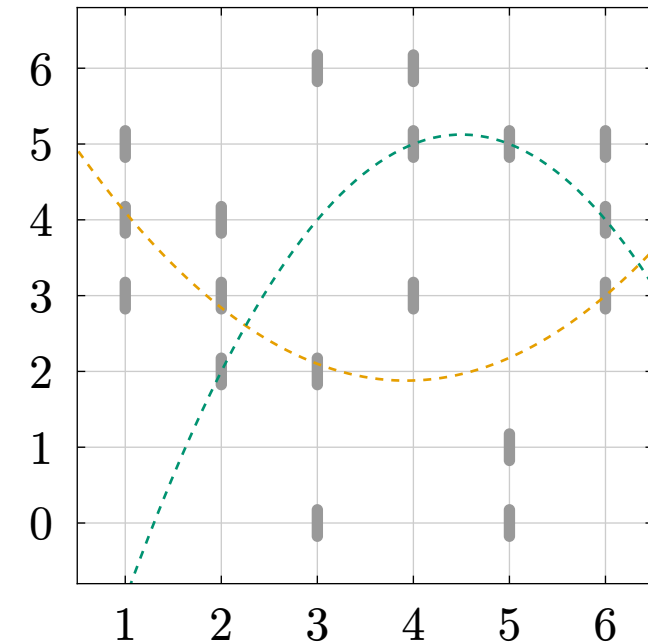
$$a_0 1 + a_1 4 + a_2 4^2 + a_3 4^3 + \dots + a_5 4^n \in \{0, 1, 8, \dots\}$$

$$a_0 1 + a_1 5 + a_2 5^2 + a_3 5^3 + \dots + a_5 5^n \in \{4, 5, 6, \dots\}$$

$$a_0 1 + a_1 6 + a_2 6^2 + a_3 6^3 + \dots + a_5 6^n \in \{3, 5, 7, \dots\}$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$a_0 1 + a_1 m + a_2 m^2 + a_3 m^3 + \dots + a_5 m^n \in \{2, 4, 8, \dots\}$$



Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

✗ don't beat random guessing

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

✗ don't beat random guessing

Coding-theoretic algorithms

OPI = List recovery for RS-Codes

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

✗ don't beat random guessing

Coding-theoretic algorithms

OPI = List recovery for RS-Codes

✗ existing algorithms fail for large $|F_i|$

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

✗ don't beat random guessing

Coding-theoretic algorithms

OPI = List recovery for RS-Codes

✗ existing algorithms fail for large $|F_i|$

Lattice-based approaches

Bleichenbacher and Nguyen (2000)

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

✗ don't beat random guessing

Coding-theoretic algorithms

OPI = List recovery for RS-Codes

✗ existing algorithms fail for large $|F_i|$

Lattice-based approaches

Bleichenbacher and Nguyen (2000)

✗ success prob. decays exponentially in p

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

✗ don't beat random guessing

Coding-theoretic algorithms

OPI = List recovery for RS-Codes

✗ existing algorithms fail for large $|F_i|$

Lattice-based approaches

Bleichenbacher and Nguyen (2000)

✗ success prob. decays exponentially in p

Prange's algorithm

- Find n linearly independent constraints
- Satisfy them (Gaussian elimination)
- Hope for the best for the other constraints

Parameter Regime:

random F_i with $|F_i| = \frac{p}{2}$

Local heuristics

Simulated Annealing, TABU Search, ...

✗ don't beat random guessing

Coding-theoretic algorithms

OPI = List recovery for RS-Codes

✗ existing algorithms fail for large $|F_i|$

Lattice-based approaches

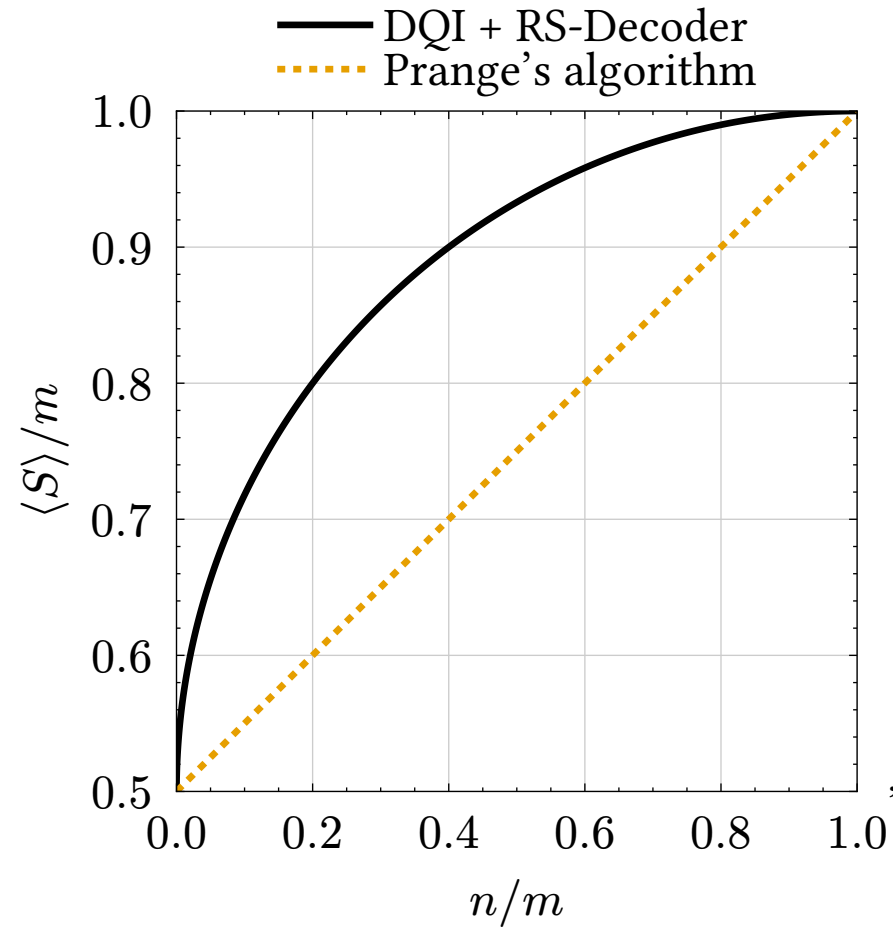
Bleichenbacher and Nguyen (2000)

✗ success prob. decays exponentially in p

Prange's algorithm

- Find n linearly independent constraints
- Satisfy them (Gaussian elimination)
- Hope for the best for the other constraints

✓ $\langle S \rangle = n + (m - n) \frac{r}{p}$



**Exercise 5**

DQI with a Reed-Solomon decoder: a closed-form estimate, no decoding needed

Your task

1. Generate a random OPI instance over $\mathbb{F}_{31}$ with $|F_i| = p/2$
2. Run DQI and compare with simulated annealing and Prange's algorithm
3. Vary the degree: how does the DQI ratio change with n/p ?

DQI-Kit

```
F = GF(31)
opi = OptimalPolynomialIntersection.random(
    F, degree=9, r=15)
est = Dqi(opi).estimate_solution_quality(details=True)
print(est.l, est.method, est.value)

SimAnnealSolver(opi, steps=2000).get_solution_quality()
PrangeSolver(opi).get_expected_solution_quality()
```


State of research:

DQI can be applied to a range of algebraic problems

State of research:

DQI can be applied to a range of algebraic problems

OPI over $\mathbb{F}_{2^s}$ (Khattar et al. 2025)

- Directly generalizable
- Efficient qubit-encoding $\Rightarrow$ smaller circuits

State of research:

DQI can be applied to a range of algebraic problems

OPI over $\mathbb{F}_{2^s}$ (Khattar et al. 2025)

- Directly generalizable
- Efficient qubit-encoding $\Rightarrow$ smaller circuits

Hermitian OPI (Gu et al. 2025)

- Replace RS codes with Hermitian codes
- Fewer qubits for same error correction

State of research:

DQI can be applied to a range of algebraic problems

OPI over $\mathbb{F}_{2^s}$ (Khattar et al. 2025)

- Directly generalizable
- Efficient qubit-encoding $\Rightarrow$ smaller circuits

Hermitian OPI (Gu et al. 2025)

- Replace RS codes with Hermitian codes
- Fewer qubits for same error correction

Multi-variate OPI (Jordan et al. 2024)

- Replace RS codes with Reed-Muller codes
- Larger design space, many open questions

State of research:

DQI can be applied to a range of algebraic problems

OPI over $\mathbb{F}_{2^s}$ (Khattar et al. 2025)

- Directly generalizable
- Efficient qubit-encoding $\Rightarrow$ smaller circuits

Hermitian OPI (Gu et al. 2025)

- Replace RS codes with Hermitian codes
- Fewer qubits for same error correction

Multi-variate OPI (Jordan et al. 2024)

- Replace RS codes with Reed-Muller codes
- Larger design space, many open questions

Hamiltonian OPI (Schmidhuber et al. 2025)

- Only Pauli- $Z \Rightarrow X, Y$ and Z
- Some applications, quantum adv. unclear

State of research:

DQI can be applied to a range of algebraic problems

OPI over $\mathbb{F}_{2^s}$ (Khattar et al. 2025)

- Directly generalizable
- Efficient qubit-encoding $\Rightarrow$ smaller circuits

Hermitian OPI (Gu et al. 2025)

- Replace RS codes with Hermitian codes
- Fewer qubits for same error correction

Multi-variate OPI (Jordan et al. 2024)

- Replace RS codes with Reed-Muller codes
- Larger design space, many open questions

Hamiltonian OPI (Schmidhuber et al. 2025)

- Only Pauli- $Z \Rightarrow X, Y$ and Z
- Some applications, quantum adv. unclear

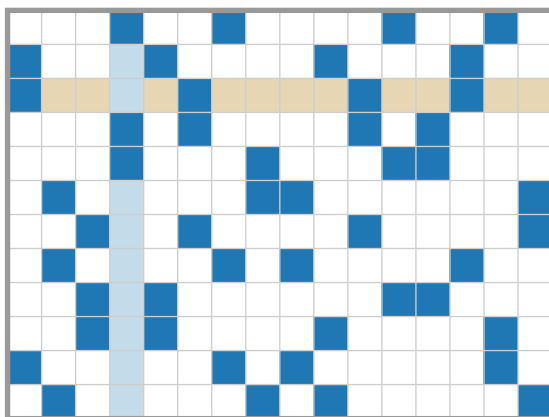
Approximating DQI classically (Gil-Fuster et al. 2026)

- Amplitudes of DQI state efficiently computable
- $\Rightarrow$ Markov chain Monte Carlo sampling
- Runtime $\approx O(1.1^n)$

Sparse Max-XORSAT

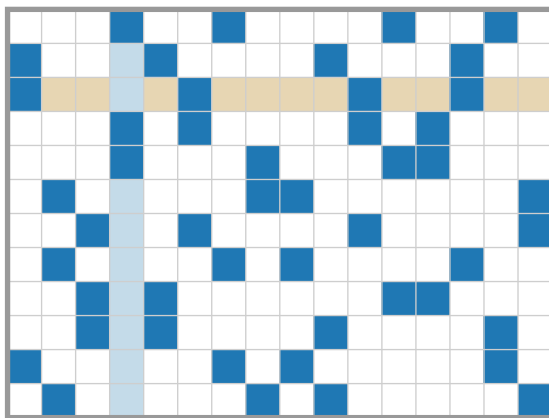


$$H = B^T \in \mathbb{F}_2^{12 \times 16},$$



(ρ, γ) -regular LDPC code

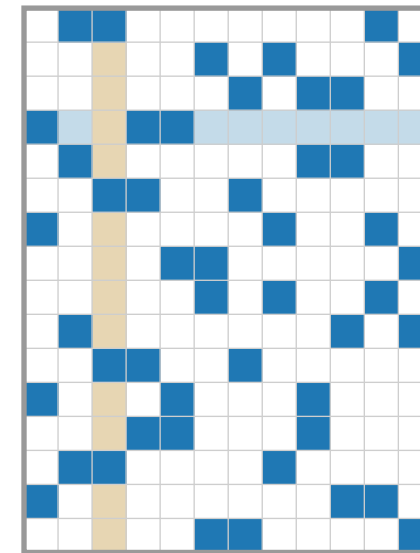
$$H = B^T \in \mathbb{F}_2^{12 \times 16},$$



(ρ, γ) -regular LDPC code



$$B \in \mathbb{F}_2^{16 \times 12}$$



Max-XORSAT instance:

γ variables per constraint

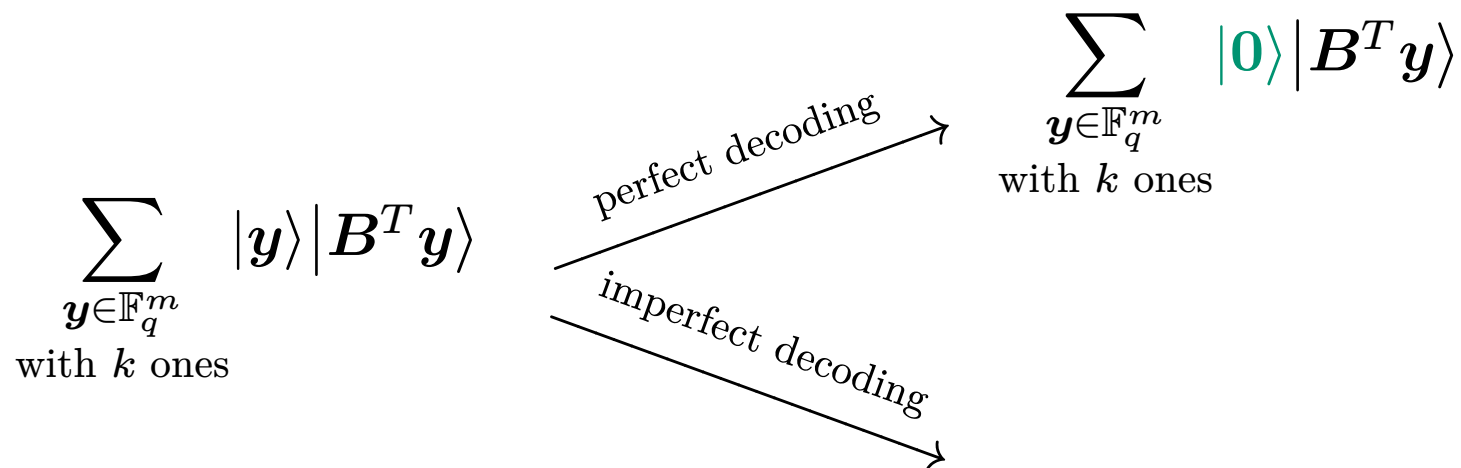
each variable included in ρ constraints

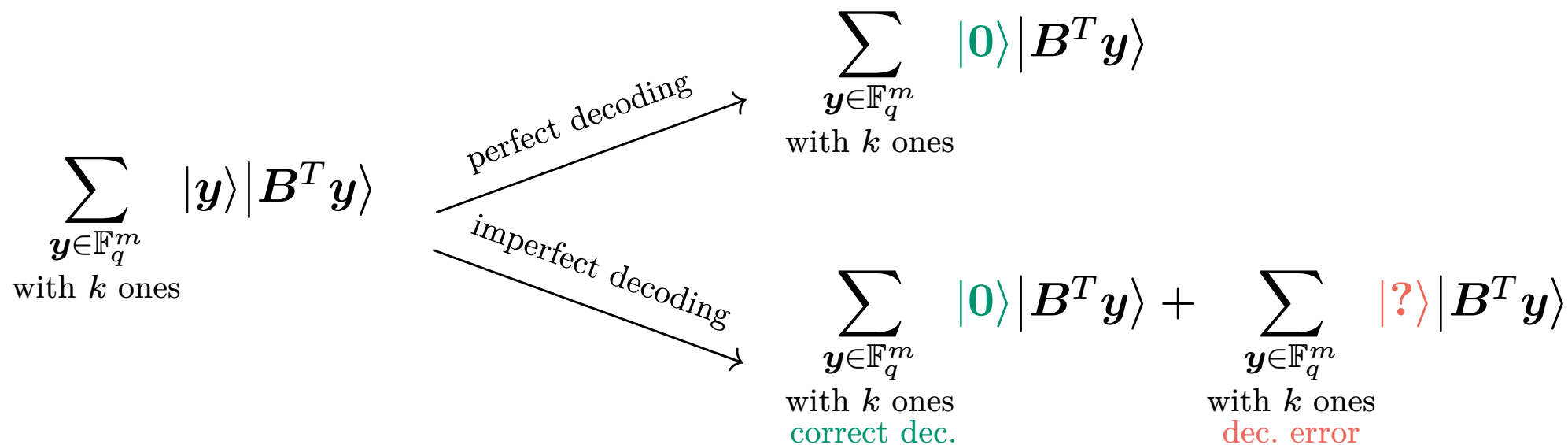
$$\sum_{\mathbf{y} \in \mathbb{F}_q^m} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle$$

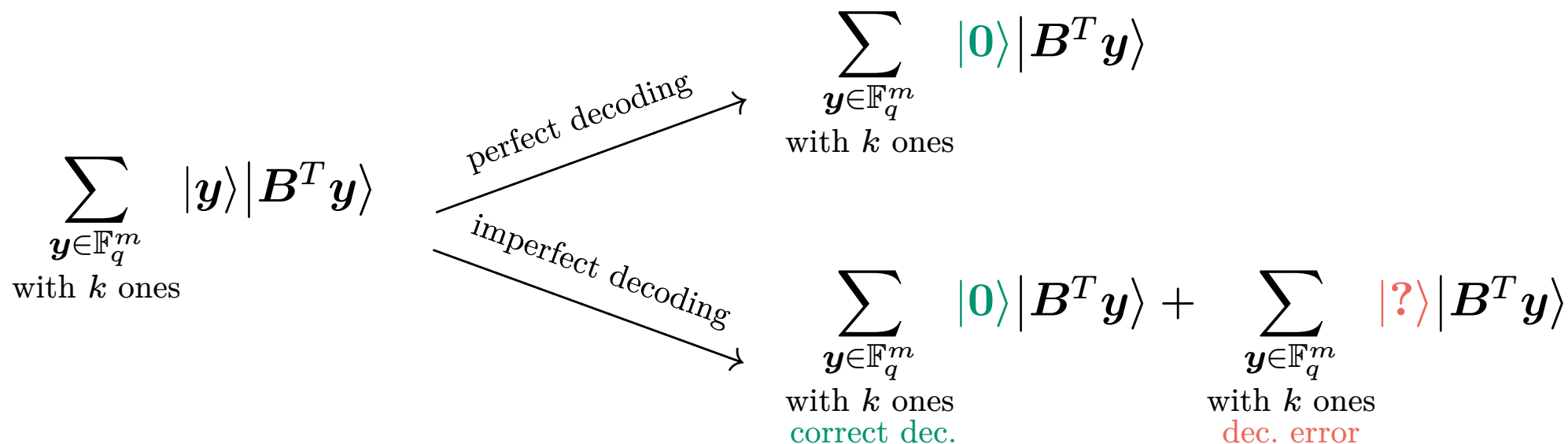
with k ones

$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_q^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle \xrightarrow{\text{perfect decoding}}$$

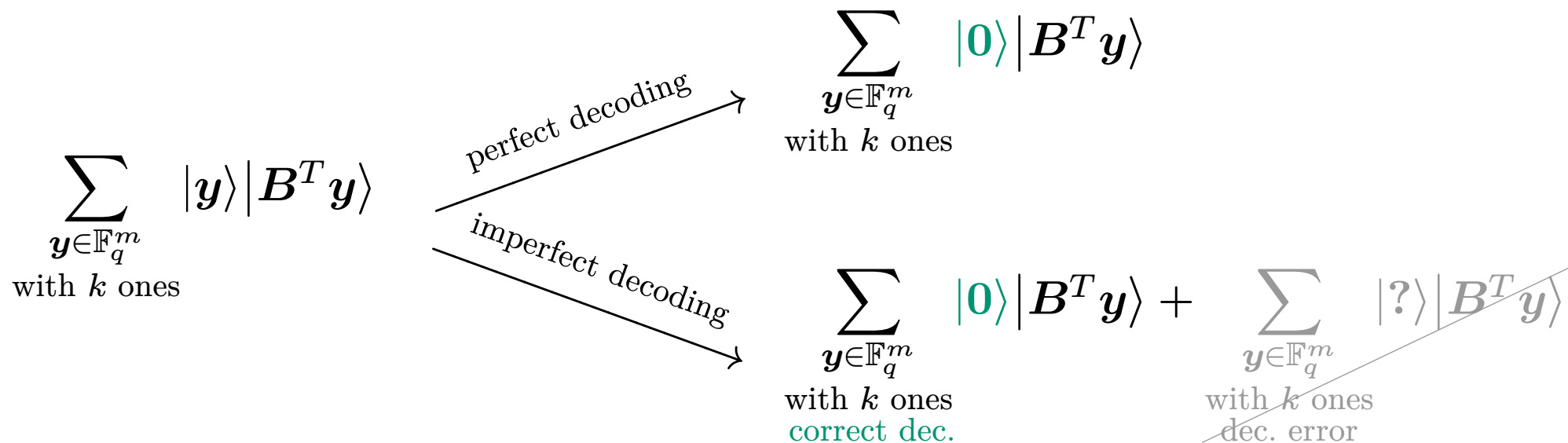
$$\sum_{\substack{\mathbf{y} \in \mathbb{F}_q^m \\ \text{with } k \text{ ones}}} |\mathbf{y}\rangle |B^T \mathbf{y}\rangle \xrightarrow{\text{perfect decoding}} \sum_{\substack{\mathbf{y} \in \mathbb{F}_q^m \\ \text{with } k \text{ ones}}} |\mathbf{0}\rangle |B^T \mathbf{y}\rangle$$



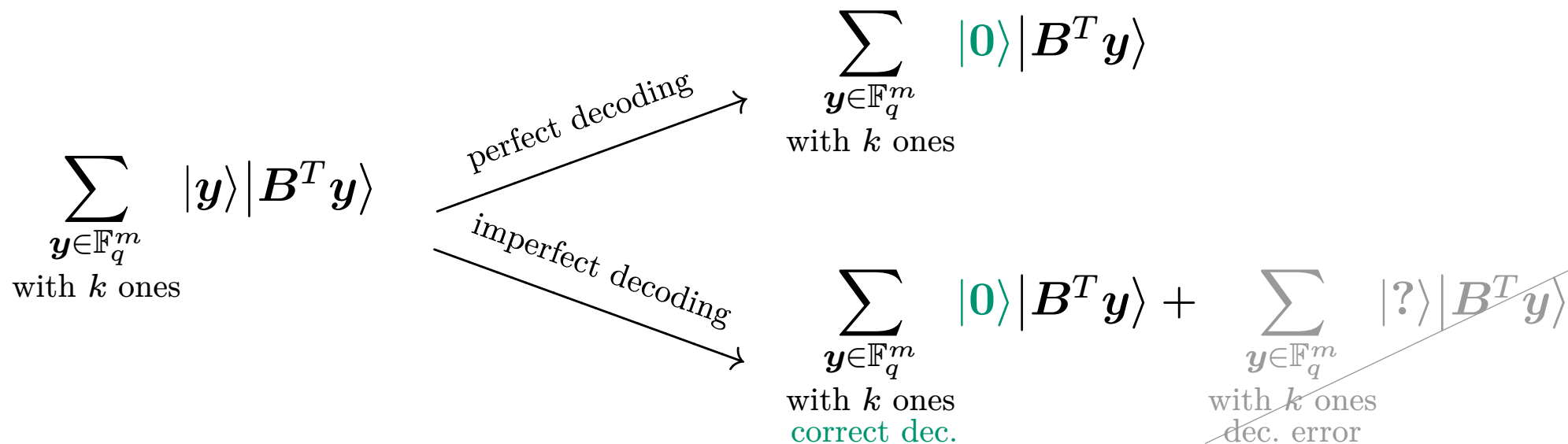




Measure first register. If measurement $\neq |0\rangle$, repeat.



Measure first register. If measurement $\neq |0\rangle$, repeat.



Measure first register. If measurement $\neq |0\rangle$, repeat.

Problem: Resulting state $\approx$ DQI State

Semi-circle law: binary case with imperfect decoder

$$\frac{\langle S \rangle}{m} = \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m} \right)}$$

for optimal polynomial P in the limit $m \rightarrow \infty$ with fixed $\frac{\ell}{m}$.

Semi-circle law: binary case with imperfect decoder

$$\frac{\langle S \rangle}{m} \geq \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m} \right)} - \varepsilon$$

for optimal polynomial P in the limit $m \rightarrow \infty$ with fixed $\frac{\ell}{m}$.

Semi-circle law: binary case with imperfect decoder

$$\frac{\langle S \rangle}{m} \geq \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m} \right)} - \varepsilon$$

for optimal polynomial P in the limit $m \rightarrow \infty$ with fixed $\frac{\ell}{m}$.

ε : probability of a decoding a ℓ -weight error incorrectly

Semi-circle law: binary case with imperfect decoder

$$\frac{\langle S \rangle}{m} \geq \frac{1}{2} + \sqrt{\frac{\ell}{m} \left(1 - \frac{\ell}{m} \right)} - \varepsilon$$

for optimal polynomial P in the limit $m \rightarrow \infty$ with fixed $\frac{\ell}{m}$.

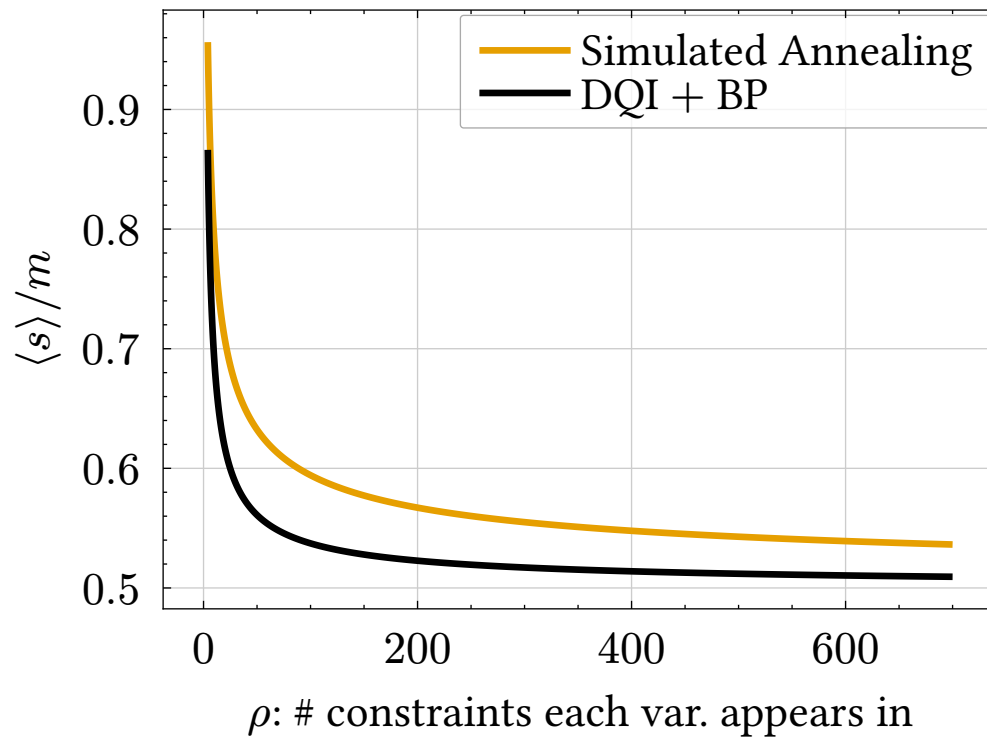
ε : probability of a decoding a ℓ -weight error incorrectly

We assume that v is uniformly random.

Results for Max-XORSAT with 3 variables per constraint ($n = 20\,000$)
(*Jordan et al. 2024*)

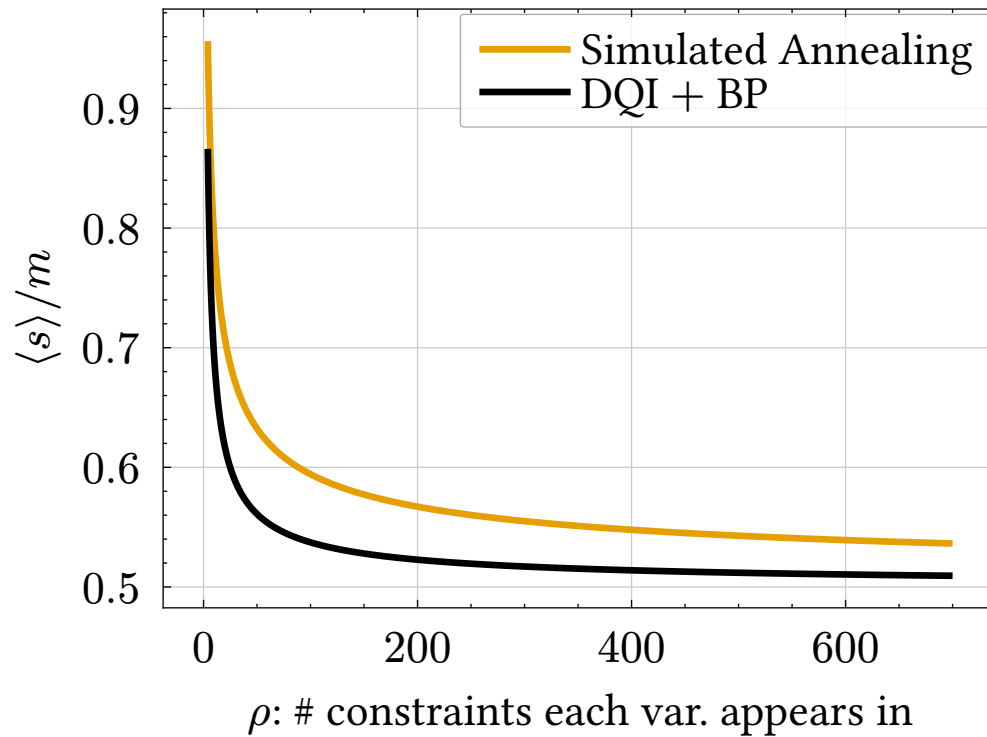
Results for Max-XORSAT with 3 variables per constraint ($n = 20\,000$)

(Jordan et al. 2024)



Results for Max-XORSAT with 3 variables per constraint ($n = 20\,000$)

(Jordan et al. 2024)



The problem

Sparse H :

good for both BP and Sim. Ann.

Dense H :

bad for both BP and Sim. Ann.

State of research:

DQI struggles to beat classical heuristics on unstructured instances

State of research:

DQI struggles to beat classical heuristics on unstructured instances

Jordan *et al.* 2024

- Instances where DQI beats SA exist.
- Custom heuristics beat DQI on these instances

State of research:

DQI struggles to beat classical heuristics on unstructured instances

Jordan *et al.* 2024

- Instances where DQI beats SA exist.
- Custom heuristics beat DQI on these instances

Parekh 2025

If $\gamma = 2$, DQI with perfect decoders is provably beaten by classical algorithms.

State of research:

DQI struggles to beat classical heuristics on unstructured instances

Jordan *et al.* 2024

- Instances where DQI beats SA exist.
- Custom heuristics beat DQI on these instances

Parekh 2025

If $\gamma = 2$, DQI with perfect decoders is provably beaten by classical algorithms.

Anschuetz *et al.* 2025

- Theoretical evidence against quantum advantage with DQI+BP
- Empirical evidence for classical + quantum algorithms beating DQI

State of research:

DQI struggles to beat classical heuristics on unstructured instances

Jordan *et al.* 2024

- Instances where DQI beats SA exist.
- Custom heuristics beat DQI on these instances

Parekh 2025

If $\gamma = 2$, DQI with perfect decoders is provably beaten by classical algorithms.

Anschuetz *et al.* 2025

- Theoretical evidence against quantum advantage with DQI+BP
- Empirical evidence for classical + quantum algorithms beating DQI

Shutty *et al.* 2026

- Special quantum decoders improve DQI performance
- Prange + local heuristic ties DQI

State of research:

DQI struggles to beat classical heuristics on unstructured instances

Jordan *et al.* 2024

- Instances where DQI beats SA exist.
- Custom heuristics beat DQI on these instances

Parekh 2025

If $\gamma = 2$, DQI with perfect decoders is provably beaten by classical algorithms.

Anschuetz *et al.* 2025

- Theoretical evidence against quantum advantage with DQI+BP
- Empirical evidence for classical + quantum algorithms beating DQI

Shutty *et al.* 2026

- Special quantum decoders improve DQI performance
- Prange + local heuristic ties DQI

Structured + Algebraic

✓ apparent exponential speedup

Structured + Algebraic

✓ apparent exponential speedup

Unstructured + Sparse

✗ no advantage yet

Structured + Algebraic

✓ apparent exponential speedup

Unstructured + Sparse

✗ no advantage yet

But what about real-world problems?

Encoding Problems for DQI



Code $\longrightarrow$ Problem



Code $\longleftarrow$ Problem

DQI-Kit:
A framework to evaluate DQI on industrial
optimisation problems

Symmetric Set Max-LINSAT

Given:

m constraints of the form

$$(B_{i1}x_1 + B_{i2}x_2 + \dots + B_{in}x_n \bmod p) \in F_i$$

with $x_j \in \mathbb{F}_p$, $B_{ij} \in \mathbb{F}_p$,

$F_i \subseteq \mathbb{F}_p$ and $|F_1| = |F_2| = \dots = |F_m|$

Find:

assignment $\mathbf{x}$ which maximizes number of satisfied constraints.

Symmetric Set Max-LINSAT

Given:

m constraints of the form

$$(B_{i1}x_1 + B_{i2}x_2 + \dots + B_{in}x_n \bmod p) \in F_i$$

with $x_j \in \mathbb{F}_p$, $B_{ij} \in \mathbb{F}_p$,

$F_i \subseteq \mathbb{F}_p$ and $|F_1| = |F_2| = \dots = |F_m|$

Find:

assignment x which maximizes number of satisfied constraints.

Max-XORSAT

Given:

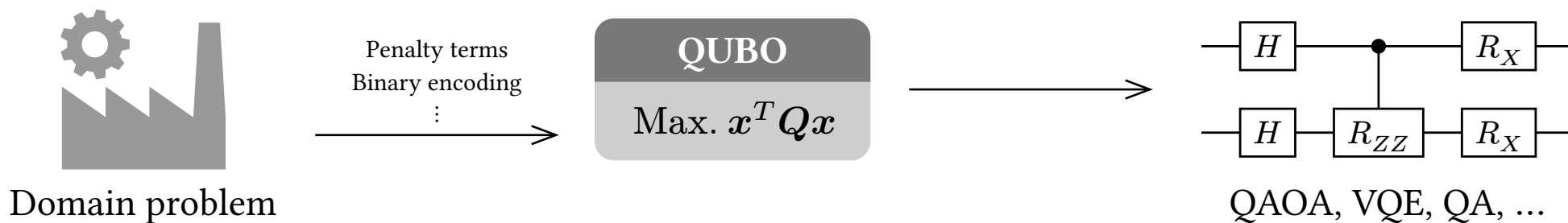
m constraints of the form

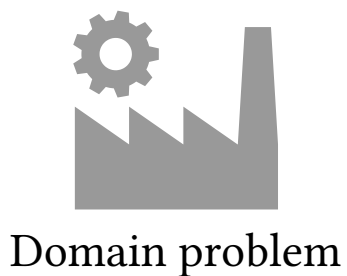
$$B_{i1}x_1 \oplus B_{i2}x_2 \oplus \dots \oplus B_{in}x_n = v_i$$

with $x_j, B_{ij}, v_i \in \{0, 1\}$

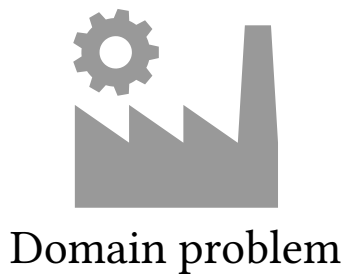
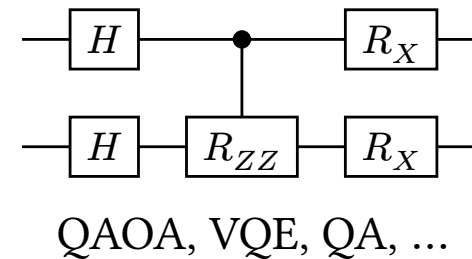
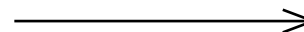
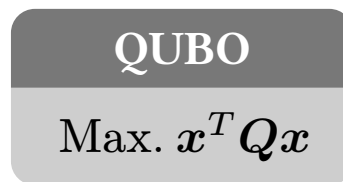
Find:

assignment x which maximizes number of satisfied constraints.

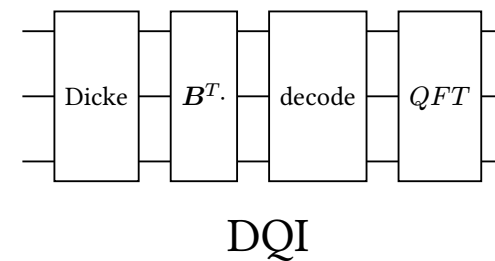
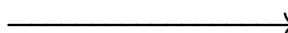
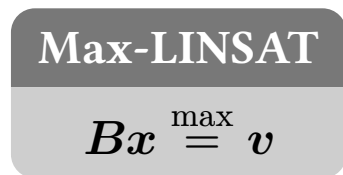




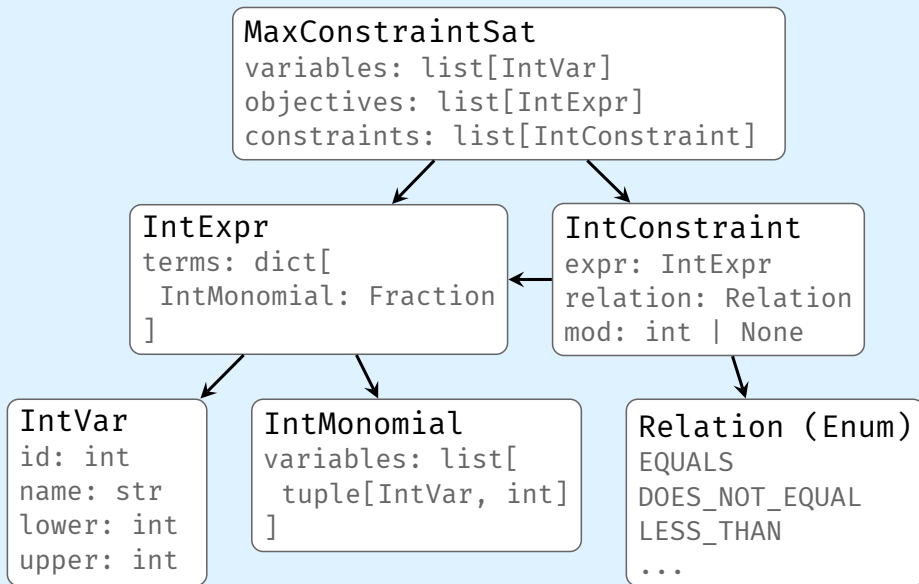
Penalty terms
Binary encoding
⋮



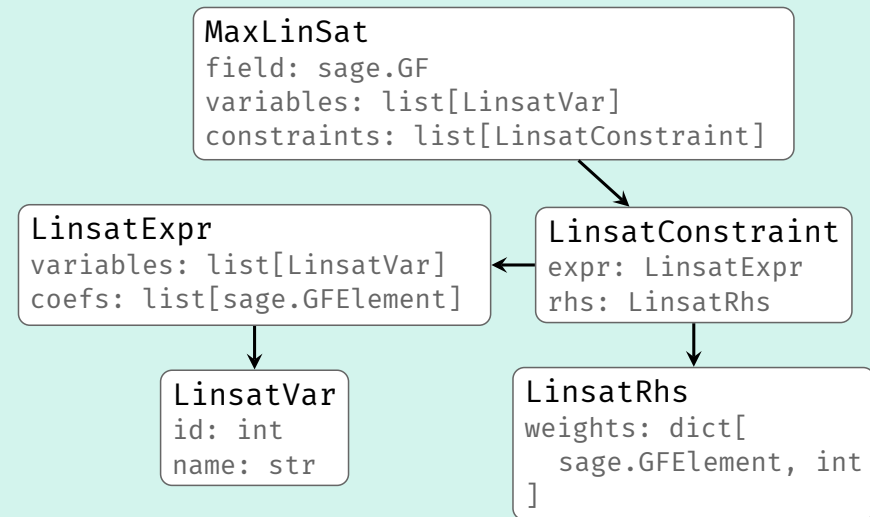
?



Higher abstraction level: Boolean/integer constraints/objectives



Lower abstraction level: LINSAT constraints



```
1 # Knapsack
2 items = [Item(w=3, val=5), ...]
3 prob = MaxConstraintSat()
4 x = [
5     prob.new_binary_var(f"x_{i}")
6     for i in range(len(items))
7 ]
8 prob.add_constraint(
9     sum(
10         x_i * i.w for x_i, i in zip(x, items)
11     ) <= W,
12     weight=2 * sum(i.val for i in items)
13 )
14 prob.add_objective(sum(
15     x_i * i.val for x_i, i in zip(x, items)
16 ))
```

```
1 # Knapsack
2 items = [Item(w=3, val=5), ...]
3 prob = MaxConstraintSat()
4 x = [
5     prob.new_binary_var(f"x_{i}")
6     for i in range(len(items))
7 ]
8 prob.add_constraint(
9     sum(
10         x_i * i.w for x_i,i in zip(x,items)
11     ) <= W,
12     weight=2 * sum(i.val for i in items)
13 )
14 prob.add_objective(sum(
15     x_i * i.val for x_i,i in zip(x,items)
16 ))
```

```
1 # Min. vertex cover
2 G = nx.Graph(...)
3 prob = MaxConstraintSat()
4 x = [
5     prob.new_binary_var(f"x_{i}")
6     for i in range(n)
7 ]
8
9 for u, v in G.edges:
10     prob.add_boolean_constraint(
11         x[u] | x[v], weight=2
12     )
13 prob.add_objective(
14     sum(x_i for x_i in x),
15     minimize=True,
16 )
```

```
1 # Knapsack
2 items = [Item(w=3, val=5), ...]
3 prob = MaxConstraintSat()
4 x = [
5     prob.new_binary_var(f"x_{i}")
6     for i in range(len(items))
7 ]
8 prob.add_constraint(
9     sum(
10         x_i * i.w for x_i,i in zip(x,items)
11     ) <= W,
12     weight=2 * sum(i.val for i in items)
13 )
14 prob.add_objective(sum(
15     x_i * i.val for x_i,i in zip(x,items)
16 ))
```

```
1 # Min. vertex cover
2 G = nx.Graph(...)
3 prob = MaxConstraintSat()
4 x = [
5     prob.new_binary_var(f"x_{i}")
6     for i in range(n)
7 ]
8
9 for u, v in G.edges:
10     prob.add_boolean_constraint(
11         x[u] | x[v], weight=2
12     )
13 prob.add_objective(
14     sum(x_i for x_i in x),
15     minimize=True,
16 )
```

```
1 # Max. 3-colourable subgraph
2 G = nx.Graph(...)
3 prob = MaxLinSat(GF(3))
4 x = [
5     prob.new_var(f"x_{i}")
6     for i in range(n)
7 ]
8 for u, v in G.edges:
9     prob.add_constraint(
10         x[u] != x[v]
11     )
```

```
1 # Knapsack
2 items = [Item(w=3, val=5), ...]
3 prob = MaxConstraintSat()
4 x = [
5     prob.new_binary_var(f"x_{i}")
6     for i in range(len(items))
7 ]
8 prob.add_constraint(
9     sum(
10         x_i * i.w for x_i,i in zip(x,items)
11     ) <= W,
12     weight=2 * sum(i.val for i in items)
13 )
14 prob.add_objective(sum(
15     x_i * i.val for x_i,i in zip(x,items)
16 ))
```

```
1 # Min. vertex cover
2 G = nx.Graph(...)
3 prob = MaxConstraintSat()
4 x = [
5     prob.new_binary_var(f"x_{i}")
6     for i in range(n)
7 ]
8
9 for u, v in G.edges:
10     prob.add_boolean_constraint(
11         x[u] | x[v], weight=2
12     )
13 prob.add_objective(
14     sum(x_i for x_i in x),
15     minimize=True,
16 )
```

```
1 # Max. 3-colourable subgraph
2 G = nx.Graph(...)
3 prob = MaxLinSat(GF(3))
4 x = [
5     prob.new_var(f"x_{i}")
6     for i in range(n)
7 ]
8 for u, v in G.edges:
9     prob.add_constraint(
10         x[u] != x[v]
11     )
```

```
1 print(SimAnnealSolver(prob).get_solution_quality())
2 dqi = Dqi(prob)
3 for l in range(0, n):
4     print(dqi.estimate_solution_quality(l))
```

Decoders

- General purpose (e.g., belief propagation)
- *Interface for specialised decoders*

Decoders

- General purpose (e.g., belief propagation)
- *Interface for specialised decoders*

Classical solvers

- Simulated annealing, constr. progr., ...
- Prange, brute force
- *Interface for custom solvers*

Decoders

- General purpose (e.g., belief propagation)
- *Interface for specialised decoders*

Classical solvers

- Simulated annealing, constr. progr., ...
- Prange, brute force
- *Interface for custom solvers*

Max-LINSAT conversion settings

- max degree (higher-order terms)
- merge strategy: weights, duplicate constr.
- ...

DQI-Kit

Optimal for DQI

Max-XORSAT

$$a \oplus b \oplus c = 1$$

$$a \oplus b + c = 1$$

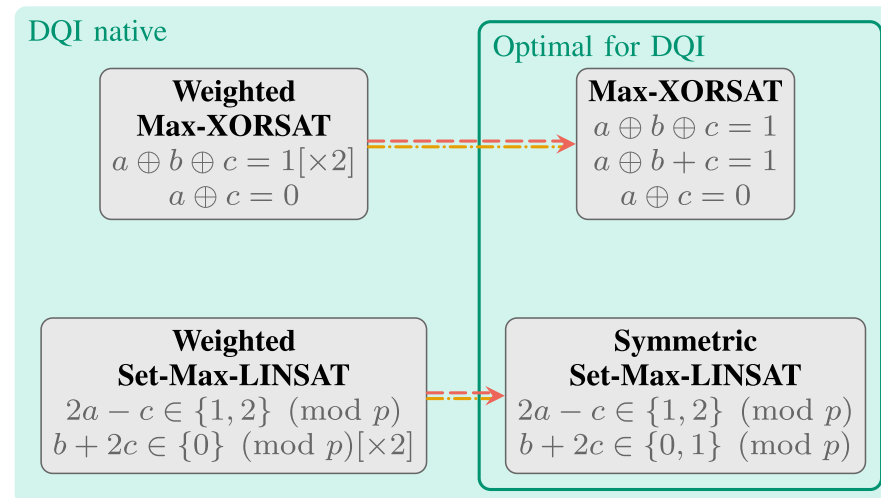
$$a \oplus c = 0$$

**Symmetric
Set-Max-LINSAT**

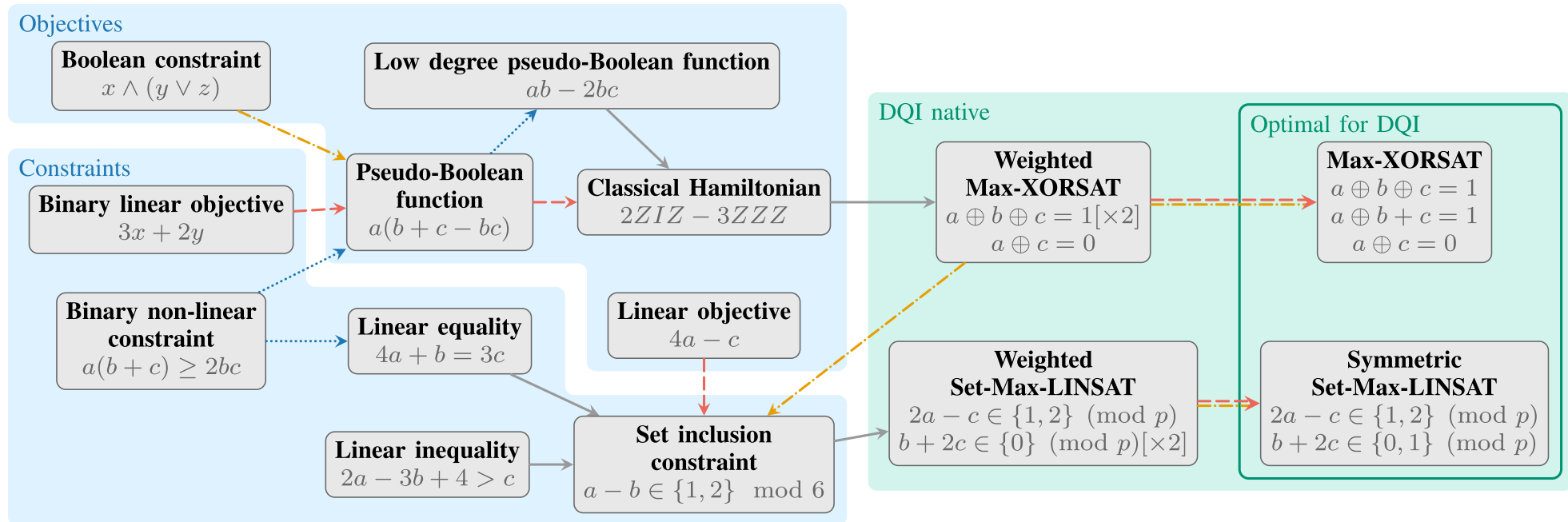
$$2a - c \in \{1, 2\} \pmod{p}$$

$$b + 2c \in \{0, 1\} \pmod{p}$$

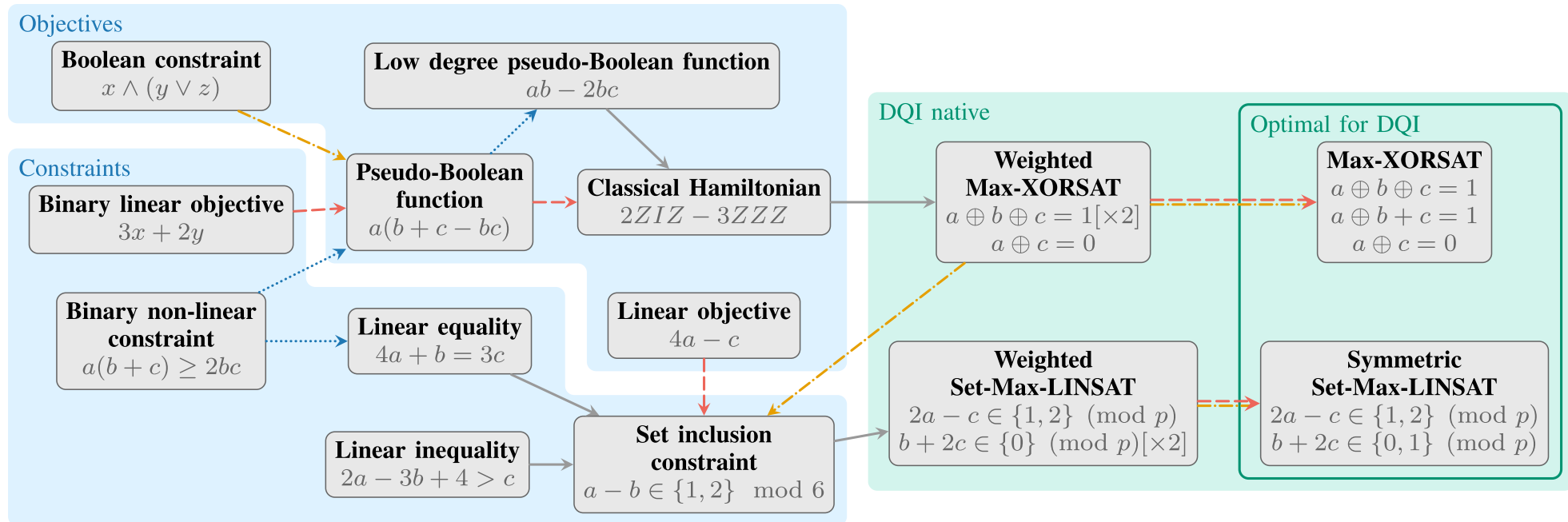
DQI-Kit



DQI-Kit



DQI-Kit



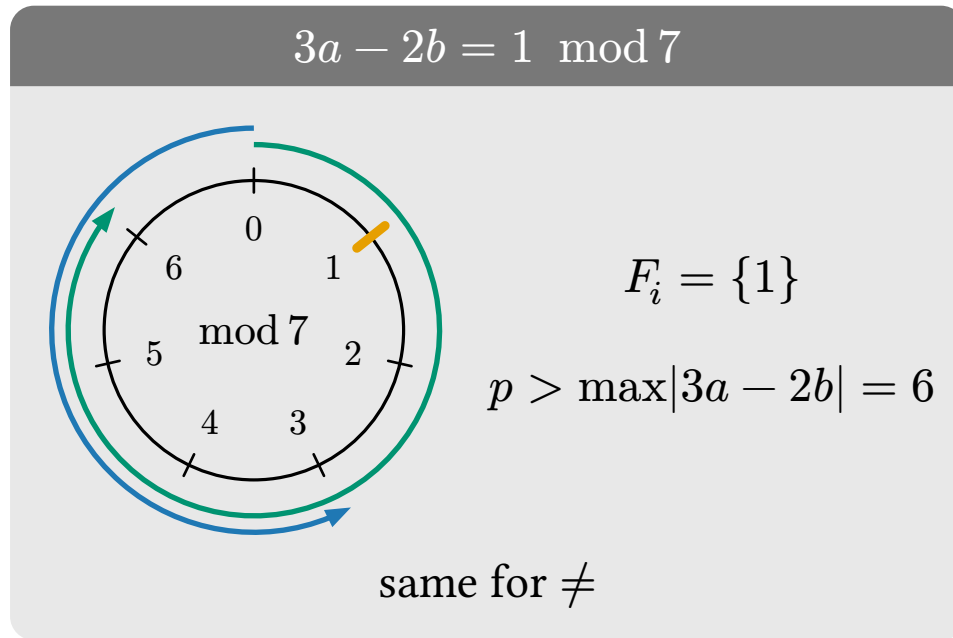
Adds constraints

Adds auxiliary variables

Adds linear dependencies

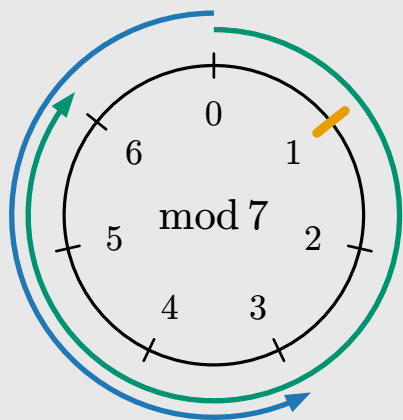
$$a, b \in \{0, 1, 2\} \implies 3a - 2b \in \{-4, \dots, 6\} = \{L, \dots, H\}$$

$$a, b \in \{0, 1, 2\} \implies 3a - 2b \in \{-4, \dots, 6\} = \{L, \dots, H\}$$



$$a, b \in \{0, 1, 2\} \implies 3a - 2b \in \{-4, \dots, 6\} = \{L, \dots, H\}$$

$$3a - 2b = 1 \pmod{7}$$

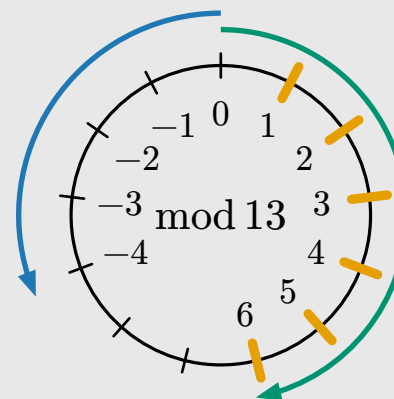


$$F_i = \{1\}$$

$$p > \max|3a - 2b| = 6$$

same for $\neq$

$$3a - 2b \geq 1 \pmod{13}$$



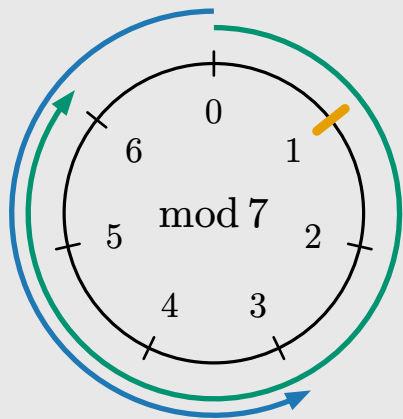
$$F_i = \{1, \dots, 6\}$$

$$p > U - L = 10$$

same for $\leq, <, >$

$$a, b \in \{0, 1, 2\} \implies 3a - 2b \in \{-4, \dots, 6\} = \{L, \dots, H\}$$

$$3a - 2b = 1 \pmod{7}$$

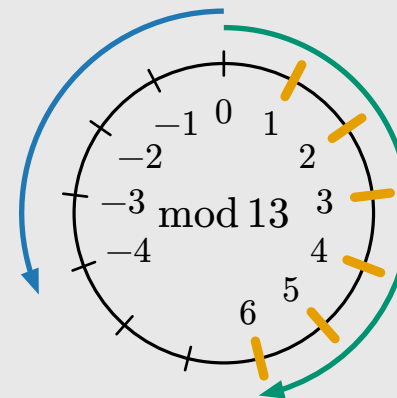


$$F_i = \{1\}$$

$$p > \max|3a - 2b| = 6$$

same for $\neq$

$$3a - 2b \geq 1 \pmod{13}$$



$$F_i = \{1, \dots, 6\}$$

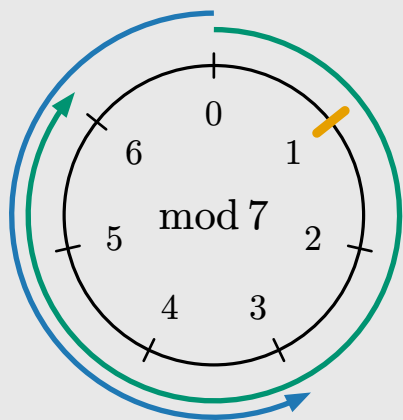
$$p > U - L = 10$$

same for $\leq, <, >$

No auxiliary variables, $O(\log p)$ gates

$$a, b \in \{0, 1, 2\} \implies 3a - 2b \in \{-4, \dots, 6\} = \{L, \dots, H\}$$

$$3a - 2b = 1 \pmod{7}$$

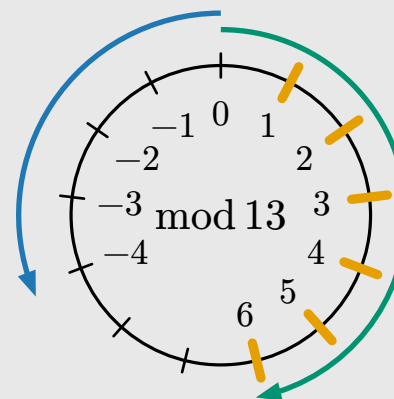


$$F_i = \{1\}$$

$$p > \max|3a - 2b| = 6$$

same for $\neq$

$$3a - 2b \geq 1 \pmod{13}$$



$$F_i = \{1, \dots, 6\}$$

$$p > U - L = 10$$

same for $\leq, <, >$

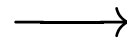
No auxiliary variables, $O(\log p)$ gates
Range constraints: $a \in \{0, 1, 2\}, b \in \{0, 1, 2\}$

Max-LINSAT has no objective function
objective term $\rightarrow$ constraint with a weight

Max-LINSAT has no objective function
objective term $\rightarrow$ constraint with a weight

Binary objective

$$\max 3x + 2y$$

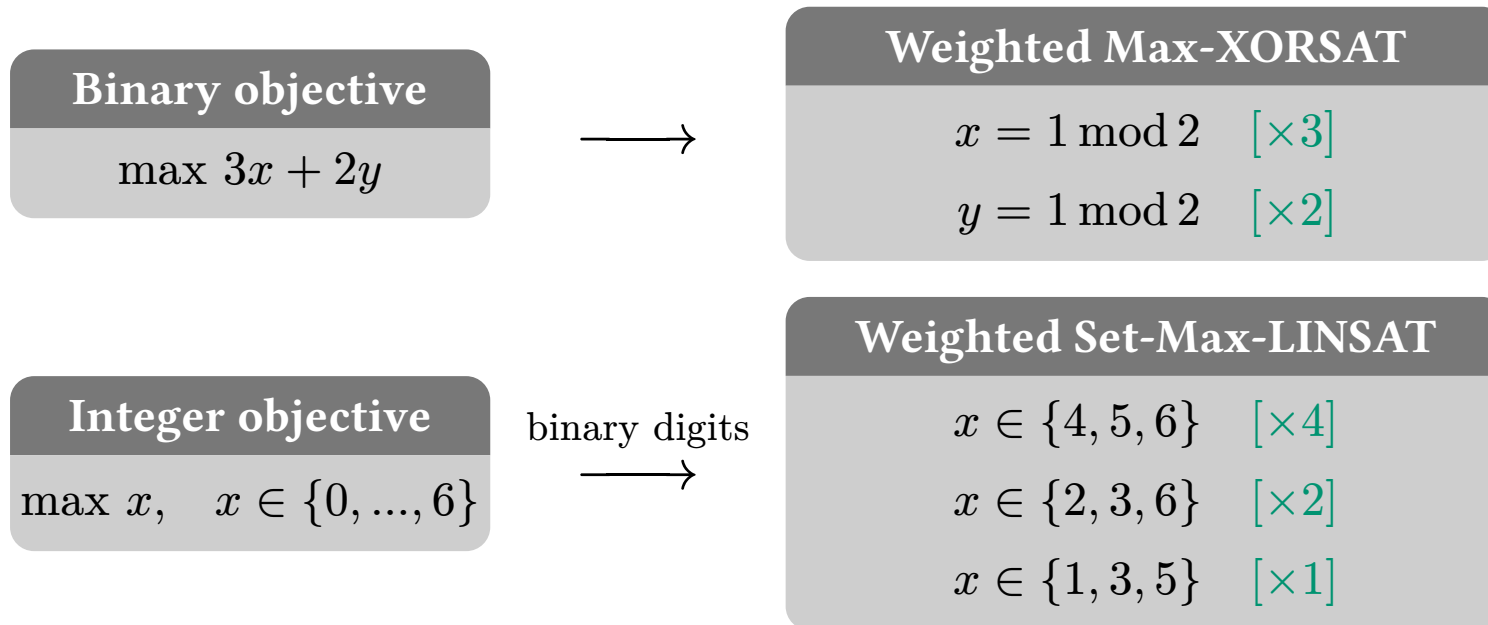


Weighted Max-XORSAT

$$x = 1 \bmod 2 \quad [\times 3]$$

$$y = 1 \bmod 2 \quad [\times 2]$$

Max-LINSAT has no objective function
objective term $\rightarrow$ constraint with a weight



$\lceil \log_2(u_i - l_i + 1) \rceil$ constraints per objective term ax_i

$$1Z_1Z_2 + 2Z_2 - 3Z_1Z_3$$

$$1Z_1Z_2 + 2Z_2 - 3Z_1Z_3$$

Weighted constraints^{1,2}

$$x_1 + x_2 = 0 \bmod 2 \quad [\times 1]$$

$$x_2 = 0 \bmod 2 \quad [\times 2]$$

$$x_1 + x_3 = 1 \bmod 2 \quad [\times 3]$$

¹Hamiltonian Decoded Quantum Interferometry for General Pauli Hamiltonians. Bu *et al.* 2026

²Multivariate Decoded Quantum Interferometry for Weighted Optimization. Bu *et al.* 2026

$$1Z_1Z_2 + 2Z_2 - 3Z_1Z_3$$

Weighted constraints^{1,2}

$$x_1 + x_2 = 0 \bmod 2 \quad [\times 1]$$

$$x_2 = 0 \bmod 2 \quad [\times 2]$$

$$x_1 + x_3 = 1 \bmod 2 \quad [\times 3]$$

$$x_1 + x_2 = 0 \bmod 2$$

$$x_2 = 0 \bmod 2$$

$$x_2 = 0 \bmod 2$$

$$x_1 + x_3 = 1 \bmod 2$$

$$x_1 + x_3 = 1 \bmod 2$$

$$x_1 + x_3 = 1 \bmod 2$$

¹Hamiltonian Decoded Quantum Interferometry for General Pauli Hamiltonians. Bu *et al.* 2026

²Multivariate Decoded Quantum Interferometry for Weighted Optimization. Bu *et al.* 2026

 Exercise 6

A real use case: linear capacity constraints and a linear profit objective

Use case

A workshop builds **chairs** (2 h of work, 1 unit of wood, profit 2) and **tables** (3 h, 2 units of wood, profit 3). 8 h and 5 units of wood are available; at most 3 chairs and 2 tables fit into the shop. Maximise the profit.

Your task

1. Encode: two integer variables, two $\leq$ constraints (weight $>$ max. profit), one objective
2. Convert: which p, n, m ? Which rows encode the objective, why so many duplicates?
3. Compare the Max-LINSAT optimum with the best production plan and with the DQI estimate

DQI-Kit

```
prob = MaxConstraintSat()
c = prob.new_var("chairs", 0, 3)
t = prob.new_var("tables", 0, 2)
prob.add_constraint(2 * c + 3 * t <= 8, weight=13)
prob.add_objective(2 * c + 3 * t)

linsat = prob.to_max_linsat()
linsat.field, linsat.get_m(), linsat.constraints
linsat.get_optimal_solution()
Dqi(linsat, SyndromeDecoder.constructor())
```

Idea $\mathbb{B} \rightarrow \mathbb{Z}$ (false $\rightarrow$ 0, true $\rightarrow$ 1)Boolean function $\rightarrow$ polynomial

Boolean operator	Pseudo-Boolean function
$\neg a$	$1 - a$
$a \wedge b$	ab
$a \vee b$	$a + b - ab$
$a \oplus b$	$a + b - 2ab$

Idea

$\mathbb{B} \rightarrow \mathbb{Z}$ (false $\rightarrow 0$, true $\rightarrow 1$)

Boolean function $\rightarrow$ polynomial

Boolean operator	Pseudo-Boolean function
$\neg a$	$1 - a$
$a \wedge b$	ab
$a \vee b$	$a + b - ab$
$a \oplus b$	$a + b - 2ab$

Example

$$a \wedge \neg(b \vee c) \mapsto a(1 - b + c - bc)$$

Bool. expression $\rightarrow$ Ising

$$x_i \mapsto \frac{1 - Z_i}{2}$$

$$Z|0\rangle = +1 \quad Z|1\rangle = -1$$

Bool. expression $\rightarrow$ Ising

$$x_i \mapsto \frac{1 - Z_i}{2}$$

$$Z|0\rangle = +1 \quad Z|1\rangle = -1$$

$$f(\mathbf{x}) = \langle \mathbf{x} | Z_i \cdots Z_j | \mathbf{x} \rangle$$

Bool. expression $\rightarrow$ Ising

$$x_i \mapsto \frac{1 - Z_i}{2}$$

$$Z|0\rangle = +1 \quad Z|1\rangle = -1$$

$$f(\mathbf{x}) = \langle \mathbf{x} | Z_i \cdots Z_j | \mathbf{x} \rangle = \begin{cases} +1 & \text{if } x_i + \cdots + x_j = 0 \bmod 2 \\ -1 & \text{if } x_i + \cdots + x_j = 1 \bmod 2 \end{cases}$$

Boolean expr.

$$x_1 \vee x_2$$

Boolean expr.

$$x_1 \vee x_2$$

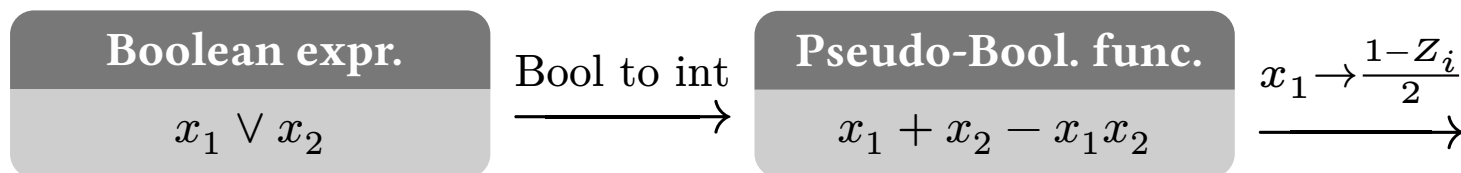
Bool to int
 $\longrightarrow$

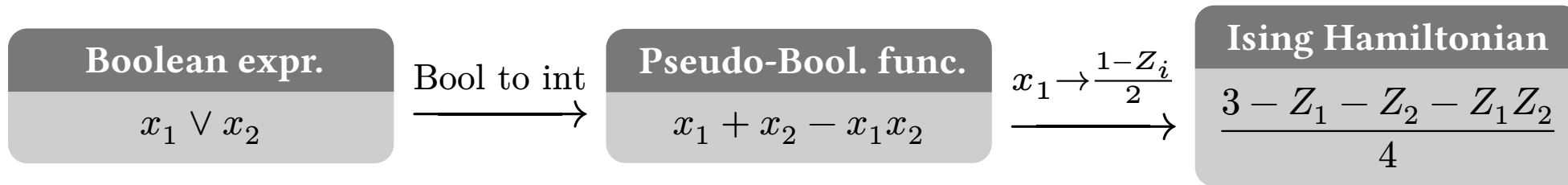
Boolean expr.

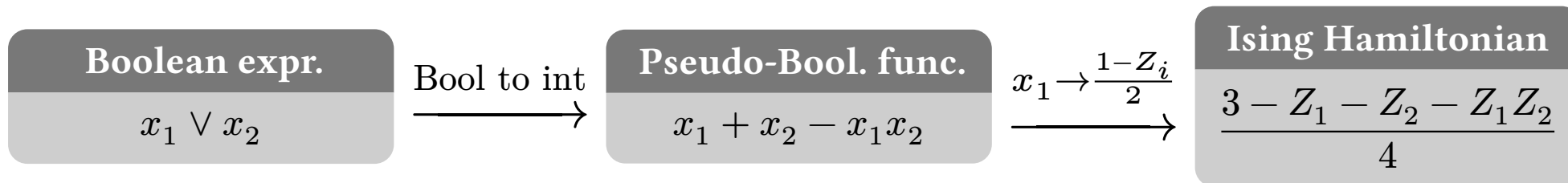
$$x_1 \vee x_2$$

Bool to int
 $\longrightarrow$ **Pseudo-Bool. func.**

$$x_1 + x_2 - x_1 x_2$$

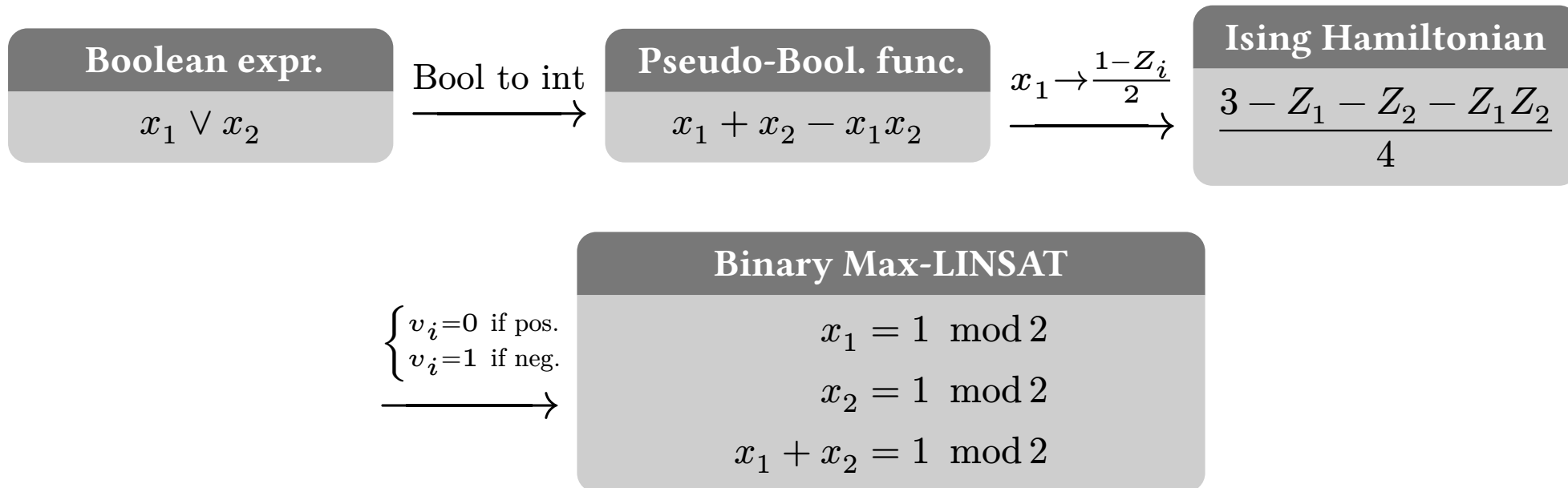






$$\begin{cases} v_i=0 & \text{if pos.} \\ v_i=1 & \text{if neg.} \end{cases}$$

$\longrightarrow$



$$x_1 \vee x_2 \vee x_3 \vee x_4$$

$$x_1 = 1 \bmod 2$$

$$x_2 = 1 \bmod 2$$

$$x_3 = 1 \bmod 2$$

$$x_4 = 1 \bmod 2$$

$$x_1 + x_2 = 1 \bmod 2$$

$$x_1 + x_3 = 1 \bmod 2$$

$$x_1 + x_4 = 1 \bmod 2$$

$$x_2 + x_3 = 1 \bmod 2$$

$$x_2 + x_4 = 1 \bmod 2$$

$$x_3 + x_4 = 1 \bmod 2$$

$$x_1 + x_2 + x_3 = 1 \bmod 2$$

$$x_1 + x_2 + x_4 = 1 \bmod 2$$

$$x_1 + x_3 + x_4 = 1 \bmod 2$$

$$x_2 + x_3 + x_4 = 1 \bmod 2$$

$$x_1 + x_2 + x_3 + x_4 = 1 \bmod 2$$

$$x_1 \vee x_2 \vee x_3 \vee x_4$$

$$x_1 = 1 \bmod 2$$

$$x_2 = 1 \bmod 2$$

$$x_3 = 1 \bmod 2$$

$$x_4 = 1 \bmod 2$$

$$x_1 + x_2 = 1 \bmod 2$$

$$x_1 + x_3 = 1 \bmod 2$$

$$x_1 + x_4 = 1 \bmod 2$$

$$x_2 + x_3 = 1 \bmod 2$$

$$x_2 + x_4 = 1 \bmod 2$$

$$x_3 + x_4 = 1 \bmod 2$$

$$x_1 + x_2 + x_3 = 1 \bmod 2$$

$$x_1 + x_2 + x_4 = 1 \bmod 2$$

$$x_1 + x_3 + x_4 = 1 \bmod 2$$

$$x_2 + x_3 + x_4 = 1 \bmod 2$$

$$x_1 + x_2 + x_3 + x_4 = 1 \bmod 2$$

Solution: auxiliary variables

$$x_1 \vee x_2 \Leftrightarrow y_1$$

$$x_3 \vee x_4 \Leftrightarrow y_2$$

$$y_1 \vee y_2$$

$$1 - ((x_1 + x_2 - x_1 x_2) - y_1)^2$$

$$1 - ((x_3 + x_4 - x_3 x_4) - y_2)^2$$

$$y_1 + y_2 - y_1 y_2$$

$$x_1 = 1 \bmod 2$$

$$x_1 y_1 = 0 \bmod 2$$

$$y_1 = 1 \bmod 2$$

$$\vdots$$

 Exercise 7

See how Boolean constraints turn into Max-XORSAT and why long clauses are expensive

Your task

1. Encode a clause $x_1 \vee x_2 \vee \dots \vee x_k$ for $k = 2, 3, 4, 5$ without a degree cap (`max_degree=k`)
2. Count the distinct Max-XORSAT constraints: how do they grow with k ?
3. Cap `max_degree`: what happens to the number of variables and constraints?

DQI-Kit

```
prob = MaxConstraintSat()
x = [prob.new_binary_var(f"x_{i}") for i in range(k)]
prob.add_boolean_constraint(x[0] | x[1] | x[2] | x[3])

xorsat = prob.to_max_linsat(max_degree=k)
len(xorsat.constraints), xorsat.get_n()
```

arXiv:2509.08328v1 [quant-ph] 10 Sep 2025

Towards solving industrial integer linear programs with Decoded Quantum Interferometry

Francesc Sabater^{1,3}, Ouns El Harzi², Geert-Jan Besjes², Marvin Erdmann¹, Johannes Klepsch¹, Jonas Hiltrop², Jean-François Bobier², Yudong Cao^{*,2}, and Carlos A. Riofrio^{†,1}

¹BMW Group, Munich, Germany

²Boston Consulting Group and BCG X AI Science Institute, Boston, USA

³Technical University of Munich, TUM School of Natural Sciences, Physics Department, 85748 Garching, Germany

September 11, 2025

Abstract

Optimization via decoded quantum interferometry (DQI) has recently gained a great deal of attention as a promising avenue for solving optimization problems using quantum computers. In this paper, we apply DQI to an industrial optimization problem in the automotive industry: the vehicle option-package pricing problem. Our main contributions are 1) formulating the industrial problem as an integer linear program (ILP), 2) converting the ILP into instances of max-XORSAT, and 3) developing a detailed quantum circuit implementation for belief propagation, a heuristic algorithm for decoding LDPC codes. Thus, we provide a full implementation of the DQI algorithm using Belief Propagation, which can be applied to any industrially relevant ILP by first transforming it into a max-XORSAT instance. We also evaluate the effectiveness of our implementation by benchmarking it against both Gurobi and a random sampling baseline.

Contents

1	Introduction	3
2	Decoded Quantum Interferometry (DQI)	4
2.1	Dicke-state preparation	5
2.2	Encoding of problem-specific phases	6
2.3	Encoding of the syndrome	6
2.4	Decoding	6
2.5	Hadamard Transform	6
2.6	Measurement and Postprocessing	7
3	Quantum implementation of a binary belief propagation decoder	7
3.1	Binary belief propagation (BP1)	7
3.2	Quantum circuit implementation	7
3.2.1	Preliminaries	7
3.2.2	Quantum circuit for the binary belief propagation algorithm	10

*cao.yudong@bcg.com

†carlos.rioerio@bmwgroup.com

Idea:

Build logical circuit implementing ILP instance

$$a \otimes b = c \longrightarrow a + b + c = 0 \bmod 2$$

$$a = \neg b \longrightarrow a + b = 1 \bmod 2$$

$$a \wedge b = c \longrightarrow \begin{cases} a + b + c = 1 \bmod 2 \\ a + c = 0 \bmod 2 \\ b + c = 0 \bmod 2 \\ c = 0 \bmod 2 \end{cases}$$

arXiv:2509.08328v1 [quant-ph] 10 Sep 2025

Towards solving industrial integer linear programs with Decoded Quantum Interferometry

Francesc Sabater^{1,3}, Ouns El Harzi², Geert-Jan Besjes², Marvin Erdmann¹, Johannes Klepsch¹, Jonas Hiltrop², Jean-François Bobier², Yudong Cao^{*,2}, and Carlos A. Riofrio^{†,1}

¹BMW Group, Munich, Germany

²Boston Consulting Group and BCG X AI Science Institute, Boston, USA

³Technical University of Munich, TUM School of Natural Sciences, Physics Department, 85748 Garching, Germany

September 11, 2025

Abstract

Optimization via decoded quantum interferometry (DQI) has recently gained a great deal of attention as a promising avenue for solving optimization problems using quantum computers. In this paper, we apply DQI to an industrial optimization problem in the automotive industry: the vehicle option-package pricing problem. Our main contributions are 1) formulating the industrial problem as an integer linear program (ILP), 2) converting the ILP into instances of max-XORSAT, and 3) developing a detailed quantum circuit implementation for belief propagation, a heuristic algorithm for decoding LDPC codes. Thus, we provide a full implementation of the DQI algorithm using Belief Propagation, which can be applied to any industrially relevant ILP by first transforming it into a max-XORSAT instance. We also evaluate the effectiveness of our implementation by benchmarking it against both Gurobi and a random sampling baseline.

Contents

1	Introduction	3
2	Decoded Quantum Interferometry (DQI)	4
2.1	Dicke-state preparation	5
2.2	Encoding of problem-specific phases	6
2.3	Encoding of the syndrome	6
2.4	Decoding	6
2.5	Hadamard Transform	6
2.6	Measurement and Postprocessing	7
3	Quantum implementation of a binary belief propagation decoder	7
3.1	Binary belief propagation (BP1)	7
3.2	Quantum circuit implementation	7
3.2.1	Preliminaries	7
3.2.2	Quantum circuit for the binary belief propagation algorithm	10

*cao.yudong@bcg.com

†carlos.rioerio@bmwgroup.com

Idea:

Build logical circuit implementing ILP instance

$$a \otimes b = c \longrightarrow a + b + c = 0 \bmod 2$$

$$a = \neg b \longrightarrow a + b = 1 \bmod 2$$

$$a \wedge b = c \longrightarrow \begin{cases} a + b + c = 1 \bmod 2 \\ a + c = 0 \bmod 2 \\ b + c = 0 \bmod 2 \\ c = 0 \bmod 2 \end{cases}$$

Problems:

- Not flexible
- Only for the binary case
- Many linear dependencies

$\Delta_{\min}$ is smallest k such that $\mathbf{H}$
has k linearly dependent columns

$\Delta_{\min}$ is smallest k such that $\mathbf{H}$
has k linearly dependent columns

$$\mathbf{H} = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

$\Delta_{\min}$ is smallest k such that H
has k linearly dependent columns

$$H = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

$$B = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$\Delta_{\min}$ is smallest k such that H
has k linearly dependent columns

$$H = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

$$B = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$x_1 + x_3 = 1$$

$$x_1 = 0$$

$$x_3 = 0$$

Common encoding patterns lead to **short linear dependencies**

Common encoding patterns lead to **short linear dependencies**

Duplicate constraints

$$x_1 + x_2 = 0 \bmod 2$$

$$x_1 + x_2 = 1 \bmod 2$$

$$(1, 1) + (1, 1) = (0, 0)$$

$$\Delta_{\min} = 2$$

Common encoding patterns lead to **short linear dependencies**

Duplicate constraints

$$x_1 + x_2 = 0 \bmod 2$$

$$x_1 + x_2 = 1 \bmod 2$$

$$(1, 1) + (1, 1) = (0, 0)$$

$$\Delta_{\min} = 2$$

AND / OR constraints

$$x_1 = 1 \bmod 2$$

$$x_2 = 1 \bmod 2$$

$$x_1 + x_2 = 0 \bmod 2$$

$$(1, 0) + (0, 1) + (1, 1) = (0, 0)$$

$$\Delta_{\min} \leq 3$$

Common encoding patterns lead to **short linear dependencies**

Duplicate constraints

$$x_1 + x_2 = 0 \bmod 2$$

$$x_1 + x_2 = 1 \bmod 2$$

$$(1, 1) + (1, 1) = (0, 0)$$

$$\Delta_{\min} = 2$$

AND / OR constraints

$$x_1 = 1 \bmod 2$$

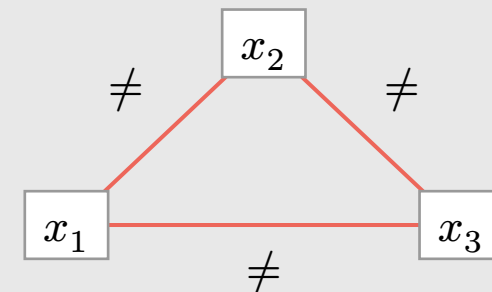
$$x_2 = 1 \bmod 2$$

$$x_1 + x_2 = 0 \bmod 2$$

$$(1, 0) + (0, 1) + (1, 1) = (0, 0)$$

$$\Delta_{\min} \leq 3$$

Short (in)equality cycles



$$\Delta_{\min} \leq 3$$

 Exercise 8

A real use case whose natural encoding has short linear dependencies

Use case

Four workers each take the **morning** (0) or the **evening** (1) shift. Alice/Bob, Bob/Carol and Alice/Carol must not share a shift, Dave works mornings, and Carol or Dave must cover the evening.

Your task

1. Encode with binary variables, convert to Max-XORSAT
2. Compute $\Delta_{\min}$, find the shortest dependent rows of B : which pattern, where from?
3. Convert with USE_STRICTEST, then drop the Alice/Carol conflict: which pattern shows up next?

DQI-Kit

```
prob = MaxConstraintSat()
a = prob.new_binary_var("alice") # b, c, d likewise
prob.add_constraint(a != b)      # ... and the others
prob.add_constraint(d == 0)
prob.add_boolean_constraint(c | d)
linsat = prob.to_max_linsat()
linsat.get_minimum_distance()
B = linsat.get_B()                # rows = constraints
B.matrix_from_rows([0, 1]).rank() # < 2: dependent
prob.to_max_linsat(
    merge_strategy=MergeStrategy.USE_STRICTEST)
```

$$x + y = 0 \bmod 5$$

$$y + z = 1 \bmod 5$$

$$z + x = 3 \bmod 5$$

$$x + y = 0 \bmod 5$$

$$y + z = 1 \bmod 5$$

$$z + x + a_1 + a_2 + a_3 + a_4 = 3 \bmod 5$$

$$x + y = 0 \bmod 5$$

$$y + z = 1 \bmod 5$$

$$z + x + a_1 + a_2 + a_3 + a_4 = 3 \bmod 5$$

$$a_1 = 0 \bmod 5$$

$$a_2 = 0 \bmod 5$$

$$a_3 = 0 \bmod 5$$

$$a_4 = 0 \bmod 5$$

$$x + y = 0 \bmod 5$$

$$y + z = 1 \bmod 5$$

$$z + x + a_1 + a_2 + a_3 + a_4 = 3 \bmod 5$$

$$a_1 = 0 \bmod 5$$

$$a_2 = 0 \bmod 5$$

$$a_3 = 0 \bmod 5$$

$$a_4 = 0 \bmod 5$$

$\Delta_{\min}$ grows by k , but so do the number of variables and constraints
 $\Rightarrow$ no gain for large instances, where DQI performance depends on ℓ/m

Anchoring: reuse pinned constraints

Pinned constraint

$p \cdot x = \tau$ at every optimum

e.g. $a_k = 0$ (last slide), one-hot rows $\sum_j x_{ij} = 1$

Anchoring

$$b \cdot x \in F$$

$$x_1 + x_2 = 0$$

$$x_1 + x_2 = 0$$

$$x_3 + x_4 = 0$$

$$x_3 + x_4 = 0$$

$$a_1 = a_2 = a_3 = 0$$

Anchoring: reuse pinned constraints

Pinned constraint

$p \cdot x = \tau$ at every optimum

e.g. $a_k = 0$ (last slide), one-hot rows $\sum_j x_{ij} = 1$

$$x_1 + x_2 = 0$$

$$x_1 + x_2 = 0$$

$$x_3 + x_4 = 0$$

$$x_3 + x_4 = 0$$

$$a_1 = a_2 = a_3 = 0$$

Anchoring

$$\left(b + \sum_k \alpha_k p_k \right) \cdot x \in F + \sum_k \alpha_k \tau_k$$

Pinned constraint

$p \cdot x = \tau$ at every optimum

e.g. $a_k = 0$ (last slide), one-hot rows $\sum_j x_{ij} = 1$

Anchoring

$$\left(b + \sum_k \alpha_k p_k \right) \cdot x \in F + \sum_k \alpha_k \tau_k$$

$$x_1 + x_2 + \mu_1(a_1 + a_2 + a_3) = 0$$

$$x_1 + x_2 + \mu_2(a_1 + a_2 + a_3) = 0$$

$$x_3 + x_4 + \mu_3(a_1 + a_2 + a_3) = 0$$

$$x_3 + x_4 + \mu_4(a_1 + a_2 + a_3) = 0$$

$$a_1 + a_2 + a_3 = 0$$

Scalar anchors,

one multiple of $a_1 + a_2 + a_3$ per row:

$$(\mu_3 - \mu_4)(b_1 - b_2) - (\mu_1 - \mu_2)(b_3 - b_4) = 0$$

$$\Rightarrow \Delta_{\min} = 4 \text{ for any } t$$

Pinned constraint

$p \cdot x = \tau$ at every optimum

e.g. $a_k = 0$ (last slide), one-hot rows $\sum_j x_{ij} = 1$

Anchoring

$$\left(b + \sum_k \alpha_k p_k \right) \cdot x \in F + \sum_k \alpha_k \tau_k$$

$$x_1 + x_2 + \gamma_1 a_1 + \gamma_1^2 a_2 + \gamma_1^3 a_3 = 0$$

$$x_1 + x_2 + \gamma_2 a_1 + \gamma_2^2 a_2 + \gamma_2^3 a_3 = 0$$

$$x_3 + x_4 + \gamma_3 a_1 + \gamma_3^2 a_2 + \gamma_3^3 a_3 = 0$$

$$x_3 + x_4 + \gamma_4 a_1 + \gamma_4^2 a_2 + \gamma_4^3 a_3 = 0$$

$$a_1 = a_2 = a_3 = 0$$

Vandermonde anchors,

one γ_r per row: $\alpha_r = (\gamma_r, \gamma_r^2, \dots, \gamma_r^t)$

$\Rightarrow \Delta_{\min} = t + 2$ (usually)

Pinned constraint

$p \cdot x = \tau$ at every optimum

e.g. $a_k = 0$ (last slide), one-hot rows $\sum_j x_{ij} = 1$

Anchoring

$$\left(b + \sum_k \alpha_k p_k \right) \cdot x \in F + \sum_k \alpha_k \tau_k$$

$$x_1 + x_2 + \gamma_1 a_1 + \gamma_1^2 a_2 + \gamma_1^3 a_3 = 0$$

$$x_1 + x_2 + \gamma_2 a_1 + \gamma_2^2 a_2 + \gamma_2^3 a_3 = 0$$

$$x_3 + x_4 + \gamma_3 a_1 + \gamma_3^2 a_2 + \gamma_3^3 a_3 = 0$$

$$x_3 + x_4 + \gamma_4 a_1 + \gamma_4^2 a_2 + \gamma_4^3 a_3 = 0$$

$$a_1 = a_2 = a_3 = 0$$

Vandermonde anchors,

one γ_r per row: $\alpha_r = (\gamma_r, \gamma_r^2, \dots, \gamma_r^t)$

$\Rightarrow \Delta_{\min} = t + 2$ (usually)

Cost:

- dense B
- increased field size p



Exercise 9

Naïve vs. scalar vs. Vandermonde anchors on the three bad patterns

Your task

1. Build the three bad patterns over $\mathbb{F}_{23}$ (two duplicated rows, an AND/OR triple, a cycle): $\Delta_{\min}$?
2. Add $t = 3$ pins $a_k = 0$. Naïve (one row per pattern) and scalar anchors $\mu_r(a_1 + a_2 + a_3)$: how far does $\Delta_{\min}$ get?
3. Vandermonde anchors with fresh γ_r : $\Delta_{\min}$ for $t = 3, 5$? Compare the DQI estimates of all variants: why does $\Delta_{\min}$ barely matter here?

DQI-Kit

```
p = 23 # safe prime: (p - 1) / 2 = 11 is prime
prob = MaxLinSat(GF(p))
x = [prob.new_var(f"x_{i}") for i in range(9)]
a = [prob.new_var(f"a_{k}") for k in range(3)]
gammas = iter(range(2, (p - 1) // 2 + 1))

g = GF(p)(next(gammas)) # fresh gamma per row
prob.add_constraint(x[0] + x[1]
    + sum(g ** (k + 1) * a[k] for k in range(3)) == 0)
for a_k in a:
    prob.add_constraint(a_k == 0) # the pins
prob.get_minimum_distance()
Dqi(prob).estimate_solution_quality(details=True)
```

How well does DQI work?

How well does DQI work?

✓ DQI on algebraic problems

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems
- ? DQI on real-world problems

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems
- ? DQI on real-world problems

DQI on real-world problems

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems
- ? DQI on real-world problems

DQI on real-world problems

- ✓ Most problems can be encoded

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems
- ? DQI on real-world problems

DQI on real-world problems

- ✓ Most problems can be encoded
- ✗ Most naïve encodings are bad

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems
- ? DQI on real-world problems

DQI on real-world problems

- ✓ Most problems can be encoded
- ✗ Most naïve encodings are bad
- ? Generalize DQI $\Rightarrow$ better encodings

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems
- ? DQI on real-world problems

DQI on real-world problems

- ✓ Most problems can be encoded
- ✗ Most naïve encodings are bad
- ? Generalize DQI $\Rightarrow$ better encodings
- ? Smarter encodings

How well does DQI work?

- ✓ DQI on algebraic problems
- ✗ DQI on unstructured/random problems
- ? DQI on real-world problems

DQI on real-world problems

- ✓ Most problems can be encoded
- ✗ Most naïve encodings are bad
- ? Generalize DQI $\Rightarrow$ better encodings
- ? Smarter encodings

DQI-Kit

github.com/lfd/qsw-2026-dqi-kit



New version later this week

Thank You!

